



**Уральский
федеральный
университет**

имени первого Президента
России Б.Н.Ельцина

**Институт радиоэлектроники
и информационных
технологий — РТФ**

В. И. СУХАНОВ

РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ НА ПЛАТФОРМЕ SPRING

Учебно-методическое пособие

Министерство науки и высшего образования
Российской Федерации

Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина

В. И. Суханов

РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ НА ПЛАТФОРМЕ SPRING

Учебно-методическое пособие

Рекомендовано методическим советом
Уральского федерального университета
для студентов вуза, обучающихся по направлениям подготовки
09.03.01, 09.04.01 — Информатика и вычислительная техника,
09.03.03, 09.04.03 — Прикладная информатика,
09.03.04, 09.04.04 — Программная инженерия

Екатеринбург
Издательство Уральского университета
2023

УДК 004.77
ББК 32.973.4
С91

Рецензенты:

д-р техн. наук, проф. В. М. Аленичев (главный научный сотрудник Института горного дела УрО РАН);

д-р техн. наук, проф. А. В. Коновалов (завотделом механики машин и технологий Института машиноведения УрО РАН)

Научный редактор — канд. техн. наук, доц. *С. И. Тимошенко*

Суханов, Владимир Иванович.

С91 Разработка веб-приложений на платформе Spring : учебно-методическое пособие / В. И. Суханов ; М-во науки и высшего образования РФ. — Екатеринбург : Изд-во Урал. ун-та, 2023. — 180 с. — ISBN 978-5-7996-3735-4. — Текст : непосредственный.

ISBN 978-5-7996-3735-4

Последовательно рассматриваются концепции современных платформ Spring, Spring Boot, Angular, популярных у профессиональных разработчиков высоконагруженных веб-приложений. Материал пособия основан на оригинальной документации разработчиков инструментальных средств, на публикациях описаний технологий программирования в интернет-ресурсах и проверенных автором примерах приложений с использованием СУБД PostgreSQL. Приведенные примеры фрагментов кода и справочные материалы позволят студентам разобраться в деталях правил использования рассматриваемых платформ, приобрести навыки разработки веб-приложений, выполнить проекты различной сложности.

Пособие восполняет недостаток учебной литературы на русском языке по описываемым платформам и рекомендовано студентам, изучающим дисциплины, связанные с разработкой веб-приложений и веб-сервисов при использовании Java EE.

Библиогр.: 12 назв. Рис. 19. Табл. 2. Прил. 2.

УДК 004.77
ББК 32.973.4

ISBN 978-5-7996-3735-4

© Уральский федеральный
университет, 2023

Список сокращений и определений

Контент	—	содержимое страницы
ООП	—	объектно-ориентированное программирование
ПК	—	персональный компьютер
СУБД	—	система управления базами данных
API	—	Application Programming Interface, программный интерфейс разработчика приложений
CSS	—	Cascade Style Sheet, таблица каскадных стилей
EIS	—	Enterprise Information System, информационная система предприятия
EL	—	Expression Language, язык выражений JSTL
GUI	—	Graphic User Interface, графический интерфейс пользователя
HTML	—	Hyper Text Markup Language, язык гипертекстовой разметки
J2EE	—	Java 2 Platform Enterprise Edition, платформа для разработки корпоративных информационных веб-приложений
JavaEE	—	Java Enterprise Edition, платформа для разработки корпоративных веб-приложений
JRE	—	Java Runtime Environment, среда исполнения программ на языке Java
JSP	—	JavaServer Pages, технология разработки страниц
JSF	—	JavaServer Faces, технология разработки страниц
JSTL	—	JavaServer Pages Standard Tag Library, стандартная библиотека тегов
POJO	—	Plain Old Java Object, простой Java-объект, не унаследованный от другого объекта и реализующий только нужные для бизнес-модели интерфейсы
SGML	—	Standard Generalized Markup Language
SPA	—	Single Page Application, одностраничное приложение
SQL	—	язык запросов к базам данных
UI	—	User Interface, интерфейс пользователя
XML	—	Extensible Markup Language, расширяемый язык разметки

Предисловие

Подготовка разработчиков программного обеспечения включает освоение целого ряда технологий и инструментальных средств для успешного применения в практической деятельности. Компьютерные технологии — довольно развитая область деятельности, имеющая разнообразное применение в информационном обществе как простейших вычислений, так и систем поддержки принятия решений с использованием искусственного интеллекта. Среди широкого спектра компьютерных технологий заметное место занимают веб-приложения, позволяющие отделить исполнительную систему и средства отображения результатов, реализовав их при помощи стандартных браузеров. Для реализации этих идей программист должен владеть полным стеком средств разработки для той или иной платформы.

Учебное пособие знакомит студента с технологиями разработки веб-приложений с использованием технологий на основе языка программирования Java. Язык Java — постоянно развивающаяся платформа, вбирающая в себя современные достижения в прикладном программировании, позволяющие уменьшить затраты на разработку и обеспечить высокое качество проекта. Платформа Spring существенно расширяет стандартные возможности объектного программирования, так как переносит на исполнительную среду большое количество функций по управлению исполнением программ, делая разработку приложений хорошо структурированным процессом.

Целью учебного пособия является описание концепций разделения функций между разрабатываемым программистом кодом и исполнительной средой при написании веб-приложений, обеспечива-

ющих работу с базами данных, минимизацию загрузки центрального сервера, дружественные интерфейсы с пользователями, что позволяет строить высоконагруженные информационные системы с использованием серийных технических и программных средств.

Автор благодарит магистров по направлению 09.04.04 «Программная инженерия» Зими́на Евге́ния Влади́мировича и Сайчи́ка Евге́ния Дми́триевича за полезное обсуждение материалов пособия, рекомендации и предоставленные фрагменты кода.

Введение

Разработка клиент-серверных приложений с веб-интерфейсом привлекает внимание программистов, администраторов и пользователей тем, что существенно упрощается как сама информационная система организации, так и ее эксплуатация. Наличие центрального сервера позволяет организовать надежное хранение данных и проверку полномочий на доступ к ним. Отсутствие специального программного обеспечения на стороне клиента минимизирует задачи развертывания и администрирования приложений. Для работы клиенту достаточно иметь компьютер с сетевым подключением и стандартный интернет-браузер. Пользователи могут выполнять свои функции с любого компьютера внутренней сети организации или, если открыт доступ, с любого компьютера, подключенного к сети интернет. Это повышает мобильность, позволяет расширить круг пользователей далеко за пределами офиса.

Существуют множество конкурирующих технологий для создания таких систем. Наиболее известные традиционные технологии используют стандартные веб серверы, например, широко распространенные Apache, Zope, IIS и т. д. На некотором поддерживаемом ими языке программирования (Perl, PHP, Python и др.) пишутся программы, генерирующие динамические страницы в ответ на запросы пользователя, в которых широко используются хранимые в СУБД данные. Всё большее внимание разработчиков привлекают Java-технологии, обладающие рядом преимуществ. Наиболее значимыми являются их надежность, обеспечиваемая сертификацией инструментов разработки мировыми лидерами программной индустрии, независимость от платформы, идеология открытого кода и их свободное распространение.

Пособие основано на официальных материалах поставщиков программного обеспечения, дополненных примерами проверенных автором программ, содержит описание инструментальных средств, используемых для разработки веб-приложений в технологии Java™ Platform Enterprise Edition (Java EE), которая определяет стандарт для разработки многоуровневых корпоративных приложений. Серверы JavaEE упрощают разработку приложений, предоставляя программистам готовые сервисы в виде стандартизованных модульных компонент. Среди большого разнообразия JavaEE серверов здесь рассматриваются Tomcat и GlassFish, простые и доступные, но обладающие хорошей функциональностью, достаточной для изучения всех аспектов технологии и разработки корпоративных информационных систем для среднего и малого бизнеса. Оригинальные описания изучаемых технологий можно найти в документации поставщиков ПО, в опубликованных материалах и интернет-ресурсах, указанных в библиографическом списке [7–10, 12].

Что надо знать, приступая к изучению этих технологий?

Во-первых, необходимо изучить программирование для веб: особенности взаимодействия сервера и клиента, как формируется, из каких частей состоит запрос, как он отправляется на сервер, в каком виде поступает ответ с сервера, язык HTML, каскадные таблицы стилей (CSS), основы языка JavaScript.

Во-вторых, необходимо знать язык программирования Java и основные элементы его технологии: среды разработки, компиляторы, основные библиотеки для обработки данных.

Во-третьих, уметь работать с СУБД, знать язык SQL создания таблиц и запросов к базам данных, необходимо иметь представление о технологии JDBC для доступа к базам данных из программ на языке Java.

Для использования данных технологий необходимо иметь программное обеспечение, желательно последних версий: JDK — основу Java технологии, среду разработки приложений на языке Java (например, IntelliJ IDEA, NetBeans, Eclipse), СУБД для работы с базами данных, драйверы JDBC для выбранной СУБД, редактор HTML кода и, наконец, сервер.

1. Платформа Spring

Платформа Spring один из самых популярных фреймворков для поддержки технологий разработки приложений [8, 12]. Она была разработана в 2003 году Родом Джонсоном. С помощью платформы Spring можно создавать любые приложения на Java. Основные преимущества использования Spring Framework при разработке Java-приложений сводятся к следующим:

- Spring позволяет программировать корпоративные приложения с помощью простых объектов, использующих стандартные контейнеры сервлетов, например Tomcat или другие серверы;
- поддерживает модульный принцип создания приложений;
- использует готовые решения и технологии ORM, JEE, JDK, MVC и другие;
- позволяет тестировать приложения;
- предоставляет API для обработки разнообразных исключений;
- использует компактные контейнеры для управления зависимостями;
- обеспечивает интерфейс управления транзакциями.

Платформа Spring основана на использовании важных понятий — **внедрение зависимостей** (Dependency Injection — DI), что порождает инверсию управления (Inversion of Control — IoC). Данные понятия позволяют при написании крупных и сложных проектов создавать независимые друг от друга классы, пригодные для повторного использования и независимого тестирования. Программная реализация DI обеспечивает связи между экземплярами этих классов, сохраняя их независимость. Все классы приложений на платформе Spring должны удовлетворять требованиям соглашений о бинах (наличие конструк-

тора без параметров и свойств, представленных полями для хранения и методами get/set для чтения и записи значений).

Основная идея DI — это внедрение экземпляра бина одного класса в экземпляр другого класса, которое выполняется путем передачи зависимых экземпляров классов через параметры конструктора зависящему классу или с помощью методов инициализации в период исполнения приложения после создания зависящего экземпляра (рис. 1).

Бины создаются контейнером на основе объявлений классов и конфигураций приложения. Созданные в нужном количестве бины помещаются в хранилище, откуда они поставляются приложению по запросу на внедрение зависимостей. Это позволяет в период исполнения не тратить время на создание экземпляров, повышая производительность приложения при обработке потока запросов.

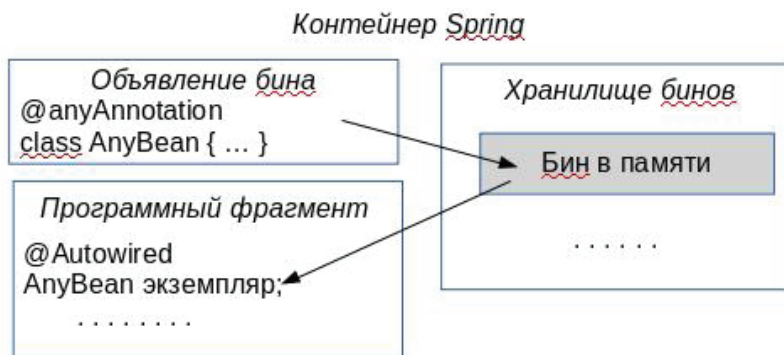
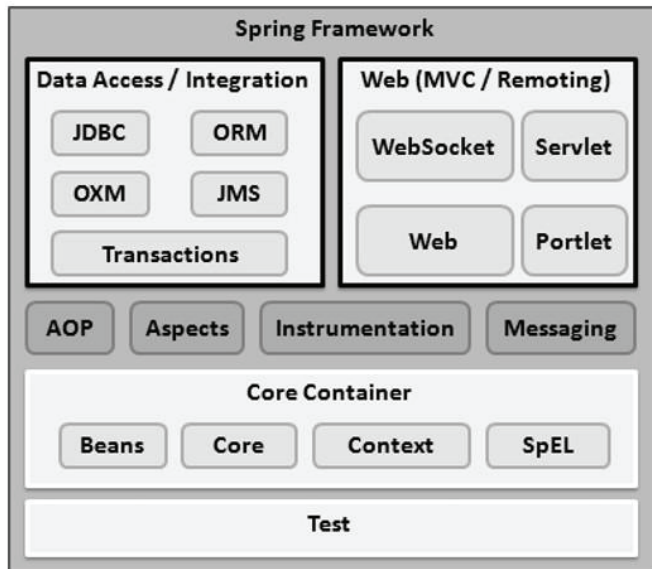


Рис. 1. Схема инъекции бина

Платформа Spring поддерживает технологию **аспекто-ориентированного программирования** (Aspect oriented programming — AOP). Данными функциями обслуживаются несколько модулей приложения и независимые от бизнес-логики приложения. В объектно-ориентированном программировании основной единицей является **класс**, а в АОП основным элементом является аспект. DI позволяет собрать приложение из отдельных классов, а АОП отделяет аспекты от объектов, на поведение которых они влияют.

Сама платформа Spring состоит из отдельных модулей, что позволяет разработчику выбирать, какие из них использовать при разработке приложения (рис. 2). Приведем краткие комментарии состава и функций модулей:

- **Основной контейнер** (Core Container) включает в себя модули Beans, Core, Context и SpEL (Spring expression language);
- Beans реализует шаблон фабрики классов BeanFactory для создания экземпляров бинов;
- модуль Core — центральная часть фреймворка, обеспечивающая функции IoC и DI;
- Context построен на основе Beans и Core и позволяет получить доступ к любому бину, который определен в конфигурации приложения, главным элементом модуля Context является интерфейс ApplicationContext;
- модуль SpEL предоставляет мощный язык Spring-выражений для манипулирования объектами во время исполнения.

Рис. 2. Архитектура платформы *Spring*

Контейнер Data Access/Integration содержит драйверы и модули JDBC, ORM, OXM, JMS, Transactions:

- JDBC обеспечивает абстрактный слой драйвера и избавляет разработчика от необходимости вручную прописывать монотонный код для взаимодействия с БД.
- ORM обеспечивает интеграцию с технологиями Hibernate, JDO, JPA и т. д.;

-
- модуль OXM предоставляет средства работы с файлами формата XML, обеспечивая связь Объект/XML — XMLBeans, JAXB и т. д.
 - модуль JMS (Java Messaging Service) выполняет создание, передачу и получение сообщений;
 - модуль Transactions поддерживает управление транзакциями для классов, которые реализуют необходимые методы.

Слой Web (MVC-Remoting) состоит из модулей WebSocket, Web, Servlet, Portlet:

- Модуль WebSocket обеспечивает поддержку связи между клиентом и сервером, используя Web-Socket для веб-приложений.
- Модуль Web выполняет загрузку, передачу файлов и т. д. Web содержит реализацию технологии «модель-вид-контроллер» Spring MVC для веб-приложений.
- Servlet — интерфейс Java, расширяющий функциональные возможности сервера в среде портлетов (компонентов пользовательского интерфейса).
- Portlet обеспечивает реализацию MVC в среде портлетов.

Кроме перечисленных выше, Spring также включает в себя ряд других важных модулей — AOP, Aspects, Instrumentation, Messaging и Test:

- AOP реализует аспекто-ориентированное программирование;
- модуль Aspects обеспечивает интеграцию с AspectJ, который также является мощным компонентом АОП;
- Instrumentation отвечает за поддержку функций class instrumentation и class loader, которые используются в серверных приложениях;
- Messaging обеспечивает поддержку получения и обработки сообщений;
- модуль Test обеспечивает тестирование с использованием инструментов TestNG или JUnit.

Один из главных принципов поддерживаемой Spring инверсии управления состоит в том, что код программы не должен зависеть от конкретных реализаций классов, а должен зависеть от абстракций, описываемых интерфейсами. Ориентация на интерфейсы — главная черта платформы Spring. Код приложения не должен знать реализацию бина, его функции. Он должен быть уверенным в том, что при запросе бина из хранилища ему будет выдана реализация с запрашиваемым интерфейсом. Это важное отличие от жесткого связывания эк-

земпляров классов в классическом объектно-ориентированном программировании.

Рассмотрим пример. Пусть у нас есть класс `Writer`, который реализует вывод на консоль приложения. Класс `Console` представляет собой простую реализацию:

```
public class Console {  
    public void writeText(String text) {  
        // вывод на консоль значения параметра text  
    }  
}
```

Для выполнения печати класс `Writer` использует консоль:

```
public class Writer {  
    private Console console;  
    public Writer() {  
        console = new Console();  
    }  
    public void writePoem() {  
        console.writeText("Платформа Spring - лучшая для Java");  
    }  
}
```

Как следует из текста, экземпляр `Console` создается непосредственно при инициализации экземпляра класса `Writer`, и мы имеем жесткую зависимость между `Writer` и конкретной реализацией `Console`. Для простых проектов это не создаст проблем с пониманием, написанием и тестированием кода, но для проектов высокой сложности появляются определенные трудности. Например, если мы решаем, чтобы вместо консоли писатель выдавал нам текст на принтер, то в текущей реализации это означает изменение класса `Writer` в нескольких местах, что является несложной операцией для простых проектов. Однако в более крупных проектах такие простые изменения становятся серьезной проблемой. На данный момент изменение требуется в нескольких местах.

Как можно решить возникшую проблему, чтобы `Writer` умел выводить текст не только в консоль, но и на принтер? Решение достаточно простое. Для этого определим интерфейс, который будет позволять выводить текст в любое устройство: консоль, принтер и т. д. Пусть имя интерфейса будет `TextOutput`:

```
public interface TextOutput {
    void writeText(String text);
}
```

Если консоль умеет выводить текст, то она реализует данный интерфейс. Исходный код консоли изменится:

```
public class Console implements TextOutput{
    @Override
    public void writeText(String text) {
        // печать текста на консоли
    }
}
```

Для того, чтобы текст можно было выводить еще и на принтер, класс `Printer` также должен реализовывать данный интерфейс:

```
public class Printer implements TextOutput {
    @Override
    public void writeText(String text) {
        // печать текста на принтере
    }
}
```

Следовательно, при в такой реализации класс `Writer` должен иметь в себе не конкретную реализацию механизма вывода текста (консоль или принтер), а нечто более абстрактное, чтобы можно было бы изменять то место, куда сделать вывод текста — экземпляр класса, поддерживающий интерфейс `TextOutput`. Обновленный класс `Writer` представляет собой следующий код:

```
public class Writer {
    private TextOutput output;
    public Writer() {
        output = new Console();
    }
    public void writePlus() {
        output.writeText("Платформа Spring - лучшая для Java");
    }
}
```

Выше был использован стандартный прием полиморфизма в ООП. Следовательно, если мы хотим вместо консоли выводить текст на принтер, то достаточно поменять код только в одном месте — в конструкторе класса `Writer`. В классической реализации ООП для подобного примера потребовались бы более серьезные изменения класса `Writer`.

В приведенных примерах проиллюстрирован принцип **инверсии управления**, где класс `Writer` сначала жестко зависел от экземпляра класса `Console`, а после класс `Writer` стал зависеть от абстракции.

В паре с принципом инверсии управления, который мы рассмотрели выше, работает принцип **внедрения зависимостей**, который логично продолжает инверсию управления. Его можно сформулировать таким образом: *вызывающий код ничего не должен знать о конкретных реализациях зависимостей, которые он вызывает*. В ранее приведенном примере класса `Writer` объектом, от которого зависит наш код (зависимость), является `output`, а конкретной реализацией экземпляра данной зависимости, которая создается в конструкторе, является `Console`. Приведенный выше код класса не удовлетворяет принципу внедрения зависимостей из-за явного создания экземпляра консоли в конструкторе. Простой способ согласовать код `Writer` с принципом внедрения зависимостей — следующая модификация:

```
public class Writer {
    private TextOutput output;
    public Writer(TextOutput output) {
        this.output = output;
    }
    public void writePoem() {
        this.output.writeText("Мой дядя самых честных правил");
    }
}
```

В этом случае конкретную реализацию `output` мы передаем в конструктор класса с помощью параметра. Возникает ситуация, когда класс `Writer` не знает, куда именно он будет писать — в консоль или на принтер. Главное, чтобы объект реализовывал интерфейс `TextOutput`. Таким образом, мы получаем код, который зависит от абстракций.

1.1. Контейнеры Spring.

Область видимости и жизненный цикл бинов

Главным элементом Spring Framework является Spring Container, выполняющий всю работу по организации функционирования приложения. Контейнер создает экземпляры объектов, связывает их вместе, настраивает и управляет ими от создания до момента уничтожения (см. рис. 1). Для управления компонентами приложения Spring Container использует **внедрение зависимостей (DI)**. Используемые в приложении компоненты являются бинами Spring Beans. Контейнер Spring Container получает инструкции о том, какие объекты создавать и как их настраивать через конфигурационные файлы или аннотации, называемые **метаданными**.

В Spring имеются два различных вида контейнеров:

- *Spring BeanFactory Container*;
- *Spring ApplicationContext Container*.

Spring BeanFactory Container — самый простой контейнер обеспечивает базовую поддержку DI, основанный на интерфейсе `org.springframework.beans.factory.BeanFactory`.

Spring ApplicationContext Container — более сложный и более продвинутый контейнер Spring: так же, как BeanFactory, ApplicationContext загружает бины, связывает их вместе и настраивает их заданным образом. Кроме того, ApplicationContext обладает дополнительной функциональностью: распознавание текстовых сообщений из файлов настройки и отображение событий, которые происходят в приложении различными способами. Данный контейнер определяется интерфейсом `org.springframework.context.ApplicationContext`.

Бины — это объекты, которые являются базовыми кирпичиками приложения и управляются Spring-контейнером. Бины создаются контейнером на основе конфигурационных метаданных. Определение бина содержит метаданные конфигурации, которые необходимы контейнеру для получения следующей информации:

- создание бина;
- жизненный цикл бина;
- зависимости бина.

Когда определяется bean в контейнере Framework, у нас есть возможность объявить свойство scope — область видимости этого ком-

понента. Контейнер Framework содержит пять возможных значений свойства `scope`:

- `singleton` — определяет один единственный экземпляр бина для каждого контейнера Spring IoC (используется по умолчанию);
- `prototype` — позволяет иметь любое количество экземпляров бина;
- `request` — создается один экземпляр бина на каждый `http`-запрос, используется исключительно в контейнере `ApplicationContext`;
- `session` — создается один экземпляр бина на каждую `http`-сессию, используется исключительно в контейнере `ApplicationContext`;
- `global-session` — создается один экземпляр бина на каждую глобальную `http`-сессию. Используется исключительно в контейнере `ApplicationContext`.

Для значения свойства `scope` равного `singleton` контейнер Spring IoC создает только один экземпляр объекта указанного бина, который помещается в хранилище таких же бинов (синглетонов), и все последующие вызовы бина с таким именем будут возвращать объект из этого хранилища. По умолчанию область видимости бина устанавливается `singleton`.

Для значения свойства `scope`, равного `prototype`, контейнер Spring IoC создает новый экземпляр бина на каждый полученный запрос.

Жизненный цикл бинов включает несколько простых операций. После создания экземпляра бина, могут понадобиться некоторые действия для того, чтобы сделать его работоспособным. При удалении бина из хранилища контейнера, необходима очистка памяти. Для управления созданием и уничтожением бина существуют методы `init` и `destroy`.

Для того чтобы вызвать функцию инициализации, интерфейс `org.springframework.beans.factory.InitializingBean` использует метод `void afterPropertiesSet () throws Exception`. Если бин конфигурируется с помощью XML-файла, то можно использовать атрибут `init-method` для задания, что будет сделано при создании экземпляра бина.

Для того чтобы вызвать функцию деструктора (уничтожения), интерфейс `org.springframework.beans.factory.DisposableBean` использует один единственный метод `void destroy () throws Exception`.

Интерфейс `BeanPostProcessor` использует два метода:

- `postProcessBeforeInitialization`;
- `postProcessAfterInitialization`.

Интерфейс `BeanPostProcessor` позволяет разработчику самому реализовывать некоторые методы бинов перед инициализацией и после уничтожения экземпляров бина. Имеется возможность настраивать несколько реализаций `BeanPostProcessor` и определить порядок их выполнения. Данный интерфейс работает с экземплярами бинов, а это означает, что контейнер Spring IoC создает экземпляр бина, а затем с ним работает `BeanPostProcessor`.

`ApplicationContext` автоматически обнаруживает любые бины, имеющие реализацию `BeanPostProcessor`, и помечает их как “post-processors” для того, чтобы создать их определенным способом.

Spring Framework позволяет осуществлять автоматическое связывание бинов (autowiring) и, как следствие, существенно сокращать объем кода в конфигурационном XML-файле, что особенно полезно при написании больших приложений.

Существует несколько режимов автоматического связывания в Spring:

- (пусто) — данный режим установлен по умолчанию и означает отсутствие автоматического связывания, т.е. для связывания необходимо использовать прямую ссылку на бин;
- `byName` — используется автоматическое связывание по имени бина. Контейнер Spring ищет бин с указанным именем, а если находит бин с таким же именем — производит автоматическое связывание;
- `byType` — автоматическое связывание по типу, т.е. контейнер Spring ищет совпадение по типу. В случае, если он его находит, то происходит автоматическое связывание;
- `constructor` — здесь все происходит так же, как и при использовании режима ‘byType’, с тем отличием, что поиск идет по аргументам конструктора;
- `autodetect` — в этом режиме сначала происходит связывание в режиме ‘constructor’, а затем (если автосвязывания не произошло) в режиме ‘byType’.

1.2. Конфигурирование с помощью аннотаций и кода Java

Первоначально конфигурация в *Spring* была доступна исключительно с помощью XML-файла, но начиная с версии Spring 2.5 стала возможна конфигурация с помощью аннотаций (доступны аннотации к классам, методам и полям). Благодаря этому мы можем связывать между собой бины, вставив аннотации непосредственно в *Java*-код.

Существует несколько видов базовых аннотаций:

- *@Required* может применяться к методам установки значений — сеттерам;
- *@Autowired* может применяться к сеттерам, обычным методам, конструкторам и свойствам;
- *@Qualifier* используется вместе с аннотациями *Autowired*, когда возможна неоднозначность при связывании (непонятно, с каким бином необходимо связывать), и определяет конкретный бин.

При конфигурировании с помощью аннотаций Java основными являются аннотации *@Configuration* и *@Bean*.

Аннотация *@Configuration*, помещенная перед классом, определяет класс для использования контейнером Spring IoC как конфигурационный класс для бинов.

Аннотация *@Bean*, помещенная перед методом, информирует Spring о том, что возвращаемый данным методом объект должен быть зарегистрирован как бин.

Контрольные вопросы

1. Внедрение зависимостей выполняется компилятором языка Java или контейнером исполнительной среды (сервером)?
2. Внедряемый бин создается в момент исполнения запроса или при запуске приложения на исполнение?
3. Нужны ли конфигурационные XML-файлы при использовании аннотаций и конфигурировании в коде Java?

2. Фреймворк Spring-MVC

Фреймворк Spring Web MVC [7] предоставляет архитектуру Model-View-Controller (MVC) и готовые компоненты, которые могут использоваться для разработки гибких и слабо связанных между этими элементами веб-приложений:

- модель инкапсулирует данные приложения и состоит из POJO;
- представление отвечает за отрисовку данных модели и генерирует выходные страницы HTML, которые может интерпретировать браузер клиента;
- контроллер отвечает за обработку запросов пользователей и построение модели, передает ее в представление для отрисовки.

2.1. DispatcherServlet

Платформа Spring MVC разработана на основе сервлета DispatcherServlet, который обрабатывает все http-запросы клиентов и формирует на них ответы (рис. 3). В Spring используются и другие сервлеты, например, для платформы JSF используется FacesServlet.

Последовательность событий, соответствующих входящему запросу http для DispatcherServlet содержит следующие шаги:

- после получения http-запроса DispatcherServlet запрашивает модуль HandlerMapping, чтобы определить соответствующий контроллер;
- Controller принимает запрос и вызывает соответствующие методы обслуживания на основе используемого в запросе метода

GET или POST. Метод контроллера `service` устанавливает данные модели на основе определенной бизнес-логики и возвращает имя представления в `DispatcherServlet`;

- `DispatcherServlet` использует информацию от `ViewResolver` при назначении представления для формирования ответа;
- как только будет создано представление, `DispatcherServlet` передает модель данных в вид, который будет отображен в браузере.

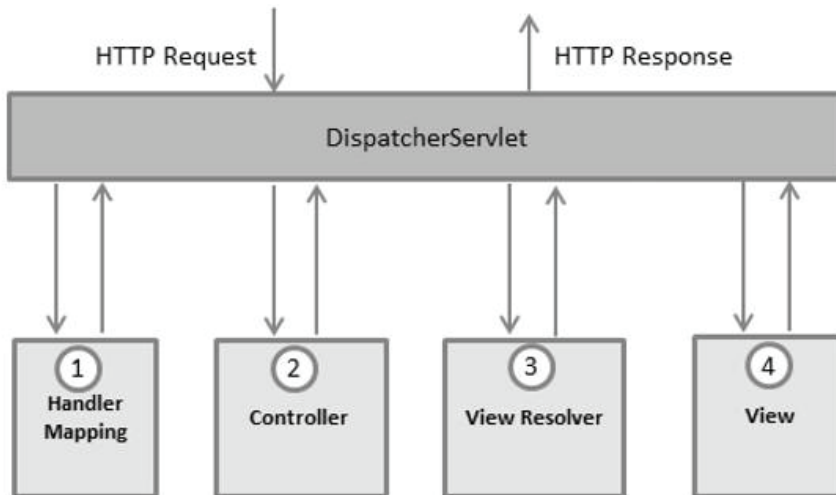


Рис. 3. Обработка запроса в Spring MVC

Все вышеупомянутые компоненты `HandlerMapping`, `Controller` и `ViewResolver` являются частями `WebApplicationContext`, который является расширением `plainApplicationContext` с некоторыми дополнительными функциями, необходимыми для веб-приложений.

2.2. Разработка контроллера

`DispatcherServlet` передает запрос контроллерам для выполнения определенных функций. Аннотация `@Controller` для класса указывает, что данный класс выступает в роли контроллера. Аннотация `@RequestMapping` используется для сопоставления обработчиком URL-адреса с классом или методом.

```

@Controller
@RequestMapping("/")
public class DeveloperController {
    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String index() {
        return "index";
    }
    @RequestMapping(value = "developer", method = RequestMethod.GET)
    public ModelAndView developer() {
        return new ModelAndView("developer", "command", new Developer());
    }
    @RequestMapping(value = "/addDeveloper", method = RequestMethod.POST)
    public String addStudent(@ModelAttribute("mvc-dispatcher")
        Developer developer,
                               ModelMap model) {
        model.addAttribute("id", developer.getId());
        model.addAttribute("name", developer.getName());
        model.addAttribute("specilaty", developer.getSpecialty());
        model.addAttribute("experience", developer.getExperience());
        return "result";
    }
}

```

Аннотация `@Controller` определяет класс как контроллер MVC. Далее первое использование `@RequestMapping` указывает на то, что все методы обработки в этом контроллере относятся в запросе к пути `«/»`. Аннотация `@RequestMapping` используется для объявления метода `developer()` как метода контроллера по умолчанию для обработки http-запросов GET. Можно аналогично определить другой метод для обработки запросов POST по тому же URL-адресу.

Вышеприведенный контроллер можно написать в другой форме, где можно использовать дополнительные атрибуты в `@RequestMapping` следующим образом:

```

@Controller
public class DeveloperController {
    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String index() {
        return "index";
    }
}

```

Атрибут `value` указывает адрес URL, на который метод назначен как обработчик, а атрибут `method` определяет ориентацию на обработ-

ку http-запроса GET. Следующие важные моменты нужно упомянуть для контроллера, определенного выше:

- бизнес-логика определяется внутри обрабатывающего метода, при необходимости можно вызывать другие методы внутри этого метода;
- на основе бизнес-логики можно пополнять модель внутри этого метода, а также добавлять различные атрибуты модели, которые будут доступны по именам свойств в представлении конечного результата (страницы в файлах JSP, в примере выше создаются атрибуты модели (пары «ключ-значение»));
- определение метода обработчика может возвращать строку, содержащую имя представления (файл JSP), которое будет использоваться для визуализации (навигация по страницам, в примере возвращается 'index' как логическое имя следующего отображаемого представления).

2.3. Создание просмотра

Spring MVC поддерживает множество типов представлений для различных технологий. К ним относятся страницы JSP, HTML, файлы в формате PDF, электронные таблицы Excel, XML, шаблоны, XSLT, в формате JSON, Atom и RSS-каналы, JasperReports и т. д. Но чаще всего используются шаблоны JSP, написанные с использованием языка JSTL.

Например, текст ссылки 'Developer new' на странице /WEB-INF/pages/index.jsp для перехода на страницу "/developer" будет оформлен следующими тегами:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
  <title>Home Page</title>
</head>
<body>
<h3><a href="/developer"> Developer new </a> </h3>
</body>
</html>
```


Здесь можно использовать любые атрибуты модели, установленные внутри контроллера, которые будут отображаться в представлении, например `${name}`.

2.4. Разработка Spring MVC веб-приложений в IntelliJ IDEA

Данный материал дает представление об основах использования среды IntelliJ IDEA для разработки веб-приложений [9]. Он демонстрирует создание простого приложения Spring MVC с возможностью передачи данных из формы, их обработки и отображения.

В проекте есть представления `index.jsp`, `developer.jsp` и `result.jsp`, имеется один контроллер `DeveloperController` и модель `Developer`.

После создания нового Maven-проекта получаем структуру, показанную на рис. 4. При появлении дополнительного диалога следует разрешить автоматическую загрузку Maven-пакетов.

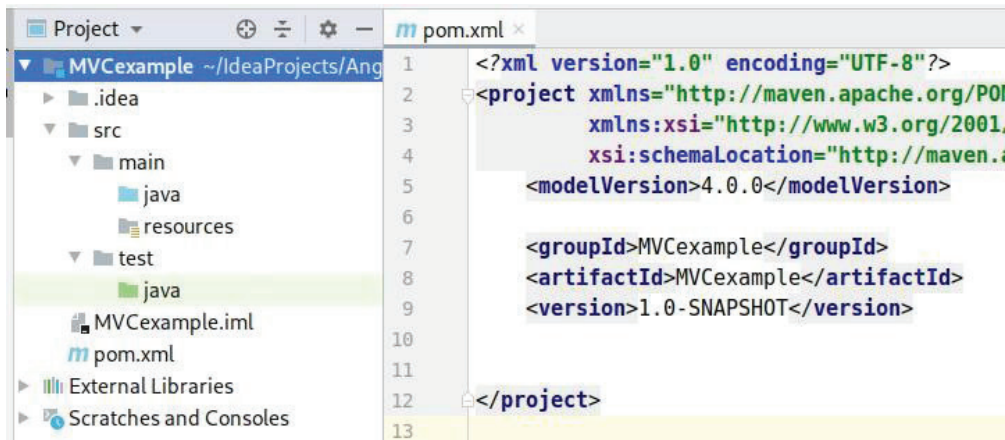


Рис. 4. Состав папок проекта

Добавляем поддержку фреймворков Spring MVC: правый клик по проекту —> Add Framework Support. Помечаем Spring и Spring MVC (рис. 5).

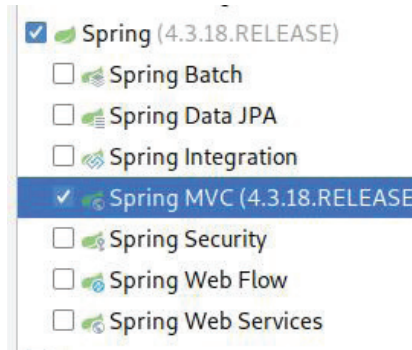


Рис. 5. Выбор фреймворка

Настраиваем файл конфигураций проекта `pom.xml`. Шаблоны нужных зависимостей можно найти на сервере репозитория Maven по адресу: <http://www.mvnrepository.com/artifact/org.springframework>.

Файл `pom.xml` имеет следующее содержание:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://maven.apache.org/POM/4.0.0"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>springMVCId</groupId>
  <artifactId>springMVC</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>springMVC demo</name>
  <description>Demo project for Spring Boot</description>
  <properties>
    <!-- Generic properties -->
    <java.version>1.8</java.version>
    <!-- Web -->
    <jsp.version>2.2</jsp.version>
    <jstl.version>1.2</jstl.version>
    <servlet.version>3.1.0</servlet.version>
    <!-- Spring -->
    <spring-framework.version>4.2.4.RELEASE</spring-
      framework.version>
  </properties>
  <dependencyManagement>
    <!--all spring dependencies -->
    <dependencies>
```

```

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-framework-bom</artifactId>
            <version>${spring-framework.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
<!--bootstrap webjars.org-->
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
    </dependency>
    <!-- Spring MVC -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
    </dependency>
    <!-- Other Servlet Web dependencies -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>jstl</artifactId>
        <version>${jstl.version}</version>
    </dependency>
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>javax.servlet-api</artifactId>
        <version>${servlet.version}</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>javax.servlet.jsp</groupId>
        <artifactId>jsp-api</artifactId>
        <version>${jsp.version}</version>
        <scope>provided</scope>
    </dependency>
</dependencies>
</project>

```

Спустя некоторое время загрузятся необходимые библиотеки поддержки выбранных ранее технологий. Пакеты сохраняются в локальном репозитории пользователя, расположенном в скрытой папке '~/m2' (рис. 6).

Домашняя папка / .m2 / repository / org / springframework ▾			
папка	Имя ▾	Размер	Дата
	boot	49 объектов	
	data	9 объектов	
	hateoas	1 объект	
	integration	2 объекта	
	org.springframework.web	1 объект	
	plugin	2 объекта	
ия	retry	1 объект	

Рис. 6. Папка с пакетами фреймворка

Файл `dispatcher-servlet.xml` содержит всю конфигурацию для Spring MVC. В нем определяются такие бины, как `ViewHandlerResolvers`, `ConverterFactories`, `Interceptors` и т. д. Все эти компоненты являются частью платформы Spring MVC, которая структурирует обработку веб-запросов, предоставляя такие полезные функции, как привязки данных, управление просмотрами и сопоставление запросов:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-
    context.xsd">
  <context:component-scan base-
    package="net.proselyte.springmvc"/>
  <bean class="org.springframework.web.servlet.view.InternalResource
    ViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
  </bean>
</beans>
```

Файл `application-context.xml` может дополнительно включаться при использовании Spring MVC или любой другой платформы. Он создает контейнер, который может использоваться для настройки других типов бинов Spring, которые обеспечивают поддержку таких функций, как сохранение данных. В принципе, в этом конфигурационном файле вы привлекаете все другие возможности Spring. Именно в контексте приложения можно настроить все проблемы, не связанные с веб:

- безопасность;
- хранение данных;
- планирование заданий и др.

Ссылки на файлы конфигурации указываются в файле `web.xml`. Пример типичного файла настройки файл `WEB-INF/web.xml` и необходимых классов приведен ниже:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">
  <display-name>Spring MVC</display-name>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/dispatcher-servlet.xml</param-value>
  </context-param>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>
      org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

Как видно из рис. 7, ресурсы являются лишь небольшой частью общей конфигурации веб-приложения Spring. Большинство веб-приложений имеют много задач, которые лежат вне конфигурации сервлета диспетчера и требуют полномасштабной загрузки кон-

текста приложения через параметр `org.springframework.web.context.ContextLoaderListener` в `web.xml`.

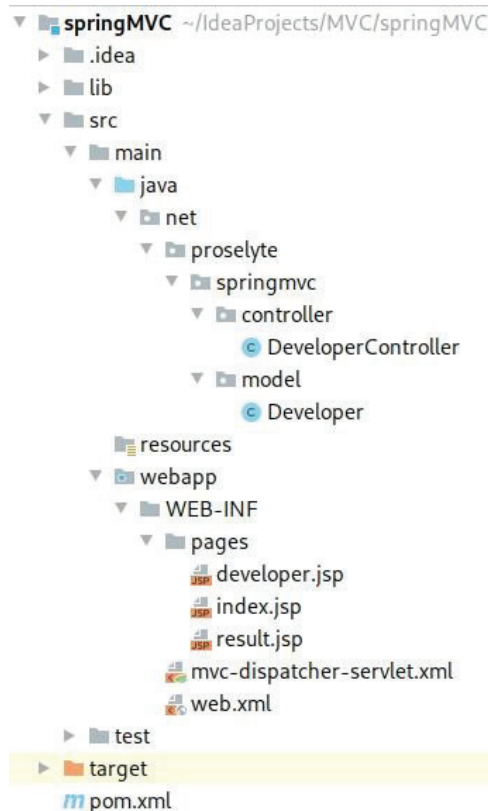


Рис. 7. Структура папок веб-приложения

Настройка конфигурационного файла `dispatcher-servlet.xml` содержит следующий текст:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns="http://www.springframework.org/<?xml version="1.0"
  encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
schema/beans"
xmlns:context="http://www.springframework.org/schema/context"
  xmlns="http://www.springframework.org/schema/beans"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```

    http://www.springframework.org/schema/beans/spring-
    beans.xsd http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-
    context.xsd">
<context:component-scan base-package="net.proselyte.Springmvc" />
<bean class="org.springframework.web.servlet.view.
    InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
</bean>
</beans>

```

Приведем файл DeveloperController.java:

```

package net.proselyte.springmvc.controller;
import net.proselyte.springmvc.model.Developer;
import org.springframework.stereotype.Controller;
import org.springframework.ui.ModelMap;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.servlet.ModelAndView;
@Controller
@RequestMapping("/")
public class DeveloperController {
    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String index() {
        return "index";
    }
    @RequestMapping(value = "developer", method = RequestMethod.GET)
    public ModelAndView developer() {
        return new ModelAndView("developer", "command", new Developer());
    }
    @RequestMapping(value = "/addDeveloper", method =
        RequestMethod.POST)
    public String addStudent(@ModelAttribute("mvc-dispatcher")
        Developer developer, ModelMap model) {
        model.addAttribute("id", developer.getId());
        model.addAttribute("name", developer.getName());
        model.addAttribute("specilaty", developer.getSpecialty());
        model.addAttribute("experience", developer.getExperience());
        return "result";
    }
}

```

При запуске приложения управление попадет в метод `index()`, так как у него стоит параметр в `@RequestMapping value = «/»` и указан метод GET. Поскольку не был указан какой-либо другой путь при разворачивании приложения (рис. 8), будет использоваться путь `«/»` по умолчанию.

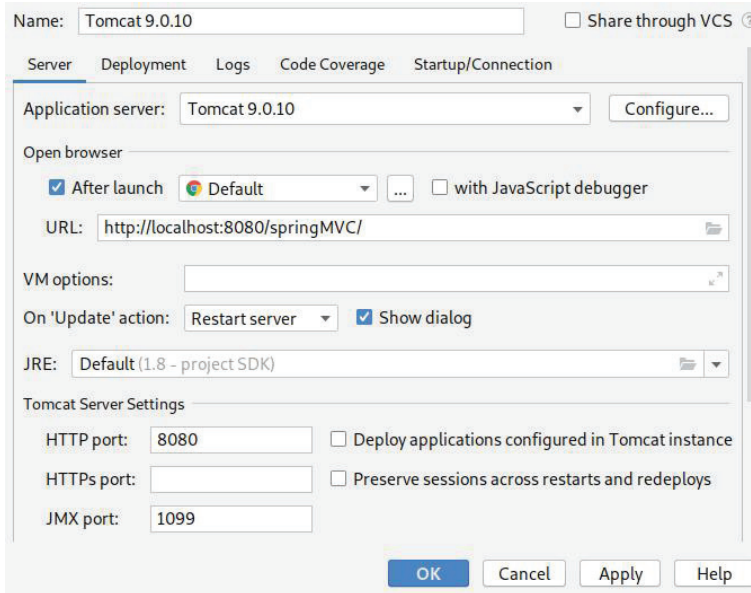


Рис. 8. Панель настройки сервера

При выборе пользователем ссылки на создание персоны управление попадает на метод `developer()`, который создает экземпляр класса `Developer` и добавляет его в модель приложения. Пользователь с помощью ссылки активирует форму `'addDeveloper.jsp'`.

При нажатии пользователем кнопки создания персоны управление попадает в метод `addStudent (@ModelAttribute («mvc-dispatcher»)) Developer developer, ModelMap model`. Здесь в модель заносятся полученные из формы атрибуты и устанавливается следующий маршрут навигации — страница `'result'`.

Модель (файл `Developer.java`) в проекте представлена простейшим классом, хранящим поля `name`, `specialty` и `experience`.

Приведем файл `Developer.java`:

```
package net.proselyte.springmvc.model;
public class Developer {
    private int id;
```



```

    private String name;
    private String specialty;
    private int experience;
    public int getId() {
        return id;
    }
// здесь размещаются геттеры и сеттеры для свойств
@Override
    public String toString() {
        return "Developer( name = " + name + " specialty = " +
            specialty + " experience = " + experience + ")";
    }
}

```

Представления содержатся в следующих файлах:

- файл **index.jsp**

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
    <title>Home Page</title>
</head>
<body>
<h3><a href="/developer"> Developer new</a> </h3>
</body>
</html>

```

- файл **developer.jsp**

```

<%@taglib uri="http://www.springframework.org/tags/form"
    prefix="form" %>
<html>
<head>
    <title>Developer</title>
</head>
<body>
<h2>Задайте информацию о разработчике</h2>
<form:form method="POST" action="addDeveloper">
    <table>
        <tr>
            <td><form:label path="id">Id</form:label></td>
            <td><form:input path="id"/></td>
        </tr>
        <tr>

```

```

        <td><form:label path="name">Name</form:label></td>
        <td><form:input path="name"/></td>
    </tr>
    <tr>
        <td><form:label path="specialty">Specialty</form:label></td>
        <td><form:input path="specialty"/></td>
    </tr>
    <tr>
        <td><form:label path="experience">experience</form:label></td>
        <td><form:input path="experience"/></td>
    </tr>
    <tr>
        <td colspan="2">
            <input type="submit" value="Submit"/>
        </td>
    </tr>
</table>
</form:form>
</body>
</html>

```

Атрибут `path=«имя»` определяет ключ данной модели, с которым связан тег при отображении этикетки или поля ввода формы.

- файл **result.jsp**

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
<html>
<head>
    <title>Developer Info</title>
</head>
<body>
<h2>Developer Information</h2>
<table>
    <tr> <td>Id</td>    <td>${id}</td>    </tr>
    <tr> <td>Name</td> <td>${name}</td> </tr>
    <tr> <td>Specialty</td> <td>${specilaty}</td> </tr>
    <tr> <td>Experience</td> <td>${experience}</td>    </tr>
</table>
</body>
</html>

```

Здесь выражения языка SpEL вида `${имя}` задают значения атрибутов модели данных с указанными именами, которые вставляются в страницу для просмотра пользователем в браузере.

Для ручного контроля состояния проекта можно использовать автоматически созданный файл настроек проекта `springMVC.iml`.

Приложение разворачивается на сервере и просматривается в браузере. На период отладки приложения целесообразно использовать популярный легковесный полнофункциональный сервер Tomcat. Панель настройки сервера приведена на рис. 8, а фрагмент вкладки разворачивания — на рис. 9.

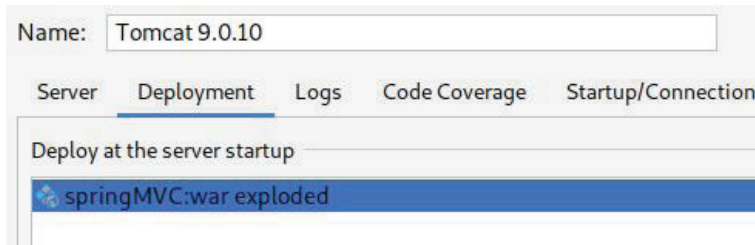


Рис. 9. Фрагмент вкладки разворачивания приложения

Контрольные вопросы

1. Платформа Spring MVC предоставляет язык программирования или средства создания веб-приложений?
2. Сервлет `DispatcherServlet` пишется прикладным программистом?
3. Контроллеры являются составной частью сервлета `DispatcherServlet` или внешними бинами?
4. Обработка запросов пользователя выполняется контейнером?
5. Приложение Spring MVC исполняется из архива JAR?

3. Фреймворк Spring Boot

3.1. Введение в Spring Boot

Платформа Spring Boot расширяет концепции платформы Spring и позволяет упростить создание автономных веб-приложений, готовых для запуска на платформе Java. Платформа Spring Boot и сторонние библиотеки позволяют вести разработку веб-приложений с минимальными усилиями. Большинство приложений Spring Boot не нуждаются в дополнительной конфигурации в виде XML-файлов, что существенно снижает трудоемкость разработки и повышает качество проекта. Spring Boot наследует от Spring все возможности и концепции с одним большим преимуществом — он создает контейнер в виде файла JAR (его обычно зовут fat jar — толстый jar), в который он внедряет абсолютно все необходимые компоненты, в том числе и сам сервер, для того чтобы приложение могло работать самостоятельно и независимо от программной платформы исполнительного ПК [5].

Например, контейнер может содержать внутри собственный сервер (Tomcat, Netty, Jetty и т. п.), чтобы обслуживать трафик, бизнес-логику и коннекторы для баз данных, брокеров сообщений (Kafka, например) и многое другое. Spring Boot можно использовать для создания приложений Java, которые запускаются из архива jar без традиционного развертывания на целевом сервере из архива war.

Основные цели платформы:

- обеспечить быстрый сценарий разработки;
- предоставить достаточный инструмент «из коробки» для быстрой разработки проекта;

- предоставить ряд средств, которые являются общими для большинства проектов (например, встроенные серверы, безопасность, метрики, проверка работоспособности, конфигурации);
- полное отсутствие генерированного платформой кода и требований конфигурирования с использованием файлов XML.

Управление зависимостями. Каждый выпуск Spring Boot имеет предопределенный список зависимостей, которые он поддерживает. На практике не нужно указывать версию для этих зависимостей в файле конфигурации `pom`, поскольку Spring Boot управляет этим сам. При обновлении самой Spring Boot данные зависимости будут постоянно обновляться. Если это необходимо, по-прежнему можете указывать версию и переопределять рекомендации Spring Boot.

Список содержит все модули Spring, которые можно использовать, а также уточненный список сторонних библиотек. Список доступен в виде стандартных спецификаций (зависимости), а также доступна дополнительная поддержка Maven и Gradle. Каждый выпуск Spring Boot связан с базовой версией Spring Framework, поэтому настоятельно рекомендуется не указывать его версию самостоятельно.

Для сборки проектов Spring Boot использует Maven. При настройке проект необходимо наследовать от стартера `spring-boot-starter-parent`, для чего в файле `pom.xml` просто прописывается родительский стартер:

```
<!-- Inherit defaults from Spring Boot -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>X.Y.Z.RELEASE</version>
</parent>
```

Если необходимо импортировать дополнительные стартеры, то можно спокойно опускать номер версии. С помощью настройки можно также переопределить отдельные зависимости, задав свойства в собственном проекте. Например, чтобы перейти на другой релиз Spring Data, можно добавить следующий фрагмент в ваш `pom.xml`.

```
<properties>
  <spring-data-releasetrain.version>Fowler-SR2</spring-data-
    releasetrain.version>
</properties>
```

Управление зависимостями позволяет авторам проекта напрямую указывать версии артефактов, которые будут использоваться, когда они встречаются в зависимостях, являющихся транзитивными, или в зависимостях, в которых версия не указана. Секция управления зависимостями (dependencyManagement) является механизмом централизации информации о зависимостях. Когда у вас есть набор проектов, которые наследуют общий родитель, можно поместить всю информацию о зависимости в общий POM и иметь более простые ссылки на артефакты в дочерних POM.

Spring Boot включает в себя плагин maven, который будет упаковывать проект в исполняемый архив. Добавьте плагин в раздел <plugins>, если вы хотите его использовать:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Стартеры — это набор удобных определений зависимостей, которые можно включить в приложение. В результате этого можно получить одну обстановку для всего приложения Spring и связанных с ним технологий, которые вам нужны, без необходимости поиска шаблонного кода и копирования дескрипторов зависимостей. Например, если необходимо начать работу с использованием Spring и JPA для доступа к базе данных, то включите зависимость spring-boot-starter-data-jpa в ваш проект.

Стартеры содержат ссылки на множество зависимостей, необходимых для быстрого запуска проекта с согласованным, поддерживаемым набором управляемых транзитивных зависимостей. Приведем ниже перечень основных используемых стартеров:

- `spring-boot-starter` — главный стартер;
- `spring-boot-starter-activemq` — для сообщений JMS;
- `spring-boot-starter-amqp` — для Spring AMQP и Rabbit MQ;
- `spring-boot-starter-aop` — для аспект-ориентированного программирования;
- `spring-boot-starter-artemis` — для JMS Apache Artemis;
- `spring-boot-starter-batch` — для Spring Batch;
- `spring-boot-starter-cache` — для Spring Framework' кеша;
- `spring-boot-starter-cloud-connectors` — для облачных коннекторов;
- `spring-boot-starter-data-cassandra` — для Cassandra БД;
- `spring-boot-starter-data-couchbase` — для Spring Data Couchbase;
- `spring-boot-starter-data-elasticsearch` — для Spring Data Elasticsearch;
- `spring-boot-starter-data-gemfire` — для Spring Data GemFire;
- `spring-boot-starter-data-jpa` — для Spring Data JPA и Hibernate;
- `spring-boot-starter-data-ldap` — для Spring Data LDAP;
- `spring-boot-starter-data-mongodb` — для Spring Data MongoDB;
- `spring-boot-starter-data-neo4j` — для Spring Data Neo4j;
- `spring-boot-starter-data-redis` — Starter for using Redis key-value data store with Spring Data Redis and the Jedis client;
- `spring-boot-starter-data-rest` — для Spring Data REST;
- `spring-boot-starter-data-solr` — для Spring Data Solr;
- `spring-boot-starter-freemarker` — для FreeMarker views;
- `spring-boot-starter-groovy-templates` — для Groovy Templates views;
- `spring-boot-starter-hateoas` — для Spring MVC и Spring HATEOAS;
- `spring-boot-starter-integration` — для Spring Integration;
- `spring-boot-starter-jdbc` — для Tomcat JDBC connection pool;
- `spring-boot-starter-jersey` — для RESTful JAX-RS;
- `spring-boot-starter-jooq` — для jOOQ SQL databases;
- `spring-boot-starter-jta-atomikos` — для JTA transactions Atomikos;
- `spring-boot-starter-jta-bitronix` — для JTA transactions Bitronix;
- `spring-boot-starter-jta-narayana` — Spring Boot Narayana JTA Starter;
- `spring-boot-starter-mail` — для Spring Framework's email;
- `spring-boot-starter-mobile` — для Spring Mobile;
- `spring-boot-starter-mustache` — для MVC Mustache;
- `spring-boot-starter-security` — для Spring Security;

- `spring-boot-starter-social-facebook` — для Spring Social Facebook;
- `spring-boot-starter-social-linkedin` — для Spring Social LinkedIn;
- `spring-boot-starter-social-twitter` — для Spring Social Twitter;
- `spring-boot-starter-test` — для JUnit, Hamcrest and Mockito;
- `spring-boot-starter-thymeleaf` — для MVC Thymeleaf;
- `spring-boot-starter-validation` — для Java Bean Hibernate Validator;
- `spring-boot-starter-web` — для RESTful;
- `spring-boot-starter-web-services` — для Spring Web Services;
- `spring-boot-starter-websocket` — для WebSocket.

3.2. Структура кода приложения

Приложение формируется из набора пакетов, содержащих определения классов и интерфейсов. Если класс не содержит объявление пакета, он считается помещенным в «пакет по умолчанию». Использование пакета «по умолчанию» не рекомендуется и следует избегать. Это может вызвать определенные проблемы для приложения Spring Boot, использующего аннотации `@ComponentScan`, `@EntityScan` или `@SpringBootApplication`, так как каждый класс из каждого пакета будет видимым.

Расположение главного класса приложения. Обычно рекомендуется размещать основной класс приложения в корневом пакете над другими классами. Аннотация `@EnableAutoConfiguration` часто помещается в основной класс, и она неявно определяет «базовый пакет для поиска» необходимых элементов. Например, для приложения JPA пакет, в котором есть класс с аннотацией `@EnableAutoConfiguration`, будет использоваться для поиска элементов `@Entity`.

Использование корневого пакета также позволяет использовать аннотации `@ComponentScan` без указания атрибута `basePackage`. Вы также можете использовать аннотацию `@SpringBootApplication`, если главный класс находится в корневом пакете.

Типичная картина пакетов приложения Spring Boot:

```
com
+- example
    +- myproject
        +- Application.java
        |
        +- domain
            +- Customer.java
            +- CustomerRepository.java
            |
            +- service
                +- CustomerService.java
                |
            +- web
                +- CustomerController.java
```

Файл *Application.java* объявляет главный метод *main* с базовой аннотацией *@Configuration* (ниже будет описана аннотация *@SpringBootApplication* — комбинация приведенных в примере аннотаций):

```
package com.example.myproject;
import ...;
@Configuration
@EnableAutoConfiguration
@ComponentScan
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Многие разработчики *Spring Boot* всегда пишут главный класс с аннотациями *@Configuration*, *@EnableAutoConfiguration* и *@ComponentScan*. Поскольку данные аннотации часто используются вместе (особенно, если вы следуете лучшим практикам), *Spring Boot* обеспечивает удобную аннотацию *@SpringBootApplication* — альтернативу упомянутой комбинации аннотаций. Аннотация *@SpringBootApplication* эквивалентна совместному использованию *@Configuration*, *@EnableAutoConfiguration* и *@ComponentScan* с атрибутами по умолчанию, например:

```
package com.example.myproject;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
    SpringApplication;
@SpringBootApplication // аналог @Configuration
    @EnableAutoConfiguration @ComponentScan
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Spring Beans и инъекции зависимостей. Для определения бинов и инъекций зависимостей можно использовать любой из стандартных приемов Spring Framework. Для простоты использования `@ComponentScan` для поиска бинов в сочетании с аннотацией `@Autowired` хорошо работают инъекции в конструкторе.

Если структурируется код, как было предложено выше (класс приложения в корневом пакете), можно добавить аннотацию `@ComponentScan` без каких-либо аргументов. Все компоненты приложения (аннотированные `@Component`, `@Service`, `@Repository`, `@Controller` и т. д.) будут автоматически зарегистрированы как бины Spring Beans.

Ниже приведен пример с аннотацией `@Service`, для которой используется инъекция через параметр конструктора для получения необходимого бина `RiskAssessor`:

```
package com.example.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

package com.example.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class DatabaseAccountService implements AccountService {
    private final RiskAssessor riskAssessor;
    @Autowired
    // вызов конструктора класса с параметром
    public DatabaseAccountService(RiskAssessor riskAss) {
```

```
        this.riskAssessor = riskAss;  
    }  
    // ...  
}
```

3.3. Запуск приложения

Одним из самых больших преимуществ упаковки вашего приложения в архив `jar` и использования встроенного в архив `http`-сервера является то, что можно запускать свое приложение так же, как любое другое приложение Java. Отладка приложения тоже простая — вам не нужны никакие специальные плагины или расширения.

Запуск из среды разработки IDE. Можно запустить приложение Spring Boot из IDE как простое приложение Java, однако сначала нужно будет импортировать проект в среду разработки (если его там нет). Шаги импорта будут изменяться в зависимости от вашего IDE и системы сборки (например, шаги жизненного цикла `clean`, `install`). Большинство IDE могут импортировать проекты Maven непосредственно.

Если нет возможности напрямую импортировать проект в IDE, можно сгенерировать метаданные IDE с помощью плагина для построения. Maven включает в себя плагины для сред Eclipse и IDEA, а Gradle предлагает плагины для различных IDE.

Запуск в виде упакованного приложения. Если в Spring Boot используется Maven — или Gradle — плагин для создания исполняемого `jar`, то можно запустить приложение с помощью команды `java-jar`. Например:

```
$ java -jar target/myproject-0.0.1-SNAPSHOT.jar
```

Кроме того, можно запустить упакованное приложение с включенной поддержкой удаленной отладки. Это позволяет присоединить отладчик к упакованному приложению.

Использование плагина Maven. Плагин Spring Boot Maven включает в себя цель запуска, которая может быть использована для быстрой компиляции и запуска приложения. Приложения выполняются в развернутой форме точно так же, как и в IDE:

```
$ mvn spring-boot:run
```

Для этой же цели можно использовать полезные переменные среды операционной системы, например:

```
$ export MAVEN_OPTS=-Xmx1024m -XX:MaxPermSize=128M
```

Выход из приложения. Каждое приложение `SpringApplication` должно использовать способ завершения работы с JVM, чтобы гарантировать закрытие `ApplicationContext` при выходе. Все стандартные обратные вызовы жизненного цикла (таких как интерфейс `DisposableBean` или аннотации `@PreDestroy`) могут быть завершены. Кроме того, бины могут реализовывать интерфейс `org.springframework.boot.ExitCodeGenerator`, если они хотят вернуть определенный код возврата для вызова `SpringApplication.exit()`. Данный код может быть передан в `System.exit()`, чтобы вернуть его как код состояния:

```
@SpringBootApplication
public class ExitCodeApplication {
    @Bean
    public ExitCodeGenerator exitCodeGenerator() {
        return new ExitCodeGenerator() {
            @Override
            public int getExitCode() {
                return 42;
            }
        };
    }
    public static void main(String[] args)
    {
        System.exit(SpringApplication.exit
            (SpringApplication.run(ExitCodeApplication.class, args)));
    }
}
```

Кроме того, интерфейс `ExitCodeGenerator` может быть реализован с помощью исключений. Когда такое исключение встречается, Spring Boot возвращает код выхода, реализованный методом `getExitCode()`.

3.4. Работы с базами данных SQL

Spring Framework предоставляет обширную поддержку для работы с базами данных SQL от прямого доступа JDBC с помощью JdbcTemplate до полной поддержки технологий «Объектно-реляционного отображения», таких как Hibernate. Технология Spring Data обеспечивает дополнительный уровень функциональности, создавая реализации репозитория непосредственно из интерфейсов и используя соглашения для автоматического создания запросов на основе имен методов. Однако следует учитывать многократное замедление работы методов Spring Data по сравнению с простым JDBC, что важно при разработке высоконагруженных приложений.

Настройка источника данных. Интерфейс `javax.sql.DataSource` предоставляет стандартный метод работы с подключениями к базе данных. Традиционно источник данных использует URL вместе с некоторыми учетными данными для установления соединения с базой данных. Конфигурирование источника данных выполняется в файле `src/main/resources/application.properties`, например:

```
## Spring DATASOURCE (DataSourceAutoConfiguration
    & DataSourceProperties)
spring.datasource.url=jdbc:postgresql://localhost:5432/test
spring.datasource.username=postgres
spring.datasource.password=postgres
# The SQL dialect makes Hibernate generate better SQL for the
    chosen database
spring.jpa.properties.hibernate.dialect =
    org.hibernate.dialect.PostgreSQLDialect
# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto = update
spring.jpa.properties.hibernate.jdbc.lob.non_contextual_creation=true
```

Свойство URL задает расположение СУБД и базы данных приложения, свойства `username` и `password` определяют полномочия вашего приложения при работе с БД. Остальные строки определяют настройки пакетов работы с БД.

Поддержка встроенной базы данных. Часто удобно разрабатывать приложения с помощью встроенной базы данных в оперативной па-

мяти ПК. Очевидно, что базы данных в памяти не обеспечивают постоянного хранения, следовательно, вам нужно будет заполнять базу данных при каждом запуске приложения и быть готовым потерять данные, когда приложение заканчивается. Такой вариант удобен для демонстрационных версий приложения.

Spring Boot может автоматически настраивать встроенные базы данных H2, HSQL и Derby. Нет необходимости предоставлять URL-адреса соединений — просто нужно включить нужную зависимость для встроенной базы данных, которую вы хотите использовать.

При использовании этой функции в тестах можно заметить, что одна и та же база данных повторно используется целым набором тестов независимо от количества используемых контекстов приложения. Если нужно убедиться, что каждый контекст имеет отдельную встроенную базу данных, то необходимо установить в файле `application.properties` свойство `spring.datasource.generate-unique-name` в `True`. Например, типичные зависимости POM будут выглядеть так:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.hsqldb</groupId>
  <artifactId>hsqldb</artifactId>
  <scope>runtime</scope>
</dependency>
```

Подключение к базе данных. Подключения к корпоративной базе данных также можно настроить автоматически с помощью пула источников данных. Последовательность шагов выбора конкретной реализации и аргументация принятых решений следующая:

- если доступен источник данных HikariCP, то он всегда выбирается за его производительность и параллелизм;
- в противном случае, если доступен источник данных Tomcat pooling, то используем его;
- если ни HikariCP, ни Tomcat pooling недоступны, а доступен Commons DBCP2, то используем его.

Если используем стартеры `spring-boot-starter-jdbc` или `spring-boot-starter-data-jpa`, то автоматически получаем зависимость для HikariCP.

Можно полностью обойти этот алгоритм и указать пул соединений для использования с помощью свойства `spring.datasource.type`. Это особенно важно, если запускается приложение в контейнере Tomcat, так как `tomcat-jdbc` предоставляется по умолчанию. Дополнительные пулы соединений всегда можно настроить вручную. Если вы определяете свой собственный бин источника данных, то не будет производиться автоматическая конфигурация.

Конфигурация источника данных управляется свойствами внешней конфигурации `spring.datasource.*`. Например, можно объявить следующий фрагмент в файле `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost/test
spring.datasource.username=dbuser
spring.datasource.password=dbpass
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

Нужно по крайней мере указать URL-адрес с помощью свойства `spring.datasource.url` или Spring Boot попытается автоматически настроить встроенную базу данных. Указывать имя класса драйвера `driver-class-name` часто не нужно, так как Spring boot может вывести его из URL-адреса для большинства баз данных.

Для создания пула источника данных `DataSource` необходимо проверить доступность класса драйвера `Driver` прежде, чем делать что-либо, т.е. если установить `spring.datasource.driver-class-name=com.mysql.jdbc.Driver`, то этот класс должен быть загружаемым. При использовании пула соединений Tomcat можно настроить множество дополнительных параметров, например:

```
# Number of ms to wait before throwing an exception if no
    connection is available.
spring.datasource.tomcat.max-wait=10000
# Maximum number of active connections that can be allocated from
    this pool at the same time.
spring.datasource.tomcat.max-active=50
# Validate the connection before borrowing it from the pool.
spring.datasource.tomcat.test-on-borrow=true
```

Подключение к источнику данных JNDI. При развертывании приложения Spring Boot на сервере приложений (например, GlassFish) может потребоваться настройка источника данных и управление им

с помощью встроенных компонентов серверов приложений (консоль администратора), а также получение к нему доступа с помощью JNDI.

Свойство `spring.datasource.jndi-name` можно использовать в качестве альтернативы свойствам `spring.datasource.url`, `spring.datasource.username` и `spring.datasource.password` для доступа к источнику данных через JNDI. Например, следующий раздел в `application.properties` показывает, как можно получить доступ к JBoss как к источнику данных:

```
spring.datasource.jndi-name=java:jboss/datasources/customers
```

Java Persistence API и Spring Data. Java Persistence API — это стандартная технология, которая позволяет сопоставлять объекты классов сущностей с записями таблиц реляционных баз данных. Инструментарий `spring-boot-starter-data-jpa` обеспечивает быстрый способ работы. Он предоставляет следующие ключевые зависимости:

- **Hibernate** — одна из самых популярных реализаций JPA;
- **Spring Data JPA** — можно легко создавать хранилища на основе jpa;
- **Spring ORM** — базовая поддержка ORM Spring Framework.

По умолчанию Spring Boot использует Hibernate 5.0.x, однако также можно использовать версии 4.3.x или 5.0.x, если необходимо.

Spring Data JPA Repositories. Методы доступа к данным определяют интерфейсами Spring Data JPA Repositories. Запросы JPA создаются автоматически из имен методов. Например, интерфейс `CityRepository` может объявить метод `Findallbystate (String state)`, чтобы найти все города с требуемыми характеристиками.

Для более сложных запросов вы можете создавать методы, используя аннотации `@Query`, позволяющие выписывать запросы на языке JPQL, по форме близкой к языку SQL.

С помощью Spring Data repositories обычно расширяются интерфейсы `Repository` или `CrudRepository`. Если вы используете автоматическую настройку, поиск в репозиториях будет производиться из пакета, содержащего основной класс конфигурации (с аннотацией `@EnableAutoConfiguration` или `@SpringBootApplication`).

Пример типичного Spring Data repository:

```
package com.example.myapp.domain;  
import org.springframework.data.domain.*;
```



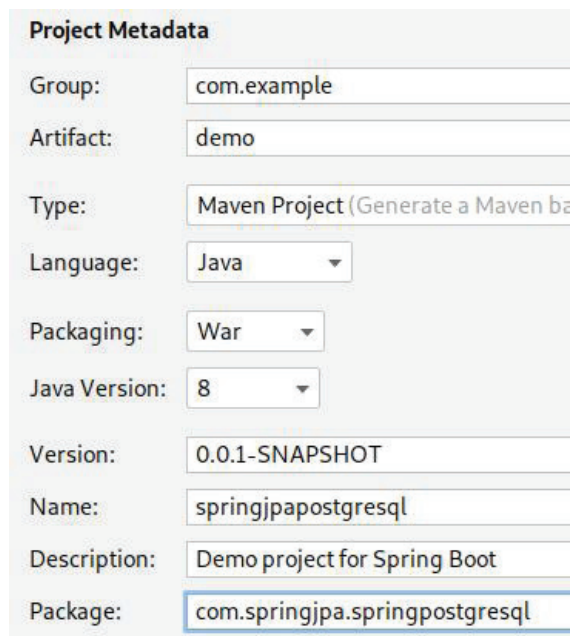
```
import org.springframework.data.repository.*;
public interface CityRepository extends Repository<City, Long>
{
    List<City> findAll(Pageable pageable);
    City findByNameAndCountryAllIgnoringCase(String name, String
        country);
}
```

Контрольные вопросы

1. Какую платформу вы бы предпочли для разработки веб-приложения: Spring MVC или Spring Boot?
2. За счет каких решений платформа Spring Boot выигрывает в объеме кода?
3. Какой архив предпочтительней при запуске приложения на платформе Spring Boot: JAR или WAR?
4. За счет чего код для доступа к базам данных на платформе Spring Boot имеет меньший объем?

4. Разработка приложения с СУБД PostgreSQL

Приведенное ниже приложение имеет учебный характер и демонстрирует формы для ввода одного данного строкового типа и последующего вывода содержимого таблицы пользователей с уточнением места проживания из связанной внешним ключом таблицы адресов. Конфигурирование приложения выполняется с использованием только Java-конфигураций и соответствующих аннотаций. Создание каркаса приложения выполняется встроенным инициализатором IntelliJ IDEA.



The image shows the 'Project Metadata' dialog box in IntelliJ IDEA. It contains the following fields and values:

Project Metadata	
Group:	com.example
Artifact:	demo
Type:	Maven Project (Generate a Maven ba
Language:	Java
Packaging:	War
Java Version:	8
Version:	0.0.1-SNAPSHOT
Name:	springjpapostgresql
Description:	Demo project for Spring Boot
Package:	com.springjpa.springpostgresql

Рис. 10. Настройки проекта

Создание приложения Spring Boot и добавление зависимостей. В меню среды IDEA выберем File => New => Project и заполним поля формы: File => New => Project => Spring Initializr => Next (см. рис. 10).

Откорректировать поле Package, указав папку для размещения проекта.

Кликнуть Next, в секции SQL: выбрать JPA и PostgreSQL, в секции Web: выбрать Web (рис. 11).

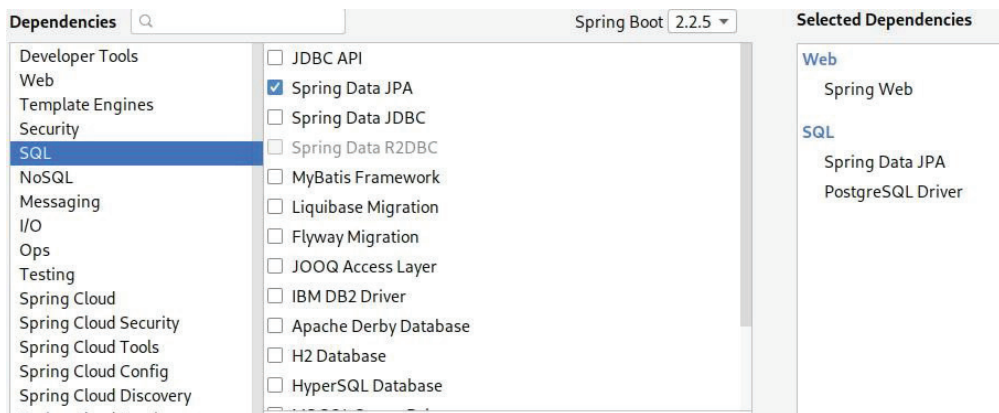


Рис. 11. Настройки опций проекта

Кликнуть Next => Finish => This Window — проект будет успешно создан.

Открыть файл pom.xml, проконтролировать и добавить нужные зависимости (dependencies):

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.javamaster</groupId>
  <artifactId>springjpapostgres</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>springjpapostgres</name>
  <description>Spring Boot JPA PSQL</description>
  <parent> <!-- lookup parent from repository -->
    <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.1.2.RELEASE</version>
<relativePath/>
</parent>
<properties>
  <java.version>1.8</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>
  <!-- JSTL tag lib -->
  <dependency>
    <groupId>javax.servlet.jsp.jstl</groupId>
    <artifactId>javax.servlet.jsp.jstl-api</artifactId>
    <version>1.2.1</version>
  </dependency>
  <dependency>
    <groupId>taglibs</groupId>
    <artifactId>standard</artifactId>
```

```

        <version>1.1.2</version>
    </dependency>
    <!-- Tomcat for JSP rendering -->
    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-jasper</artifactId>
        <scope>provided</scope>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

Конфигурация Spring JPA. Нужно создать папки для файлов и папки для программ на языке Java (рис. 12).

Открыть `application.properties` и ввести следующие строки:

```

spring.mvc.view.prefix = /WEB-INF/views/
spring.mvc.view.suffix = .jsp
spring.mvc.static-path-pattern=/resources/**
spring.datasource.url=jdbc:postgresql://localhost:5432/usersmanagement
spring.datasource.username=postgres
spring.datasource.password=postgres

```

Первые три строки определяют конфигурацию сервлета `FacesServlet`, выполняющего обработку всех запросов к приложению. Атрибут `spring.mvc.view.prefix` задает путь к страницам приложения относительно корневой папки `webapp`. В папке `webapp/WEB-INF/views` будут расположены `jsp`-файлы страниц приложения.

Настройки определяются остальными строками правил доступа к таблицам базы данных СУБД PostgreSQL.

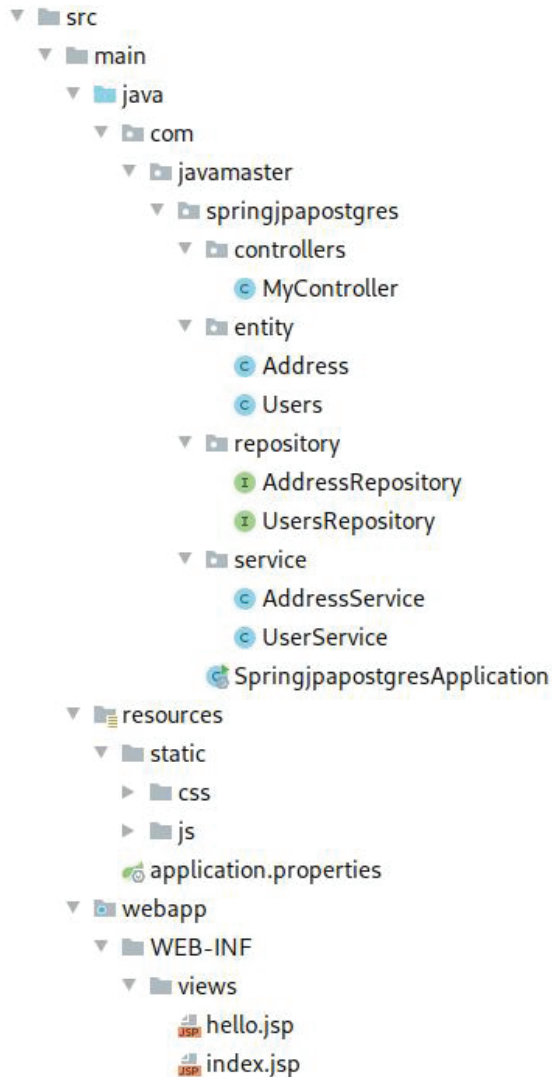


Рис. 12. Структура папок приложения

Создание сущностей модели данных. В пакете `com.javamaster.springjpapostgres.entity` нужно создать класс в файле `Address.java`:

```

package com.javamaster.springjpapostgres.entity;
import javax.persistence.*;
@Entity
@Table(name = "address")
public class Address {

```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column
    private String city;
    @Column
    private String street;
    @Column(name = "home_number")
    private String homeNumber;
    //getters and setters
    public String getHomeNumber() {
        return homeNumber;
    }
    public void setHomeNumber(String homeNumber) {
        this.homeNumber = homeNumber;
    }
    @Override
    public String toString() {
        return "Address{" +
            "id=" + id +
            ", city='" + city + '\'' +
            ", street='" + street + '\'' +
            ", homeNumber='" + homeNumber + '\'' +
            '}';
    }
}

```

Аннотация `@Entity` означает, что `Address` есть сущность, а `@Table` задает таблицу БД (имя `address`) для `@Entity`.

`@ID` указывает на первичный ключ и `@GeneratedValue` указывает на стратегию генерации для значения первичного ключа добавляемых записей.

`@Column` сопоставляет столбец (в таблице) с хранимым полем (в экземпляре класса).

Аналогичным образом создается класс `Users`:

```

package com.javamaster.springjppapostgres.entity;
import javax.persistence.*;
@Entity
@Table(name = "users")
public class Users {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```
private Long id;
@Column
private String name;
@Column
private String email;
@ManyToOne(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
@JoinColumn(name = "address_id")
private Address address;
//getters and setters
@Override
public String toString() {
    return "Users{" +
        "id=" + id +
        ", name='" + name + '\'' +
        ", email='" + email + '\'' +
        ", address=" + address +
        '}';
}
}
```

Создание интерфейса Spring JPA Repository. Создать пакет repository и в нем интерфейсы AddressRepository и UsersRepository. Эти интерфейсы позволяют реализовать все методы CRUD (CReate, Update, Delete) для классов Address и Users.

Создадим файл AddressRepository.java:

```
package com.javamaster.springjpapostgres.repository;
import com.javamaster.springjpapostgres.entity.Address;
import org.springframework.data.jpa.repository.JpaRepository;
public interface AddressRepository extends JpaRepository<Address,
Long> {
    //List<Address> findAll();//простое правильное название метода
    даст возможность избежать запросов на SQL
}
```

Создадим файл UsersRepository.java:

```
package com.javamaster.springjpapostgres.repository;
import com.javamaster.springjpapostgres.entity.Users;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;
public interface UsersRepository extends JpaRepository<Users,
Long> {
```

```

List<Users> findAllByName(String name); //просто правильное название
    метода даст возможность
//избежать запросов на SQL
@Query("select u from Users u where u.email like '%@gmail.com%'")
    //если этого мало можно написать
//собственный запрос на языке похожем на SQL
List<Users> findWhereEmailIsGmail();
@Query(value = "select * from users where name like '%smith%",
    nativeQuery = true)
    //если и этого мало - можно написать запрос на чистом SQL
    // и все это будет работать
List<Users> findWhereNameStartsFromSmith();
}

```

Создание Web Controller. Контроллер получает запросы от клиента, используя репозиторий для обновления, получения данных и возврата результатов. Создайте пакет `com.javamaster.springjapostgres.controllers`, а в нем класс `MyController`.

Создадим файл `MyController.java`:

```

package com.javamaster.springjapostgres.controllers;
import com.javamaster.springjapostgres.entity.Users;
import com.javamaster.springjapostgres.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import java.util.List;
@Controller
    public class MyController {
        @RequestMapping("/")
        public String before(){
            return "index";
        }
        @RequestMapping("/index")
        public String after(@RequestParam("name") String replace, Model
            model) {
            model.addAttribute("name", replace);    // Сохранили в модели
            return "index";
        }
        @Autowired
        UserService userService;
    }

```

```
@PostMapping("/hello")
public String sayHello(@RequestParam("name") String name,
Model model) {
    model.addAttribute("name", name); // Сохранили ответ
    List<Users> lstUser = userService.findAll(); // Нашли всех
    пользователей и сохранили в модели
    model.addAttribute("lstUser", lstUser);
    return "hello";
}
}
```

В методах веб-контроллера, которые аннотируются `@RequestMapping`, `@PostMapping`, использованы автоматические методы репозитория, реализуют интерфейс `CrudRepository`. Реализация этих методов выполняется Spring Boot автоматически на основе имен полей в период исполнения приложения («на лету») и не требует участия программиста. Аннотация `@RequestParam` ('name') извлекает параметр с именем 'name' из строки запроса и передает его значение параметру `name` метода.

Создание сервисов приложения. Сервисы приложения предоставляют реализации методов доступа к таблицам базы данных.

Создадим файл `AddressService.java`:

```
package com.javamaster.springjpapostgres.service;
import com.javamaster.springjpapostgres.entity.Address;
import com.javamaster.springjpapostgres.repository.AddressRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class AddressService {
    @Autowired
    private final AddressRepository addressRepository;
    public AddressService(AddressRepository addressRepository) {
        this.addressRepository = addressRepository;
    }
    public void createAddress(Address address) {
        addressRepository.save(address);
    }
}
```

Создадим файл `UserService.java`:

```
package com.javamaster.springjpapostgres.service;
import com.javamaster.springjpapostgres.entity.Users;
import com.javamaster.springjpapostgres.repository. UsersRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class UserService {
    @Autowired
    private final UsersRepository usersRepository;
    public UserService(UsersRepository usersRepository){
        this.usersRepository = usersRepository;
    }
    public void createUsers(Users users) {
        usersRepository.save(users);
    }
    public List<Users> findAll(){
        return usersRepository.findAll();
    }
    public Users findById(Long userId){
        return usersRepository.findById(userId).orElse(null);
    }
    public List<Users> findAllByName(String name){
        return usersRepository.findAllByName(name);
    }
    public List<Users> findWhereEmailIsGmail(){
        return usersRepository.findWhereEmailIsGmail();
    }
    public List<Users> findWhereNameStartsFromSmith(){
        return usersRepository.findWhereNameStartsFromSmith();
    }
}
```

Главный класс приложения. Создадим файл `SpringjpapostgresApplication.java`:

```
package com.javamaster.springjpapostgres;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure. SpringBootApplication;
@SpringBootApplication
public class SpringjpapostgresApplication {
    public static void main(String[] args) {
```

```
        SpringApplication.run(SpringjppapostgresApplication.class, args);
    }
}
```

Страницы приложения. Создадим файл index.jsp:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html;
charset=ISO-8859-1">
    <!-- Static content -->
    <link rel="stylesheet" href="/resources/css/style.css">
    <script type="text/javascript" src="/resources/js/app.js"></script>
    <title>Spring Boot</title>
</head>
<body>
<h1>Spring Boot - MVC web application Index</h1>
<hr>
<div class="form">
    <form action="hello" method="post" onsubmit="return validate()">
        <table>
            <tr>
                <td>Введите ваше имя</td>
                <td><input id="name" name="name" value="Это я, господа!"></td>
            <tr>
                <td><input type="submit" value="Принять"></td>
            </tr>
        </table>
    </form>
</div>
</body>
</html>
```

Создадим файл hello.jsp:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="Content-Type" content = "text/html;
        charset=ISO-8859-1">
```

```

    <!-- Static content -->
    <link rel="stylesheet" href="/resources/css/style.css">
    <script type="text/javascript" src="/resources/js/app.js"></script>
    <title>Spring Boot</title>
</head>
<body>
<h1>Spring Boot - MVC web application Hello</h1>
<hr>
<div class="form">
    <form action="index" method="post" onsubmit="return validate()">
        <table>
            <tr>
                <td>Проверьте имя</td>
                <td><input id="name" name="name" value="{name}"></td>
            <tr>
                <td><input type="submit" value="Принять"></td>
            </tr>
        </table>
    </form>
</div>
<h2>Вы ввели имя {name}</h2>
<br>
<h3>Таблица пользователей</h3>
<br>
<table border="1">
    <tr>
        <th>Имя</th>
        <th>E-mail</th>
        <th>Город</th>
    </tr>
    <c:forEach items="{lstUser}" var="user">
        <tr>
            <td>"{user.name}"</td>
            <td> "{user.email}"</td>
            <td> "{user.address.city}"</td>
        </tr>
    </c:forEach>
</table>
<br>
</body>
</html>

```

Создание таблиц в PostgreSQL. Операторы создания таблиц имеют следующий вид:

```
CREATE TABLE public.address
(
    id serial,
    city text,
    street text,
    home_number character varying(5),
    CONSTRAINT address_pkey PRIMARY KEY (id)
)
WITH (
    OIDS=FALSE
);
ALTER TABLE public.address
    OWNER TO postgres;
-- Index: public.address_id_uindex
CREATE TABLE public.users
(
    id serial,
    name text,
    email character varying(20),
    address_id integer NOT NULL,
    CONSTRAINT users_pkey PRIMARY KEY (id),
    CONSTRAINT users_address_id_fk FOREIGN KEY (address_id)
        REFERENCES public.address (id) MATCH SIMPLE
        ON UPDATE NO ACTION ON DELETE NO ACTION
)
WITH (
    OIDS=FALSE
);
ALTER TABLE public.users
    OWNER TO postgres;
```

Запуск Spring Boot Application и получение результата. В панели Maven Projects задания команд жизненного цикла Lifecycle выполнить команды:

```
clean
package
```

Запустить проект в режиме Spring Boot App из главного меню с помощью команды Run (Shift+F10). Проверить работоспособность приложения в браузере, выполнив запрос (рис. 13, а). Результат запроса показан на рис. 14.

a **Spring Boot - MVC web application Index**

Введите ваше имя

б **Spring Boot - MVC web application Hello**

Проверьте имя

Вы ввели имя Иван

Таблица пользователей

Имя	E-mail	Город
"Smith"	"someEmail@gmail.com"	"Екатеринбург"
"Smith"	"someEmail@gmail.com"	"Екатеринбург"
"Smith"	"someEmail@gmail.com"	"Екатеринбург"

Рис. 13. Запрос на ввод имени (*a*) и результат запроса (*б*)

Конфигурирование сервера GlassFish 5. Открыть окно Run/Dedug Configurations конфигурирования запуска приложения и установить нужные параметры сервера GlassFish Server Local (рис. 14).

На вкладке Deployment задать имя архива springjapostgresql: war exploded (кнопка «+»).

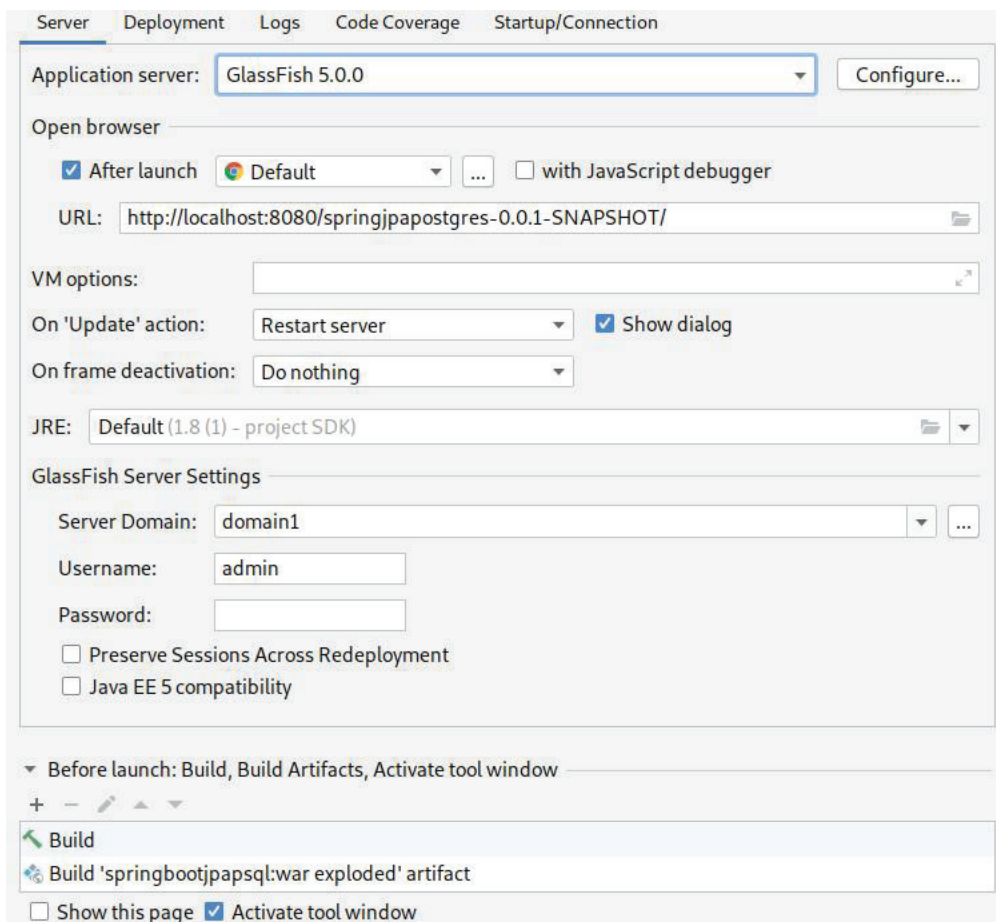


Рис. 14. Конфигурирование сервера приложений

Контрольные вопросы

1. Почему каркас приложения создается с помощью встроенного инициализатора IntelliJ IDEA?
2. Какова роль объявлений сущностей в объектно-реляционном отображении (ORM) для работы с таблицами баз данных?
3. Использование репозитория для доступа к базам данных сокращает объем кода?
4. Какова роль сервисов в разработке приложений?
5. Каким сервером необходимо воспользоваться при отладке приложения?

5. Проверка данных

В качестве механизмов контроля вводимых данных можно использовать валидаторы, подключение которых выполняется с помощью зависимости:

```
<dependency>
  <groupId>org.hibernate.validator</groupId>
  <artifactId>hibernate-validator</artifactId>
</dependency>
```

Доступны несколько механизмов контроля правильности данных, используемых в различных контекстах приложения: при определении сущностей, в контроллерах для метода POST, на страницах пользовательского интерфейса. Ниже рассмотрен пример, иллюстрирующий приемы верификации данных.

Для этого приложения используется язык шаблонов Thymeleaf, поскольку Thymeleaf — современный серверный механизм Java-шаблонов для веб, способный обрабатывать HTML, XML, JavaScript, CSS и даже простой текст. Основной целью Thymeleaf является создание удобного способа шаблонизации. Thymeleaf основывается на концепции Natural Templates, чтобы внедрить свою логику в файлы шаблонов таким образом, чтобы шаблон не влиял на отображение дизайна прототипа в браузерах. Thymeleaf разработан с учетом стандартов Web, HTML5, что позволяет создавать соответствующие стандартам шаблоны.

В то время как JSP с использованием библиотек тегов может включать фрагмент кода, который не может отображаться непосредственно в браузере, например: `<form: inputText name="userName" value="${user.name}"/>`

Thymeleaf позволит достичь нужной функциональности с помощью обычного тега HTML с дополнительными атрибутами:

```
<input type="text" name="userName" value="Петр"
      th:value="${user.name}" />
```

Кроме того, тег будет правильно отображаться всеми браузерами, потому что он не содержит тегов, отличных от HTML (браузеры игнорируют все атрибуты, которые они не понимают, например 'th: value'). Но это также позволяет (необязательно) указать в нем атрибут value («Петр Иванов»), который будет отображаться, когда прототип открыт в браузере и будет заменен значением, полученным в результате вычисления выражения *\${user.name}* во время обработки шаблона.

Подробности о шаблонах Thymeleaf можно посмотреть на сайте <https://habr.com/ru/post/350862/>.

Поддержка шаблонов Thymeleaf требует указания зависимости в файле pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

5.1. Создание объекта PersonForm

Приложение использует форму с именем класса *PersonForm* и будет контролировать свойства *name* и *age*, для чего создается класс для приема данных с формы при создании новой персоны. Ссылка на бин формы будет нужна при написании шаблона страницы ввода данных от пользователя приложения. Соответствие полей ввода формы и свойств бина формы поддерживается автоматически библиотеками среды исполнения приложения:

```
src/main/java/hello/PersonForm.java
package hello;
import javax.validation.constraints.Min;
import javax.validation.constraints.NotNull;
```

```
import javax.validation.constraints.Size;
public class PersonForm {
    @NotNull
    @Size(min=2, max=30)
    private String name;
    @NotNull
    @Min(18)
    private Integer age;
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Integer getAge() {
        return age;
    }
    public void setAge(Integer age) {
        this.age = age;
    }
    public String toString() {
        return "Person(Name: " + this.name + ", Age: " + this.age + ")";
    }
}
```

Класс *PersonForm* имеет два свойства: *name* и *age*. Они аннотированы стандартными проверками:

- `@Size (min=2, max=30)` — разрешает имени быть длиной от 2 до 30 символов;
- `@NotNull` — не разрешает имени быть пустым;
- `@Min (18)` — не разрешает возрасту быть меньше 18 лет.

5.2. Создание веб-контроллера

После того как вы определили объект для формы, необходимо создать простой веб-контроллер.

Создадим файл *WebController.java*:

```
src/main/java/hello/WebController.java
package hello;
```

```
import javax.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import
    org.springframework.web.servlet.config.annotation.View-
    ControllerRegistry;
import
    org.springframework.web.servlet.config.annotation.
    WebMvcConfigurer;

@Controller
public class WebController implements WebMvcConfigurer {
    @Override
    public void addViewControllers(ViewControllerRegistry
    registry) { registry.addViewController("/results").
    setViewName("results");
    }

    @GetMapping("/") // обслуживание запроса на ввод
    public String showForm(PersonForm personForm) {
        return "form";
    }

    @PostMapping("/") // обслуживание приёма информации
    public String checkPersonInfo(@Valid PersonForm personForm,
    BindingResult bindingResult) {
        if (bindingResult.hasErrors()) {
            return "form";
        }
        return "redirect:/results";
    }
}
```

Данный контроллер обрабатывает запросы с методами GET и POST, которые сопоставлены с путем «/» в запросе. GET-метод `showForm` возвращает страницу формы. Он включает `PersonForm` в определении параметров метода, чтобы шаблон Thymeleaf мог связать атрибуты формы ввода с классом `PersonForm`.

POST-метод `checkPersonFormInfo` принимает два аргумента:

- объект `personForm` помечен аннотацией *@Valid* для контроля атрибутов, прописанных в классе `PersonForm`, экземпляр которого вы собираетесь сформировать.

- объект `bindingResult` класса `BindingResult` выполнит проверку по умолчанию и содержит сведения об обнаруженных ошибках задания данных.

Контроллер выполняет передачу всех данных из формы, привязанной к объекту `PersonForm`. В коде получения данных выполняется проверка на наличие ошибок, при обнаружении которых пользователь возвращается с помощью контроллера к исходной странице формы ввода. В этом случае автоматически отображаются все ошибки в атрибутах персоны.

Если все атрибуты персоны правильные, браузер перенаправляется на страницу окончательных результатов.

5.3. Страницы для пользователя

Главная страница приложения содержит ссылку на форму ввода данных `th: object="${personForm}"` в соответствии с правилами Thymeleaf. Страница отправляет на сервер данные методом POST, адресуя запрос с помощью атрибута `th: action="@{/}"` корню приложения:

`src/main/resources/templates/form.html`

```
<html>
  <body>
    <form action="#" th:action="@{/}"
      th:object="${personForm}" method="post">
      <table>
        <tr>
          <td>Name:</td>
          <td><input type="text" th:field="*{name}" />
          </td>
          <td th:if="${#fields.hasErrors('name')}"
            th:errors="*{name}">Name Error</td>
        </tr>
        <tr>
          <td>Age:</td>
          <td><input type="text" th:field="*{age}" /></td>
          <td th:if="${#fields.hasErrors('age')}"
            th:errors="*{age}">Age Error</td>
        </tr>
      </table>
    </form>
  </body>
</html>
```

```
        <tr>
            <td><button type="submit">Submit</button></td>
        </tr>
    </table>
</form>
</body>
</html>
```

Страница содержит простую форму с полями в отдельных клетках таблицы. Форма предназначена для отправки запроса по адресу корня приложения «/». Он помечается как обработчик объекта формы `personForm`, который был рассмотрен в методе `GET` веб-контроллера (см. с. 76). Этот объект называется бин-формой. В бине `PersonForm` есть два свойства, и на странице они использованы с атрибутами `th: field="{name}"` и `th: field="{age}"` для связи со свойствами класса. Рядом с каждым свойством находится дополнительный элемент `th: errors`, используемый для отображения ошибок результатов проверки. Наконец, есть кнопка для отправки. Как правило, если пользователь вводит имя или возраст, нарушающие ограничения `@Valid`, он возвращается на эту страницу с отображаемым сообщением об ошибке. После ввода допустимого имени и возраста пользователь перенаправляется контроллером `"redirect:/results"` на следующую веб-страницу.

```
src/main/resources/templates/results.html
<html>
    <body>
        Поздравляю! Вы достаточно взрослый.
    </body>
</html>
```

5.4. Создание класса `Application`

```
src/main/java/hello/Application.java
package hello;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
    SpringBootApplication;
```

@SpringBootApplication

```
public class Application {  
    public static void main(String[] args) throws Exception {  
        SpringApplication.run(Application.class, args);  
    }  
}
```

Для того, чтобы активировать Spring MVC, обычно добавляют *@EnableWebMvc* в класс приложения. Но Spring Boot уже имеет аннотацию *@SpringBootApplication*, когда обнаруживает spring-webmvc в вашем classpath. Эта же аннотация позволяет найти аннотированный класс *@Controller* и его методы.

Конфигурация Thymeleaf также заботится о *@SpringBootApplication*: по умолчанию шаблоны расположены в classpath в папке templates/ и используются как представления путем удаления суффикса '.html' из имени файла. Настройки Thymeleaf могут быть изменены и отменены различными способами в зависимости от того, что нужно для достижения цели¹.

Контрольные вопросы

1. Зачем нужно встраивать контроль вводимых данных в веб-приложение?
- 2) Язык шаблонов Thymeleaf используется в Java-коде или на страницах веб-приложения?
- 3) При обнаружении ошибки ввода Thymeleaf приостанавливает работу приложения или позволяет пользователю исправить ввод?

¹ Об этом подробнее см.: Руководство Spring Boot и Thymeleaf. URL: <https://o7planning.org/ru/11545/spring-boot-and-thymeleaf-tutorial> (дата обращения: 15.09.2020).

6. Микросервисная архитектура приложений

Разработка высоконагруженных веб-приложений является комплексной задачей, требующей от программиста знания всего спектра средств информационных технологий. Для такой разработки используют полный стек инструментов, включающий владение техникой создания информационного обеспечения, выбор архитектуры проекта, языки кодирования данных, инструменты и технологии программирования на различных языках, библиотеки и средства поддержки стандартных операций и многое другое.

Высоконагруженные приложения требуют высокой скорости подготовки ответа на запросы клиентов. Время реакции сервера складывается из времени на транспортировку запроса от клиента к серверу, подготовку ответа сервером, обратной передачи ответа клиенту и его визуализации. Длина запроса и время на его передачу обычно незначительны. Основное время тратится на подготовку и передачу ответа клиенту. Традиционные архитектуры веб-приложений на каждый запрос готовят новый HTML-документ, на что затрачивается значительное время, часто определяемое необходимостью поиска информации в базах данных, ее преобразование, форматирование при формировании результирующей страницы и необходимых ей ресурсов. Источником сокращения времени на подготовку ответа является перенос формирования вида страницы на сторону клиента, на сервере остается только формирование необходимых данных. Эту задачу решает переход на микросервисную архитектуру приложения, когда клиент взаимодействует с набором сервисов, поставляющих данные по запросу клиента, а клиент сам формирует вид страницы для пользователя.

Для разработки сервисов эффективной технологией является RESTful API, реализуемая средствами Spring Boot, — это так называемый 'Back end'. Клиентская часть (Front end) в этом случае реализуется с помощью средств разработки одностраничных приложений (SPA — Single Page Application). Такими платформами являются Angular, React и др.

Платформа Angular

Angular — это фреймворк от компании Google для создания клиентских приложений [11]. Прежде всего, он нацелен на разработку SPA — одностраничных приложений, содержание которых динамически изменяется при выполнении манипуляций пользователя органами управления приложением. Для динамического изменения содержимого страниц на стороне браузера используется поддерживаемый всеми браузерами язык JavaScript, библиотеки для манипулирования содержимым страницы, а также объектная модель документа DOM. Программируя сценарии на языке JavaScript, можно заменять как отдельные элементы страниц, так и всё содержимое сразу. Для управления приложением на странице размещают обычные средства формирования событий, а также создаются функции для их обработки, запрашивающие у сервисов информацию и форматирующие ее для размещения в элементах DOM. Платформа Angular предоставляет именно такую функциональность для управления видом загруженной страницы путем разбиения приложения на компоненты, которые содержат описание программной части в виде классов ООП, шаблонов отображения данных, которые, в свою очередь, содержат вставки выражений с использованием свойств экземпляра класса. Для управления форматами отображения информации широко используются библиотеки тегов, атрибуты тегов и стили.

Angular — это платформа для создания одностраничных клиентских приложений с использованием HTML и TypeScript. Она реализует основные и дополнительные функции в виде набора библиотек TypeScript, которые нужно импортировать в свои приложения.

Angular предоставляет различную функциональность для взаимодействия вида с программной частью класса, а именно связывание,

позволяющее динамически изменять отображаемую информацию при изменении данных, и, наоборот, обработку событий, средства асинхронного взаимодействия с источниками данных, шаблоны, маршрутизацию и многое другое, что составляет функционально полный набор инструментов для разработчика.

Одной из ключевых особенностей Angular является то, что платформа использует в качестве языка программирования TypeScript, компилируемый перед исполнением в JavaScript, поддерживаемый всеми браузерами. Поэтому перед началом изучения Angular рекомендуется ознакомиться с основами языка TypeScript, про который можно прочитать на ресурсе <https://metanit.com/web/typescript/>.

Для работы с Angular необходимо установить сервер Node.js и пакетный менеджер npm. Команда установки пакетов в операционной системе Linux Fedora имеет вид

```
dnf install npm
```

Команда выполнит установку пакетного менеджера и сервера NodeJS. Для развертывания новых приложений Angular нужно установить вспомогательный интерпретатор интерфейса командной строки (CLI):

```
npm install -g @angular/cli
```

Проверить версии загруженных пакетов можно с помощью команды: `ng version`

Для создания скелета нового приложения целесообразно использовать интерфейс командной строки (CLI). Для этого выполняется команда терминала в папке для приложений Angular:

```
ng new <имя-приложения>
```

Можно воспользоваться веб-конструктором для работы с Angular по адресу <https://cli.angular.io/>. Конструктор создаст для нового приложения структуру папок, загрузит необходимые библиотеки и файлы демонстрационного примера, вернув архив. Аналогичную работу можно выполнить из среды разработки IntelliJ IDEA, содержащей все средства разработки Angular-приложений.

Запуск сервера для исполнения приложения выполняется из папки приложения с помощью команды `ng serve`.

Протокол запуска приложения выглядит примерно так:

```
** Angular Live Development Server is listening on localhost:4200,
   open your browser on http://localhost:4200/ **
Date: 2019-06-04T12:46:33.183Z
Hash: f542cab8cd37e80d56e2
Time: 18513ms
. . . . .
wdm: Compiled successfully.
```

Для просмотра приложения в браузере выполняется запрос:

`http://localhost:4200/`

Создание компонента Angular. Компоненты представляют основные строительные блоки приложения Angular. Каждое приложение Angular имеет, как минимум, один компонент. Структура стандартного приложения показана на рис. 15.

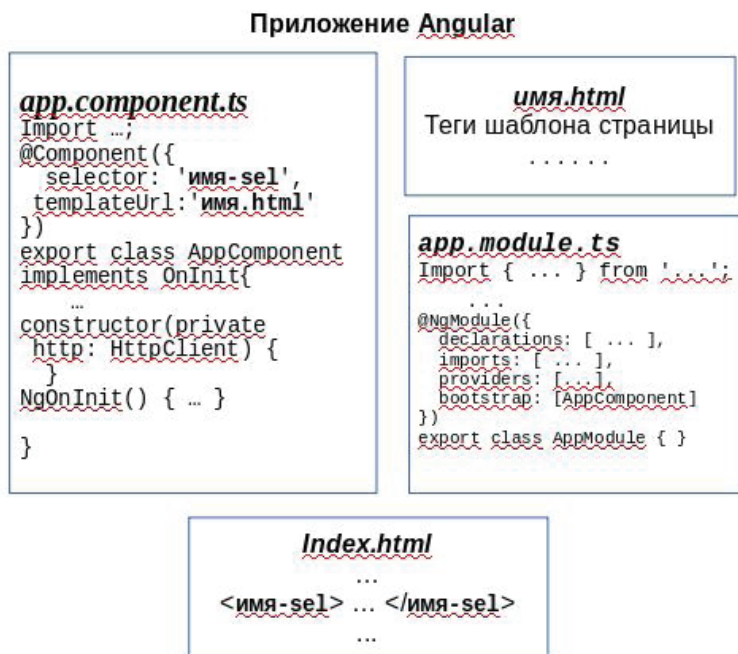


Рис. 15. Структура приложения Angular

Angular — это фреймворк для построения клиентских приложений на HTML и JavaScript, а также на таком языке, как TypeScript, который компилируется в JavaScript. Фреймворк состоит из нескольких библиотек, часть которых является базовой, а другая часть — факультативной. Создание Angular приложения требует подготовки HTML-шаблона с разметкой, написания классов компонентов для управления этими шаблонами, а также реализации логики приложений в службах и объединения компонент и служб в модули.

Запуск приложения приводит к загрузке корневого модуля. Angular выполняет управление приложением, демонстрируя его содержимое в браузере и реагируя на взаимодействие с пользователем в соответствии с предоставленными интерфейсами.

Архитектура приложения Angular опирается на фундаментальные концепции. Основными строительными блоками являются модули. Angular имеет свою собственную модульную систему, называемую Angular modules или NgModules. В приложении всегда есть, по крайней мере, корневой модуль, условно называемый модулем приложения, который включает в себя загрузку и большое число функциональных модулей.

Модульная система NgModules предоставляет контекст компиляции для компонентов и собирает связанный код в функциональные наборы. Хотя корневой модуль может быть единственным модулем в небольшом приложении, большинство приложений имеют гораздо больше функциональных модулей, которые представляют собой целостный блок кода, соответствующий домену приложения, рабочему процессу или набору функций.

NgModule — это класс, снабженный функцией декоратора *@NgModule*. *@NgModule* принимает объект метаданных, который сообщает Angular о том, как компилировать и запускать код модуля. Он идентифицирует собственные компоненты модуля, директивы и каналы, делая некоторые из них общедоступными, чтобы внешние компоненты могли их использовать. *@NgModule* может добавлять поставщиков услуг в инъекторы зависимостей приложений.

Декораторы — это функции, которые модифицируют классы JavaScript. Angular имеет множество декораторов, которые прикрепляют метаданные к классам, чтобы он знал, что эти классы означают и как они должны работать. Angular модуль, будь то корень или объект, — это класс с декоратором *@NgModule*.

NgModule — это функция декоратора, которая принимает один объект метаданных, свойства которого описывают модуль. Наиболее важными свойствами являются:

- **declarations** — классы представлений, принадлежащие этому модулю. Angular имеет три вида классов представления — компоненты, директивы и трубы;
- **exports** — подмножество объявлений, которые должны быть видимыми и использованы в шаблонах компонентов других модулей;
- **imports** — другие модули, экспортируемые классы которых необходимы шаблонам компонентов, объявленным в этом модуле;
- **providers** — создатели сервисов, вносящие этот модуль в глобальную коллекцию сервисов, которые становятся доступными во всех частях приложения;
- **bootstrap** — основное представление приложения, называемое корневым компонентом, в котором размещаются все остальные представления приложения. Только корневой модуль должен установить свойство **bootstrap**.

Пример корневого модуля:

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

@NgModule({
  imports:      [ BrowserModule ],
  providers:    [ Logger ],
  declarations: [ AppComponent ],
  exports:      [ AppComponent ],
  bootstrap:    [ AppComponent ]
})
export class AppModule { }
```

Модули, компоненты и сервисы — это классы, которые используют декораторы, декларирующие их тип и сообщающие метаданные, которые информируют Angular о том, как их использовать.

Метаданные для класса компонента связывают его с шаблоном, определяющим множество страниц. Шаблон объединяет обычный HTML с директивами Angular и разметкой привязки, которые позволяют Angular менять и настраивать элементы HTML перед отображением.

Метаданные для класса сервисов предоставляют информацию, необходимую Angular, чтобы сделать ее доступной для компонентов посредством внедрения зависимостей (DI).

Компоненты приложения обычно определяют множество представлений, расположенных иерархически. Angular предоставляет сервис Router, который определяет способы навигации между представлениями. Маршрутизатор обеспечивает сложные навигационные возможности в браузере.

В папке *src/app* есть файл *app.component.ts*, в котором для примера может быть определен следующий код компонента:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<label>Введите Ваше имя:</label>
    <input [(ngModel)]="name" placeholder="name">
    <h1>Добро пожаловать {{name}}!</h1>`
})
export class AppComponent {
  name = '';
```

В начале файла директива *import* импортирует класс *Component* из модуля *angular/core*, предоставляя декоратор *@Component*. Далее функция-декоратор *@Component* ассоциирует метаданные в формате JSON с классом компонента *AppComponent*. В этой структуре первым определяется ключ *selector* для HTML-элемента, который будет использован для отображения компонента в окне браузера. Вторым здесь определяется ключ *template* или *templateURL*, который указывает на шаблон для визуализации компонента. В этом шаблоне задана двусторонняя привязка с помощью выражения `[(ngModel)]='name'` и односторонняя привязка `{{name}}` к некоторому объекту модели с именем `<name>`. И в конце экспортируется класс компонента *AppComponent*, в котором определяется объект данных модели *name* — пустая строка.

Создание модулей приложения. Приложение Angular строится из модулей. Модульная структура позволяет легко подгружать и задействовать только те модули, которые необходимы приложению. Каждое приложение имеет как минимум один корневой модуль, поэтому в папке *src/app* размещается файл *app.module.ts*. Например, следующего содержания:

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
```

```
import { AppComponent } from './app.component';
import { AppRoutingModuleModule } from './app-routing.module';
@NgModule({
  declarations: [ AppComponent ],
  imports: [ BrowserModule, AppRoutingModuleModule ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

С помощью директив *import* импортируется ряд нужных модулей:

- **NgModule** — основной модуль исполнительной системы;
- **BrowserModule** — модуль для работы с браузером;
- **FormsModule** — модуль для использования элементов ввода input или элементов формы.

Далее импортируются компоненты приложения **AppComponent** и др. Данный модуль с предопределенным именем **AppModule** играет роль входной точки в приложение.

Запуск приложения. Запуск приложения на исполнение выполняется с помощью команды операционной системы из терминала или из среды разработки:

```
ng serve
```

Команда выполнит компиляцию компонентов и развертывание приложения для запуска в браузере по адресу <http://localhost:4200>.

Запуск приложения Angular выполняется с использованием скрипта с именем *main.ts*, расположенного в папке *src* (на уровень выше файлов *app.component.ts* и *app.module.ts*), со следующим содержимым:

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';
import { environment } from './environments/environment';
if (environment.production) {
  enableProdMode();
}
platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Данный код инициализирует платформу и затем использует ее для загрузки модуля *AppModule*, ключ которого с именем *bootstrap*:

[AppComponent] задает имя первого компонента, загружаемого в главное окно приложения тегом `<app-root></app-root>`, в файле *index.html*:

```
<!doctype html>
<html lang="en">
<head>
. . .
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

В папке *src* есть файл *polyfills.ts* примерно следующего содержания:

```
import 'core-js/es6';
// для поддержки Reflect Api
import 'core-js/es7/reflect';
// zone используется angular
import 'zone.js/dist/zone';
```

Файл определяет полифилы — инструменты, необходимые для исполнения приложения Angular старыми браузерами. В частности, первая группа определяет пакеты для работы в IE версий 9—11. Если не планируется поддерживать этот браузер, то данную группу импорта пакетов можно не включать. Всегда файл должен содержать импорт пакета *zone*:

```
import 'zone.js/dist/zone'.
```

Определение конфигурации. Для определения используемых в приложении версий пакетов также в корневой папке проекта есть файл *package.json* примерно следующего содержания:

```
{
  "name": " . . . ",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    . . .
  },
  "private": true,
```



```

"dependencies": {
  "@angular/common": "~8.2.0",
  "@angular/compiler": "~8.2.0",
  "@angular/core": "~8.2.0",
  "zone.js": "~0.9.1"
  . . .
},
"devDependencies": {
  "@angular/cli": "~8.2.0",
  "@angular/compiler-cli": "~8.2.0",
  "@angular/language-service": "~8.2.0",
  "@types/node": "~8.9.4",
  "typescript": "~3.5.3",
  . . .
}
}

```

Файл *tsconfig.json* определяет настройки для компилятора TypeScript. Опция 'compilerOptions' устанавливает параметры компиляции. Оба файла всегда присутствуют в каждом проекте. Файл *sconfig.json* имеет примерно следующее содержание:

```

{
  "compileOnSave": false,
  "compilerOptions": {
    "baseUrl": "./",
    . . .
  }
}

```

Контрольные вопросы

1. За счет чего приложения микросервисной архитектуры увеличивают производительность информационных систем?
2. Почему правомочно использование термина «одностраничное приложение»?
3. Чем отличаются языки JavaScript и TypeScript?
4. Сколько серверов задействовано для работы микросервисного приложения?
5. Что входит в состав компонента платформы Angular?

7. Принципы работы приложений Angular

7.1. Компоненты

Компоненты являются основными элементами приложения Angular. Компонент управляет отображением представления на экране, хранением информации в модели, ее получением, обработкой событий, связью с другими компонентами и сервисами. Подход в разработке компонент опирается в значительной степени на декларативное описание конечного продукта и в меньшей степени на процедурное программирование. Так, при создании первого приложения в прошлой главе был определен следующий компонент [11]:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<label>Введите имя:</label>
             <input [(ngModel)]='name' placeholder='name'>
             <h1>Добро пожаловать {{name}}!</h1>`
})
export class AppComponent {
  name: ''
}
```

Ключевое слово `export` необходимо для возможности использовать класс в других модулях приложения. В классе определена лишь одна переменная в составе модели, которая хранит строку.

Декораторы являются инструментом декларативного программирования, они позволяют добавить к классам и их членам метадан-

ные и тем самым изменить их поведение без изменения их кода. Декораторы представляют собой функции, которые могут применяться к классам, методам, методам доступа (геттерам и сеттерам), свойствам и параметрам. Для создания компонента необходимо импортировать функцию декоратора `@Component` из библиотеки `@angular/core`. Декоратор `@Component` определяет класс как компонент. Если не использовать декоратор `@Component` для класса `AppComponent`, то класс `AppComponent` не считался бы компонентом приложения Angular.

Декоратор `@Component` принимает в качестве параметра объект JSON, задающий конфигурацию, которая указывает исполнительной системе и компилятору, как работать с компонентом и его представлением.

Свойство `template` задает шаблон разметки HTML со вставками кода Angular, определяющий представление, которое увидит пользователь на странице браузера при работе с приложением. Компонент должен иметь один шаблон. Определять шаблон напрямую с помощью ключа `template` необязательно. Можно шаблон вынести во внешний файл с расширением HTML, а для его подключения использовать ключ `templateURL`. Шаблон может быть одно- или многострочным. Если шаблон многострочный, то в ключе `template` он заключается в кавычки ('), отличающиеся от стандартных одинарных кавычек (<). Во внешнем файле с расширением `.html` записываются теги шаблона с использованием тех же вставок объектов из определения класса компонента: привязки, другие атрибуты и функции.

Ключ `selector` определяет имя тега для включения компонента в представление. В тег с этим селектором Angular будет добавлять представление (шаблон) компонента. В примере выше селектор имеет значение `my-app`. Если HTML-страница содержит элемент `<my-app></my-app>`, то вместо этого элемента будет подставлен указанный шаблон компонента:

```
<!DOCTYPE html>
<html>
<head>
  . . .
</head>
<body>
  <my-app>Загрузка...</my-app>
  . . .
</body>
</html>
```

Именно этот элемент будет использоваться для рендеринга (отрисовки) представления компонента. Текст внутри блочного тега будет появляться на странице клиента только на время загрузки нужного шаблона.

7.2. Модули

Модули — отличный способ организовать приложение и расширить его возможностями с помощью внешних библиотек Angular, таких как `FormsModule`, `HttpModule` и `RouterModule`. Многие сторонние библиотеки доступны в виде `NgModules`, например `Material Design`, `Ionic`, `AngularFire2`.

`NgModules` объединяют компоненты, директивы и каналы в единые блоки функциональности. Каждый блок сосредоточен на определенной бизнес-области приложения, рабочем процессе или общем наборе утилит. Модули также могут добавлять сервисы в приложение. Такие службы могут быть разработаны внутри, например регистратор приложений. Сервисы могут поступать из внешних источников, таких как `Angular router` и `Http client`.

Модули могут быть загружены синхронно (*eagerly*), когда приложение запускается. Они также могут быть загружены маршрутизатором асинхронно (*lazy*).

`NgModule` — это класс, украшенный метаданными `@NgModule`. Метаданные делают следующее:

- объявляют, какие компоненты, директивы и каналы принадлежат модулю;
- делают некоторые из этих классов общедоступными, чтобы их могли использовать другие шаблоны компонентов;
- импортируют другие модули с компонентами, директивами и каналами, необходимыми для компонентов в этом модуле;
- предоставляют сервисы на уровне приложений, которые может использовать любой компонент приложения.

Приложение Angular обычно включает несколько модулей, среди которых есть один корневой модуль (*root module*), который в соответствии с принятым соглашением имеет имя `AppModule`. Например, выше использовался следующий корневой модуль:

```
import . . .;
import { AppComponent } from './app.component';
@NgModule({
  imports:      [ . . . ],
  declarations: [ AppComponent ],
  bootstrap:    [ AppComponent ]
})
export class AppModule { }
```

Для работы модуля ему необходимо подключить ряд библиотек, поэтому в начале файла находятся директивы `import`. Имена библиотек платформы Angular начинаются с префикса *@angular*. Библиотеки устанавливаются для приложения путем вызова в папке приложения пакетного менеджера `npm install -g @angular/библиотека`

В этой части файла мы должны импортировать все модули и классы, которые использует данный модуль. Для `AppModule` необходимы модули:

- *NgModule* — функциональность декоратора `NgModule`, без которой нельзя создать модуль;
- *BrowserModule* — для работы с браузером;
- *FormsModule* — для работы с формами HTML, содержащими элементы `input`;
- *AppComponent* — функциональность корневого компонента приложения. Модуль объявляет класс `AppModule`, который экспортируется для использования исполняющей системой Angular:

```
export class AppModule { }
```

В Angular каждый модуль должен определяться с декоратором *@NgModule*. Декоратор определяет метаданные для модуля. Модуль `NgModule` предоставляет функцию декоратора, свойства которого описывают метаданные модуля. Наиболее важными свойствами являются:

- `declarations` — классы представлений (`view`), которые принадлежат модулю. Angular имеет три типа классов представлений — компоненты (`components`), директивы (`directives`), каналы (`pipes`);
- `exports` — набор классов представлений, которые должны использоваться в шаблонах компонентов из других модулей;
- `imports` — другие модули, классы которых необходимы для шаблонов компонентов из текущего модуля;

- `providers` — классы, создающие сервисы, используемые модулем;
- `bootstrap` — корневой компонент, который вызывается по умолчанию при загрузке приложения.

В примере выше единственным классом представлений является компонент `AppComponent`, поэтому он указывается в свойствах `declarations` и `bootstrap`. И поскольку его действие зависит от модулей `BrowserModule` и `FormsModule`, то данные модули указываются в свойстве `imports`. При необходимости можно использовать и другие свойства.

```
@NgModule({
  imports:      [ BrowserModule, FormsModule ],
  declarations: [ AppComponent ],
  bootstrap:    [ AppComponent ],
  exports:      [ ],
  providers:    [ ]
})
```

7.3. Загрузка приложения

Приложение запускается с помощью файла `main.ts`, который имеет следующее содержимое:

```
import { platformBrowserDynamic } from '@angular/platform-browser-
  dynamic';
import { AppModule } from './app/app.module';
const platform = platformBrowserDynamic();
platform.bootstrapModule(AppModule);
```

Первая строка файла импортирует функциональность модуля `platformBrowserDynamic` из пакета `@angular/platform-browser-dynamic`. Экземпляр, возвращаемый `platformBrowserDynamic`, использует метод `bootstrapModule` для загрузки главного модуля.

Объект `platformBrowserDynamic` запускает импортированный во второй строке модуль `AppModule`. После этого начинается исполнение кода главного модуля приложения `AppModule`.

В модуле AppModule задается имя компонента, используемого в качестве основного при загрузке, для чего используется ключ `bootstrap` декоратора `NgModule`:

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule }   from '@angular/forms';
import { AppComponent }  from './app.component';
@NgModule({
  imports:      [ BrowserModule, FormsModule ],
  declarations: [ AppComponent ],
  bootstrap:   [ AppComponent ]
})
export class AppModule { }
```

В данном случае значение `bootstrap: [AppComponent]` задает для загрузки в качестве основного компонента `AppComponent`. При отсутствии значения параметра `bootstrap: []` или самого параметра `bootstrap` на веб-страницу не будет загружаться по умолчанию представление из компонента `AppComponent`.

7.4. Стили и шаблоны компонента

Для каждого компонента Angular можно определить не только шаблон HTML, но и стили CSS, которые работают с этим шаблоном, указав любые селекторы, правила и медиазапросы, которые вам нужны.

Один из способов сделать это — установить свойство `Styles` в директиве `@Component` компонента. Свойство `Styles` принимает массив строк, которые содержат код CSS. Обычно указывается одна строка, как в следующем примере:

```
styles: ['h1 { font-weight: normal; }']
```

Параметр `styles` содержит перечень стилей, которые будут использоваться компонентом. При использовании стилей следует учитывать, что они применяются локально только к разметке внутри шаблона этого компонента. Это — значительное улучшение модульности по сравнению с тем, как традиционно работает CSS:

- можно использовать имена классов CSS и селекторы, которые имеют наибольшее значение в контексте каждого компонента;
- имена классов и селекторы являются локальными для компонента и не вступают в противоречие с классами и селекторами, используемыми в других местах приложения;
- изменение стилей в другом месте приложения не влияет на стили компонента;
- можно совмещать код CSS каждого компонента с кодом TypeScript и HTML этого компонента, что приводит к аккуратной структуре проекта;
- можно изменить или удалить код CSS компонента, не просматривая все приложение, чтобы найти, где еще используется код.

Если стилей много, то код компонента может быть слишком раздут, и в этом случае их следует вынести в отдельный файл *css*. Параметр *styleUrls* позволяет указать набор файлов *css*, которые применяются для стилизации. Обычно файл *css* в проекте располагается в папке *app*. Полное руководство по обсуждаемым вопросам можно найти в работе [11].

7.5. Привязки

Angular поддерживает механизм привязки фрагментов шаблона к некоторым данным, определенным в компоненте (подстановка значений выражений). В Angular есть четыре формы привязки данных:

- **привязка элемента DOM** к значению компонента (односторонняя). В двойных фигурных скобках указывается выражение, к которому выполняется привязка в виде `{{выражение}}`, например: `<h1>Добро пожаловать {{name}}!</h1>`;
- **привязка свойства элемента HTML** к значению компонента (односторонняя, записывается с использованием квадратных скобок). Например: `<input type="text" [value]='name' />`;
- **привязка метода компонента** (односторонняя) к событию в DOM (генерация события в DOM вызывает метод компонента), например: `<button (click)='addItem (text, price) '>Добавить</button>`;
- **двусторонняя привязка**, когда элемент DOM привязан к отображаемому значению, при этом изменения на одном конце при-

вязки сразу приводят к изменениям на другом конце, например:

```
<input [(ngModel)]='name' placeholder='name'>.
```

На рисунке 16 показаны варианты используемых в Angular привязок. Первый вид привязки использует фигурные скобки, в которые заключается значение из компонента. Например, пусть у нас будет определен следующий компонент:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p>Имя: {{name}}</p>
             <p>Возраст: {{age}}</p>`
})
export class AppComponent {
  name = 'Василий';
  age = 25;
}
```

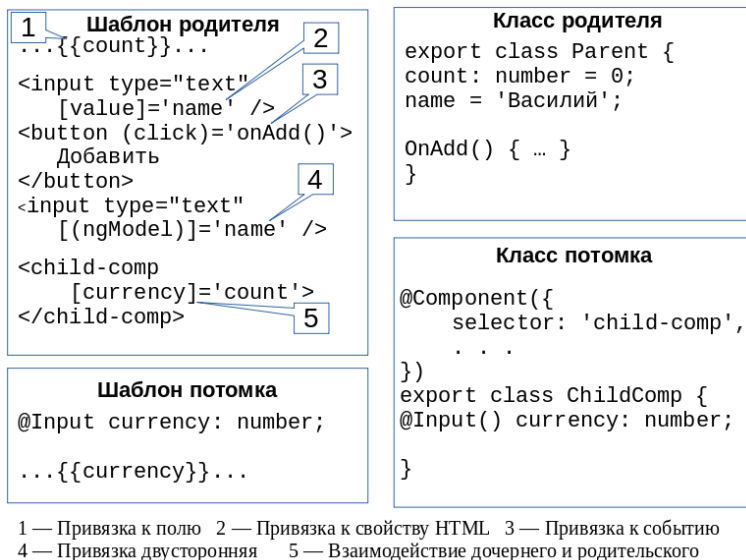


Рис. 16. Схема привязок *Angular*.

1 — привязка к полю; 2 — привязка к свойству HTML; 3 — привязка к событию;
4 — привязка двусторонняя; 5 — взаимодействие дочернего и родительского полей

При запуске приложения выражения типа `{{name}}` будут автоматически заменяться соответствующими значениями, определенными в классе компонента. И если при работе приложения свойства `name`

и `age` в компоненте изменят свое значение, то изменится также значение элементов HTML, которые привязаны к этим свойствам.

Шаблонные выражения. Выражение в шаблоне создает значение и помещается в двойных фигурных скобках `{{}}`. Angular вычисляет выражение и присваивает его свойству цели привязки; целью может быть элемент HTML, компонент или директива.

Интерполяционные скобки `{{v + 1}}` окружают шаблонное выражение `v + 1`. В привязке свойства выражение шаблона помещается в кавычках справа от символа «=», как `[property]="expression"`. С точки зрения синтаксиса шаблонные выражения похожи на JavaScript. Многие выражения JavaScript являются допустимыми шаблонными выражениями, за некоторыми исключениями.

Нельзя использовать выражения JavaScript, которые имеют побочные эффекты или способствуют им, в том числе:

- присваивания (`=`, `+=`, `-=`, `...`);
- операторы, такие как `new`, `typeof`, `instanceof` и т. д.;
- цепные выражения `with`, `or`;
- операторы увеличения и уменьшения `++` и `--`;
- некоторые из операторов ES2015 +.

Другие заметные отличия от синтаксиса JavaScript:

- нет поддержки для побитовых операторов `|` и `&`;
- новые операторы выражения шаблона (`|`, `?`, `!`).

Angular вычисляет шаблонные выражения после каждого цикла обнаружения изменений. Циклы обнаружения изменений запускаются многими асинхронными действиями, такими как разрешение обещаний, результаты `http`, события таймера, нажатие клавиш и перемещение мыши.

Выражение шаблона не должно изменять никакого другого состояния приложения (отсутствие побочных эффектов), кроме значения целевого свойства. Данное правило имеет важное значение для политики Angular — «однонаправленный поток данных», т. е. чтение значения компонента не может изменить другое отображаемое значение. Представление должно быть стабильным в течение одного прохода рендеринга.

Привязка свойств элементов HTML. HTML-элементы могут иметь атрибуты — глобальные, применяемые для всех HTML-элементов, и собственные. Атрибуты прописываются в открывающем теге элемента и содержат имя и значение, указываемые в формате `имя_`

атрибута=«значение». Атрибуты позволяют изменять свойства и поведение элемента, для которого они заданы.

Атрибуты — это HTML-сущности, с помощью которых можно добавить определенные данные к элементам в HTML-коде.

Когда браузер запрашивает некоторую страницу, он получает ее исходный HTML-код. После этого он анализирует этот код и строит на его основе DOM. Во время этого процесса HTML-атрибуты элементов переводятся в соответствующие DOM-свойства. Например, браузер при чтении HTML-строчки кода `` создаст для этого элемента следующие DOM-свойства: `id`, `className`, `src` и `alt`.

Объектная модель документа (DOM) является программным интерфейсом для HTML-, XML- и SVG-документов, что обеспечивает структурированное представление документа (дерева) и определяет способ, по которому структура может быть доступна для программы, для изменения структуры документа, его стиля и содержания. DOM обеспечивает представление документа в виде структурированной группы узлов и объектов, которые имеют свойства и методы. По сути, она связывает веб-страницы со скриптами или языками программирования.

Можно привязать значение объекта экземпляра класса к свойству элемента HTML. В этом случае свойство указывается в квадратных скобках:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<input type="text" [value]='name' />`
})
export class AppComponent {
  name = 'Василий';
}
```

Хотя в этом случае фрагмент можно было записать в форме задания атрибута:

```
template: `<input type="text" value='{{name}}' />`
```

Однако важно понимать, что здесь имеется привязка не к атрибуту, а именно к свойству DOM-элемента, который представляет данный

элемент HTML. Например, можно было привязать некоторое значение к текстовому содержимому элемента:

```
template: `
```

У HTML-элемента `<p>` нет атрибута `textContent`. Но у интерфейса `Node`, который представляет данный элемент DOM, есть свойство `textContent`, к которому можно осуществить привязку.

Привязка к событию. Привязка событий позволяет прослушивать заданные события, такие как нажатие клавиш, движение мыши, щелчок кнопок и касания клавиш. Синтаксис привязки событий Angular состоит из имени **целевого события** в круглых скобках слева от знака равенства и инструкции шаблона справа. Привязка к событию позволяет связать с событием элемента метод-обработчик из числа функций компонента, например:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `

Количество {{count}}</p>
             <button (click)='onAdd()'>Добавить</button>`
})
export class AppComponent {
  count: number = 0;
  onAdd() : void {
    this.count++;
  }
}


```

В шаблоне определен элемент `button`, у которого есть событие `click`. Для обработки данного события в классе `AppComponent` есть метод `onAdd()`, который увеличивает значение переменной `count`. При нажатии на кнопку сработает данный метод.

Привязка передает информацию о событии, которая может включать данные, такие как объект события `$event`, строка или число.

Целевое событие определяет форму объекта `$event`. Если целевое событие является собственным событием элемента DOM, то `$event` — это объект события DOM со свойствами, такими как `target` и `target.value`, например:

```
<input [value]="currentItem.name" (input)="currentItem.name =
    $event.target.value" >
```

Двусторонняя привязка. Двусторонняя привязка дает вашему приложению возможность обмениваться данными между классом компонента и его шаблоном. Двусторонняя привязка позволяет динамически менять значения на одном конце привязки при изменениях на другом. Обычно двусторонняя привязка используется при работе с элементами ввода, например, типа *input*:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p>Привет {{name}}</p>
    <input type="text" [(ngModel)]='name' /> <br/><br/>
    <p>Вы набрали имя: {{name}} </p>`
})
export class AppComponent {
  name: string='Василий';
}
```

Здесь к свойству `name` класса `AppComponent` привязаны сразу три элемента: текст параграфа и два текстовых поля. Текстовые поля связаны со свойством *name* двусторонней привязкой. Для создания двусторонней привязки применяется выражение `[(ngModel)]='выражение'`. В итоге изменения в текстовом поле *name* будут отражаться во втором содержимом параграфа.

Пример, в котором `AppComponent.fontSizePx` — двусторонняя привязка к `SizerComponent`:

```
content_copy<app-sizer [(size)]="fontSizePx"></app-sizer>
<div [style.font-size.px]="fontSizePx">Изменяемый размер
  текста</div>
```

В файле *app.component.ts* компонента `AppComponent` устанавливается *fontSizePx* в начальное значение `SizerComponent.size`:

```
fontSizePx = 16;
```

С помощью кнопок элемента `app-sizer` можно обновить `AppComponent.fontSizePx` — объект двусторонней привязки. Изме-

ненное значение `AppComponent.fontSizePx` попадает в привязку стиля, делая отображаемый текст больше или меньше.

7.6. Работа с компонентами

Приложение Angular на своих страницах может демонстрировать одновременно несколько компонентов. Каждый вспомогательный компонент может занимать на странице отдельный блок для решения некоторой своей задачи, например реклама, вывод справочной информации, ввод данных и др.

Для структуризации проекта целесообразно каждый компонент располагать в отдельной папке внутри `src/app`. Например, шаблон основного компонента `app.component.html` может содержать включение других компонентов:

```
<app-header></app-header>
<router-outlet></router-outlet>
<app-footer></app-footer>
```

Здесь компоненты `app-header` и `app-footer` занимают соответственно верхнюю и нижнюю части страницы. Тег `router-outlet` в центральной части страницы предназначен для размещения внедряемых компонентов с использованием средств навигации. Пример шаблона компонента заголовка `src/app/header/header.component.html` для демонстрации меню приложения с использованием элементов библиотеки Angular Materials может быть следующий:

```
<mat-toolbar color="primary">
  <span></span>
  <a mat-button routerLink="/home" routerLinkActive="active"><span
    class="fa fa-home fa-lg"></span> Домой</a>
  <a mat-button routerLink="/about" routerLinkActive="active"><span
    class="fa fa-info fa-lg"></span> Информация</a>
  <a mat-button routerLink="/menu" routerLinkActive="active"><span
    class="fa fa-list fa-lg"></span> Меню</a>
  <a mat-button routerLink="/contact" routerLinkActive="active"><span
    class="fa fa-address-card fa-lg"></span> Контакты</a>
```

```

<span class="flex-spacer"></span>
<a mat-button (click)="openLoginForm()"><span class="fa
    fa-sign-in fa-lg"></span>Войти</a>
</mat-toolbar>

```

Текст объявления самого компонента `src/app/header/app-header.ts` имеет вид:

```

import { Component, OnInit } from '@angular/core';
import { MatDialog, MatDialogRef } from '@angular/material'; //
import { LoginComponent } from '../login/login.component';
@Component({
  selector: 'app-header',
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.scss']
})
export class HeaderComponent implements OnInit {
  constructor(public dialog: MatDialog) { }
  ngOnInit() {
  }
  openLoginForm() {
    this.dialog.open(LoginComponent, {width: '500px', height: '450px'});
  }
}

```

Для того чтобы использовать все определенные в проекте компоненты, они должны быть указаны в главном модуле приложения в параметре `declarations`. Определим в файле `app.module.ts` следующий модуль:

```

import { NgModule }      from '@angular/core';
import { BrowserModule }  from '@angular/platform-browser';
import { FormsModule }    from '@angular/forms';
import { AppComponent }   from './app.component';
import { HeaderComponent } from './header/header.component';
@NgModule({
  imports:      [ BrowserModule, FormsModule ],
  declarations: [ AppComponent, HeaderComponent ],
  bootstrap:   [ AppComponent ]
})
export class AppModule { }

```

Результат прогона показывает следующее: несмотря на то что один компонент включен в другой с помощью тега `<app-header></app-`

`header>`, стили одного компонента не применяются к другому. Каждый компонент очерчивает свою область с помощью шаблона, свое представление, которым он и управляет. Также каждый компонент использует свои значения свойств, т. е. компоненты фактически существуют информационно относительно независимо.

Элемент **ng-content** определяет внутренность вызванного тега и позволяет родительским компонентам внедрять код HTML в дочерние компоненты. Так, изменим компонент `header.component.html` следующим образом:

```
<mat-toolbar color="primary">
  . . .
</mat-toolbar>
<ng-content></ng-content>
```

Вместо элемента `<ng-content>` извне можно будет передать любое содержимое.

Изменим шаблон основного компонента `app.component.html`:

```
<app-header>
  
</app-header>
<router-outlet></router-outlet>
<app-footer></app-footer>
```

В элемент `<app-header>` здесь передается тег изображения ``. Затем это изображение будет вставлено в дочерний компонент `src/app/header/header.component.html` на место `<ng-content>`. Причем разметкой, которая вставляется в `<ng-content>`, будет управлять `AppComponent`, поэтому именно этот компонент задает стили и выражения привязки для вставляемого фрагмента HTML. Почему так? Автор родительского компонента должен быть уверен в том, что дочерний компонент, используя свои локальные стили и модель данных, не исказит переданный фрагмент.

7.7. Взаимодействие между компонентами

Выше был рассмотрен способ вызова компонента из главного компонента. Однако по умолчанию эти компоненты никак не взаимодействуют, они независимы. Каждый компонент определяет свои выражения привязки. Но если мы хотим свойства дочернего компонента привязать к свойствам из главного компонента, то для этого определим следующий дочерний компонент:

```
import { Input, Component } from '@angular/core';
@Component({
  selector: 'child-comp',
  template: `<p>Имя пользователя: {{userName}}</p>
              <p>Возраст пользователя: {{userAge}}</p>`
})
export class ChildComponent {
  @Input() userName: string;
  @Input() userAge: number;
}
```

Главным здесь является определение входных свойств с помощью декоратора *@Input()*. Для того чтобы использовать декоратор, его надо импортировать из библиотеки Angular:

```
import { Input } from '@angular/core';
```

Основной возможностью таких входных свойств является то, что они могут устанавливаться извне, например из главного компонента.

Теперь изменим код главного компонента:

```
import { Component } from '@angular/core';

@Component({
  selector: 'my-app',
  template: `<child-comp
              [userName]='name' [userAge]='age'></child-comp>
              <input type="text" [(ngModel)]='name' />`
})
export class AppComponent {
  name:string='Василий';
  age:number = 24;
}
```

Поскольку свойство *userName* в дочернем компоненте определено как входное с декоратором `Input`, в главном компоненте можно его использовать как атрибут и применить фактически привязку свойств (см. рис. 16):

```
<child-comp [userName]='name' [userAge]=  
  'age'></child-comp>
```

Аналогично выглядит привязка свойства *userAge*. Описанные выше компоненты определяют связи значений свойств *userAge* и *userName* со значениями из главного компонента.

Привязка к сеттеру. Кроме привязки к свойству мы можем установить привязку к сеттеру дочернего компонента. Напомним, что сеттеры в объектно-ориентированном программировании — это методы экземпляра, позволяющие устанавливать новые значения свойствам экземпляра. Данный метод бывает необходим, когда в дочернем компоненте надо осуществлять проверку или даже изменение значения, получаемого от главного компонента. Например, пусть главный компонент устанавливает возраст пользователя:

```
import { Component } from '@angular/core';  
@Component({  
  selector: 'my-app',  
  template: `<<child-comp [userName]='name' [userAge]='age'>  
    </child-comp>  
    <input type='number' [(ngModel)]='age' />`  
})  
export class AppComponent {  
  name:string='Василий';  
  age:number = 24;  
}
```

а в дочернем компоненте будем получать переданный возраст через сеттер:

```
import { Input, Component } from '@angular/core';  
@Component({  
  selector: 'child-comp',  
  template: `

Имя пользователя: {{userName}}</p>  
  <p>Возраст пользователя: {{userAge}}</p>`  
})


```

```
export class ChildComponent{
  @Input() userName: string;
  _userAge: number;
  @Input()
  set userAge(age:number) {
    if(age<0) this._userAge=0;
    else if(age>100) this._userAge=100;
    else this._userAge = age;
  }
  get userAge() { return this._userAge; }
}
```

В главном компоненте клиент может ввести любое значение в поле ввода, в том числе и отрицательные числа. В дочернем компоненте сеттер оценивает введенное значение и при необходимости его корректирует.

7.8. Привязка к событиям дочернего компонента

Для организации взаимного управления функциональностью различных компонентов Angular предоставляет возможность проследить события дочернего компонента, например, при регистрации или авторизации пользователя. Форму такого взаимодействия представляет привязка к событиям дочернего компонента. Для примера определим следующий дочерний компонент:

```
import { Component, EventEmitter, Input, Output } from
  '@angular/core';
@Component({
  selector: 'child-comp',
  template: `<button (click)='change(+1) '>Добавить</button>
    <button (click)='change(-1) '>Убавить</button>`
})
export class ChildComponent{
  @Output() onChanged = new EventEmitter<number>();
  change(increased:number) {
    this.onChanged.emit(increased);
  }
}
```

В этом компоненте с кнопками связаны события `click`, которые вызывают метод *change*, передавая ему значение `+1` или `-1`. В дочернем компоненте мы можем проследживать и обрабатывать эти события. Но если необходимо передать событие родительскому компоненту, то для этого надо использовать тип *EventEmitter* при определении свойства *onChanged*. Поскольку мы будем передавать числовое значение `+1` или `-1`, данное свойство типизируется типом `number`. При этом свойство *onChanged* должно быть выходным, поэтому оно помечается с помощью декоратора *@Output*.

Фактически свойство *onChanged* будет определять событие, которое обрабатывается в методе *change ()* при нажатии на кнопки «Добавить» и «Убавить». Результаты обработки передаются главному компоненту.

Далее определим код главного компонента:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<h2>Количество кликов: {{clicks}}</h2>
    <child-comp (onChanged)='onChanged($event)'></child-comp>`
})
export class AppComponent {
  clicks:number = 0;
  onChanged(increased:number){
    increased==+1?this.clicks++:this.clicks--;
  }
}
```

С помощью выражения *(onChanged)='onChanged(\$event)'* выполняется привязка метода *onChanged* к событию *onChanged ()*, происходящему в дочернем компоненте. Параметр *\$event* содержит все данные, которые передаются из дочернего компонента о событии.

В результате нажатия на кнопки «Добавить» и «Убавить» в дочернем компоненте обработчик событий главного компонента будет увеличивать или уменьшать значение счетчика *clicks*.

Двусторонняя привязка. В примере выше мы определяли привязку к событию дочернего компонента: при возникновении события в дочернем компоненте обрабатывалось это событие в главном компоненте с помощью метода *onChanged*. Но также можно использовать двустороннюю привязку между свойствами главного и дочерне-

го компонента. Например, пусть дочерний компонент будет иметь следующий вид:

```
import { Component, Input, Output, EventEmitter } from
    '@angular/core';
@Component({
    selector: 'child-comp',
    template: `<input [ngModel]='userName'
        (ngModelChange)='onNameChange($event)' />`
})
export class ChildComponent{
    @Input() userName:string;
    @Output() userNameChange = new EventEmitter<string>();
    onNameChange(modelName: string){
        this.userName = modelName;
        this.userNameChange.emit(modelName);
    }
}
```

Здесь определено входное свойство `userName`, к которому привязано текстовое поле *input*. Для связи используется атрибут `[ngModel]`, который связывает значение атрибута `value` у текстового поля со свойством `userName`.

Для отслеживания изменения модели этого поля с помощью атрибута `(ngModelChange)` привязываем метод, который срабатывает при изменении значения, т.е. `ngModelChange` — это фактически событие изменения введенного значения, поэтому здесь действует привязка к событию.

Поскольку рассматривается односторонняя привязка, в методе-обработчике получаем введенное значение, изменяем свойство `userName` и генерируем событие `userNameChange`, которое определено как выходной параметр.

Таким образом, здесь извне получаем значение для свойства `userName` и устанавливаем его для текстового поля. При изменении пользователем значения в этом поле генерируется событие `userNameChange`.

Код главного компонента будет иметь вид:

```
import { Component } from '@angular/core';

@Component({
    selector: 'my-app',
```

```
template: `<child-comp [(userName)]='oldName'></child-comp>
          <div>Выбранное имя: {{oldName}}</div>`
  })
  export class AppComponent {
    oldName: string = 'Василий';
  }
```

Здесь устанавливается двусторонняя привязка свойств `userName` дочернего компонента и свойства `oldName` главного компонента. При этом не надо уже вручную обрабатывать событие `userNameChange`, все будет делаться автоматически.

7.9. Жизненный цикл компонента

Директивы и экземпляры компонентов имеют жизненный цикл, за который Angular создает, обновляет и уничтожает их. Разработчики могут использовать ключевые моменты в этом жизненном цикле, внедрив одну или несколько ловушек жизненного цикла из библиотеки `core Angular`.

Каждая ловушка имеет единственный метод вызова, именем которого является имя интерфейса с префиксом `ng`. Angular вызывает метод ловушки директивы (компонента), только если он определен. Интерфейс `OnInit` имеет обработчик `ngOnInit()`, который вызывается после создания компонента, например:

```
export class PeekABoo implements OnInit {
  constructor(private logger: LoggerService) { }
  // реализует интерфейс OnInit метод `ngOnInit`
  ngOnInit() { this.logIt(`OnInit`); }
  logIt(msg: string) {
    this.logger.log(`#${nextId++} ${msg}`);
  }
}
```

После создания компонента фреймворк Angular вызывает у этого компонента ряд методов, которые соответствуют различным этапам жизненного цикла:

- *ngOnChanges()* — вызывается до метода *ngOnInit()* и всякий раз, когда изменяется одно или несколько связанных со свойством данных ввода, когда Angular устанавливает или переустанавливает входные свойства, связанные с данными. Метод получает объект *SimpleChanges* с текущими и предыдущими значениями свойств;
- *ngOnInit()* — инициализирует директиву (компонент) после того, как Angular впервые отобразит связанные с данными свойства и установит входные свойства директивы (компонента). Вызывается один раз после первого *ngOnChanges()*;
- *ngDoCheck()* — вызывается при каждой проверке изменений свойств компонента сразу после методов *ngOnChanges* и *ngOnInit*. Обнаруживает и реагирует на изменения, которые Angular не сможет обнаружить самостоятельно;
- *ngAfterContentInit()* — вызывается после того, как *Angular* вставляет внешнее содержимое в представление компонента. Вызывается один раз после первого *ngDoCheck()*;
- *ngAfterContentChecked()* — вызывается фреймворком Angular при проверке изменений содержимого, которое добавляется в представление компонента. Вызывается после метода *ngAfterContentInit()* и после каждого последующего вызова метода *ngDoCheck()*;
- *ngAfterViewInit()* — вызывается фреймворком Angular после того, как инициализируются представления главного и дочернего компонентов. Вызывается один раз после первого *ngAfterContentChecked()*;
- *ngAfterViewChecked()* — вызывается фреймворком Angular после проверки на изменения в представлении компонента, а также проверки представлений дочерних компонентов. Вызывается после первого вызова метода *ngAfterViewInit()* и после каждого последующего вызова *ngAfterContentChecked()*;
- *NgOnDestroy* — очистка перед тем, как Angular уничтожит директиву (компонент). Отменяет подписку *Observables* и отсоединяет обработчики событий, чтобы избежать утечек памяти. Вызывается непосредственно перед тем, как Angular уничтожит директиву (компонент).

Каждый такой метод определен в отдельном интерфейсе, который называется по имени метода без префикса 'ng'. Например, метод

`ngOnInit` определен в интерфейсе `OnInit`, поэтому, если мы хотим отслеживать какие-то этапы жизненного цикла компонента, класс компонента должен реализовывать соответствующие интерфейсы:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p>Hello Angular 2</p>`
})
export class AppComponent implements OnInit, OnDestroy {
  name:string='Василий';
  constructor(){ this.log(`constructor`); }
  ngOnInit() { this.log(`onInit`); }
  ngOnDestroy() { this.log(`onDestroy`); }
  private log(msg: string) {
    console.log(msg);
  }
}
```

Метод `ngOnInit`. Метод `ngOnInit()` вызывается после создания экземпляра конструктором класса для дополнительной комплексной инициализации компонента. Здесь можно выполнять загрузку данных с сервера или из других источников данных. Конструктор также может выполнять незначительную инициализацию объекта, в то же время что-то более сложное в конструкторе делать не рекомендуется. Конструктор должен быть по возможности простым и выполнять базовую инициализацию. Что-то более сложное, например загрузку данных с сервера, которая может занять продолжительное время, лучше делать методом `ngOnInit`. Конструктор используется для инъекции зависимостей через его параметры. Такими зависимостями обычно являются экземпляры сервисов, классов с описанием данных, библиотечных объектов и др., например:

```
constructor(public dialog: MatDialog) { }
```

Объявленные в конструкторе параметры становятся свойствами экземпляра класса и доступны в методах с использованием префикса `this`, например `this.dialog`.

Метод `ngOnInit()` вызывается только один раз!

Метод `ngOnDestroy`. Метод `ngOnDestroy()` вызывается перед удалением компонента. В этом методе можно освобождать те использу-

емые ресурсы, которые не удаляются автоматически сборщиком мусора, для предотвращения утечки памяти. Здесь также можно удалять подписку на какие-то события элементов DOM, останавливать таймеры, отписаться от событий Observables и DOM, отменить регистрацию всех обратных вызовов, которые зарегистрированы в глобальных службах или службах приложений.

Метод ngOnChanges. Angular вызывает метод ngOnChanges(), когда обнаруживается изменение свойств в привязке к входной переменной с декоратором @Input. С помощью параметра метода типа SimpleChanges в методе можно получить текущее и предыдущее значение измененного свойства. Например, пусть у нас есть следующий дочерний компонент:

```
import { Component, Input, OnInit, OnChanges, SimpleChanges }
  from '@angular/core';
@Component({
  selector: 'child-comp',
  template: `

Привет {{name}}

`})
export class ChildComponent implements OnInit, OnChanges {
  @Input() name: string;
  constructor(){ this.log(`constructor`); }
  ngOnInit() { this.log(`onInit`); }
  ngOnChanges(changes: SimpleChanges) {
    for (let propName in changes) {
      let chng = changes[propName];
      let cur = JSON.stringify(chng.currentValue);
      let prev = JSON.stringify(chng.previousValue);
      console.log(`${propName}: currentValue = ${cur},
        previousValue = ${prev}`);
    }
  }
}
```

И пусть этот компонент используется в главном компоненте:

```
import { Component, OnChanges, SimpleChanges } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `

```

```
export class AppComponent implements OnChanges {
  name:string='Василий';
  age:number = 25;
  ngOnChanges(changes: SimpleChanges) {
    for (let propName in changes) {
      let chng = changes[propName];
      let cur = JSON.stringify(chng.currentValue);
      let prev = JSON.stringify(chng.previousValue);
      console.log(`${propName}: currentValue = ${cur},
        previousValue = ${prev}`);
    }
  }
}
```

Значение для свойства `name` передается в дочерний компонент `ChildComponent` из главного компонента `AppComponent`, причем в главном компоненте тоже реализован метод `ngOnChanges()`.

При запуске приложения можно увидеть, что при каждом изменении свойства `name` в главном компоненте вызывается метод `ngOnChanges`. Данный метод вызывается только при изменении свойств с декоратором `@Input`, поэтому изменение свойства `age` в `AppComponent` в этом примере не будет прослеживаться.

Метод `DoCheck()`. `DoCheck` используется, чтобы обнаруживать и реагировать на изменения, которые Angular не улавливает самостоятельно. `DoCheck` расширяет метод `OnChanges`:

```
// DoCheckComponent (ngDoCheck)
ngDoCheck() {
  . . .
  if (this.changeDetected) {
    this.noChangeCount = 0;
  } else {
    // log that hook was called when there was no relevant
    change.
    let count = this.noChangeCount += 1;
    let noChangeMsg = `DoCheck called ${count}x when no change to
    hero or power`;
    if (count === 1) {
      // добавить новое сообщение "no change"
      this.changeLog.push(noChangeMsg);
    } else {
      // заменить последнее сообщение на "no change"
```

```

        this.changeLog[this.changeLog.length - 1] = noChangeMsg;
    }
}
this.changeDetected = false;
}

```

Такой код проверяет определенные значения, захватывая и сравнивая их текущее состояние с предыдущими значениями. Он записывает специальное сообщение в журнал, если нет существенных изменений, чтобы можно было видеть, как часто вызывается `DoCheck`.

Хотя метод `ngDoCheck()` может определять момент изменения значения, он имеет большие затраты. Данный метод вызывается с большой частотой — после каждого цикла обнаружения изменений, независимо от того, где изменение произошло. В этом примере он вызывается более двадцати раз, прежде чем пользователь сможет что-либо сделать.

Переменные шаблона. Шаблонные переменные позволяют определить некоторые переменные внутри шаблона компонента и затем ссылаться на эти переменные из этого же шаблона. Для определения подобных переменных применяется знак решетки (`#`). Например, определим шаблонную переменную `userName` в компоненте:

```

import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `
      <p #userName>{{name}}</p>
      <p>{{userName.textContent}}</p>
      <input type='text' [(ngModel)]= 'name' />`
})
export class AppComponent {
  name:string='Василий';
}

```

Определение переменной может выглядеть следующим образом:

```
<p #userName>{{name}}</p>
```

Определение переменной `userName` в элементе `p` означает, что она будет ссылаться на данный фрагмент, т. е. на `p` разметки HTML. И далее можно обращаться к этому фрагменту через данную переменную. Например, используя свойство `userName.textContent` можно полу-

чить текстовое содержимое *p*. При этом если привязанное к фрагменту значение переменной *name* изменится, то соответственно изменится и значение *userName.textContent*.

Следует иметь в виду, что данную переменную можно использовать *только внутри* шаблона. Использование шаблонных переменных — дополнительный способ взаимодействия между родительским и дочерним компонентом. Например, определим следующий дочерний компонент *ChildComponent*:

```
import { Component } from '@angular/core';
@Component({
  selector: 'child-comp',
  template: `<p>{{counter}}</p>`
})
export class ChildComponent {
  count: number = 0;
  increment() { this.count++; }
  decrement() { this.count--; }
}
```

В этом компоненте определяется переменная счетчика *counter*. Для ее увеличения или уменьшения применяются методы *increment* и *decrement*.

В коде главного компонента будем вызывать дочерний компонент:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<child-comp #counter></child-comp>
    <button (click)='counter.increment()'>+</button>
    <button (click)='counter.decrement()'>-</button>`
})
export class AppComponent { }
```

В данном случае шаблонная переменная *counter* определена внутри тега *<child-comp>*, поэтому она будет представлять компонент *ChildComponent*. Следовательно, далее можно ссылаться на компонент *ChildComponent* с помощью данной переменной, например установить для кнопок привязку к методам дочернего компонента *ChildComponent*. В результате по нажатию на кнопки в главном компоненте будут вызываться методы из дочернего компонента.

Декоратор *ViewChild*. Однако шаблонные переменные имеют свои ограничения: они не могут применяться вне шаблона, даже в коде класса компонента, например нельзя написать так:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<child-comp #counter></child-comp>
    <button (click)='increment()'>+</button>
    <button (click)='decrement()'>-</button>`
})
export class AppComponent {

  increment() { this.counter++; }
  decrement() { this.counter--; }
}
```

Здесь для класса *AppComponent* свойство *this.counter* не существует — оно существует только для шаблона.

Для того чтобы иметь возможность обращаться к методам и прочей функциональности дочернего компонента, надо использовать декоратор *ViewChild*. Итак, изменим главный компонент следующим образом:

```
import { Component, ViewChild } from '@angular/core';
import { ChildComponent } from './child.component';
@Component({
  selector: 'my-app',
  template: `<child-comp></child-comp>
    <button (click)='increment()'>+</button>
    <button (click)='decrement()'>-</button>`
})
export class AppComponent {
  @ViewChild(ChildComponent)
  private counterComponent: ChildComponent;
  increment() { this.counterComponent.increment(); }
  decrement() { this.counterComponent.decrement(); }
}
```

С помощью декоратора *ViewChild* к свойству *counterComponent* устанавливаем, что это свойство будет содержать объект дочернего компонента, который внедряется через элемент *<child-comp></child-comp>*. И в этом случае уже можно не использовать шаблонные переменные в шаблоне.

Привязка ViewChild к переменным шаблона. Несмотря на то что выше не были использованы переменные, с помощью декоратора ViewChild также можно связать свойство и переменную из шаблона. Итак, изменим код главного компонента:

```
import { Component, ViewChild, ElementRef } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p #nameText>{{name}}</p>
              <p>{{nameText.textContent}}</p>
              <button (click)='change()'>Изменить</button>`
})
export class AppComponent {
  @ViewChild('nameText')
  nameParagraph: ElementRef;
  name: string = 'Василий';
  change() {
    console.log(this.nameParagraph.nativeElement.textContent);
    this.nameParagraph.nativeElement.textContent = 'Петр';
  }
}
```

Здесь в шаблоне определяется переменная nameText, которая представляет код параграфа, а в декораторе ViewChild передается имя этой переменной, поэтому свойство nameParagraph, к которому применяется декоратор, будет указывать на эту переменную nameText. Причем свойство nameParagraph имеет тип ElementRef, который используется для ссылки на элементы HTML. После нажатия на кнопку выводится и изменяется текстовое содержимое этой переменной.

Декоратор ContentChild. Кроме ViewChild для связи с шаблонными переменными мы можем применять другой декоратор — ContentChild. В какой ситуации он может понадобиться? Допустим, что в родительском компоненте определен следующий код:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<child-comp>
              <h3 #headerContent>Добро пожаловать {{name}}!</h3>
            </child-comp>`
})
export class AppComponent {
  name: string = 'Василий';
}
```

Здесь определена переменная `#headerContent`, которая указывает на элемент заголовка `h3`. Причем поскольку данные из родительского компонента передаются в дочерний напрямую, для получения этих данных в дочернем компоненте будет использоваться элемент `ng-content`:

```
import { Component, ContentChild, ElementRef } from '@angular/core';
@Component({
  selector: 'child-comp',
  template: `<ng-content></ng-content>
             <button (click)='change()'>Изменить</button>`
})
export class ChildComponent {
  @ContentChild('headerContent')
  header: ElementRef;
  change() {
    console.log(this.header);
    this.header.nativeElement.textContent = 'Привет миру!';
  }
}
```

Для того чтобы получить переменные, которые передаются с кодом через `ng-content`, дочерний компонент применяет декоратор `ContentChild`. В этот декоратор также передается название переменной. Само свойство декоратора также представляет объект класса `ElementRef`. И далее мы можем манипулировать этим объектом.

Контрольные вопросы

1. Какой стиль кодирования компонент используется платформой *Angular* — процедурный или декларативный?
2. Где и когда компонент появляется на странице приложения?
3. Связаны ли привязки с передачей данных между определением класса и страницей на экране дисплея?
4. Есть ли возможность завершить работу приложения и закрыть окно браузера в ответ на нажатие кнопки?
5. Кто распоряжается содержимым тега `<router-outlet>`?

8. Директивы

Angular предлагает два вида встроенных директив: директивы атрибутов и структурные директивы.

Директивы определяют набор инструкций, которые применяются при рендеринге (визуализации) HTML-кода. Директива представляет класс с директивными метаданными. В TypeScript для прикрепления метаданных к классу применяется декоратор *@Directive*.

В Angular есть три типа директив:

- компоненты — компонент по сути также является директивой, а декоратор *@Component* расширяет возможности декоратора *@Directive* с помощью добавления функционала по работе с шаблонами;
- атрибутивные — изменяют поведение уже существующего элемента, к которому они применяются, например `ngModel`, `ngStyle`, `ngClass`;
- структурные — изменяют структуру DOM с помощью добавления, изменения или удаления элементов HTML, например, директивы `ngIf` и `ngFor`.

8.1. Структурные директивы

Структурные директивы отвечают за макет HTML. Они формируют или изменяют структуру DOM, обычно добавляя, удаляя и манипулируя элементами страницы, к которым они присоединены. Есть три структурных директивы: `ngIf`, `ngSwitch` и `ngFor`.

Директива `ngIf`. Директива `ngIf` — условная директива, позволяющая скрыть или, наоборот, визуализировать элемент при заданном в директиве условии. Например, создадим следующий компонент:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p *ngIf='condition'>
    Привет мир!
  </p>
  <p *ngIf='!condition'>
    До свидания, мир!
  </p>
  <button (click)='toggle()'>Переключить</button>`
})
export class AppComponent {
  condition: boolean=true;
  toggle(){
    this.condition = !this.condition;
  }
}
```

Здесь в зависимости от значения свойства `condition` будет отображаться либо первый, либо второй параграф.

Начиная с версии Angular 4.0 можно задавать альтернативные выражения с помощью директивы `ng-template`. Так, предыдущий пример будет аналогичен следующему определению компонента:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<p *ngIf='condition;else unset'>
    Привет мир!
  </p>
  <ng-template #unset>
    <p>До свидания, мир!</p>
  </ng-template>
  <button (click)='toggle()'>Переключить</button>`
})
export class AppComponent {
  condition: boolean=true;
  toggle(){
    this.condition = !this.condition;
  }
}
```

Выражение `*ngIf="condition; else unset"` означает, что если `condition` равно `false`, то срабатывает блок `<ng-template #unset>`, либо можно определить более тонкую логику, изменив шаблон компонента следующим образом:

```
template: ` <div *ngIf='condition; then thenBlock else elseBlock'></div>
           <ng-template #thenBlock>Then template</ng-template>
           <ng-template #elseBlock>Else template</ng-template>
           <button (click)='toggle()'>Переключить</button>`
```

Здесь если `condition` равно `true`, то отображается блок `thenBlock`, иначе — отображается блок `elseBlock`. Следует отметить, что управлять видимостью можно целиком у компонента, например:

```
<item-detail *ngIf="isActive"></item-detail>
```

Директива `ngFor`. Директива цикла `ngFor` позволяет перебирать в шаблоне элементы массива. Например:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<ul>
                <li *ngFor='let item of items'>{{item}}</li>
            </ul>`
})
export class AppComponent {
  items = ['ПК', 'Ноутбук', 'Планшет', 'Смартфон'];
}
```

В качестве значения директива принимает значение цикла типа `foreach: let item of items`. Каждый перебираемый элемент на очередной итерации цикла помещается в переменную `item`, которую можно вывести на страницу в примере как элемент списка.

При переборе элементов доступен текущий индекс элемента через переменную `index`, которую также можно использовать, например:

```
<div>
  <p *ngFor='let item of items; let i = index'>{{i+1}}. {{item}}</p>
</div>
```

Надо учитывать, что индексация начинается с **нуля**, поэтому чтобы в данном случае отсчет шел с единицы, к переменной *i* прибавляется единица.

Символы звездочки. Обращает внимание, что при использовании директив `ngFor` и `ngIf` перед именем ставится символ «звездочка», который упрощает применение директивы. Например, определение `ngIf` может выглядеть так:

```
<p *ngIf='condition'>
  Привет мир
</p>
<p *ngIf='!condition'>
  До свидания, мир!
</p>
```

а по факту будет представлять следующий код:

```
<template [ngIf]='condition'>
  <p> Привет мир </p>
</template>
<template [ngIf]='!condition'>
  <p> До свидания, мир! </p>
</template>
```

В итоге фрагмент и его текст перемещаются внутрь элемента `<template>`. Сама директива тоже помещается в тег `<template>`, в котором применяется привязка свойства. Булево значение привязанного свойства указывает на то, надо ли отображать соответствующий контент.

В итоге можно выбирать либо первый способ со звездочкой, который более компактный, либо второй способ с элементами `template`.

То же самое касается и директивы `ngFor`:

```
<ul>
  <li *ngFor='let item of items'>{{item}}</li>
</ul>
```

Данный код будет эквивалентен следующему:

```
<ul>
  <template ngFor let-item [ngForOf]='items'>
```

```
    <li>{{item}}</li>
  </template>
</ul>
```

Директива ngSwitch. Директива реализует классический переключатель фрагментов в процедурных языках программирования. С помощью директивы ngSwitch можно встроить в шаблон конструкцию `switchcase` и в зависимости от результата ее выполнения выводить тот или иной блок, например:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<div [ngSwitch]='count'>Дальность:
    <ng-template *ngSwitchCase='1'>{{count * 10}}</ng-template>
    <ng-template *ngSwitchCase='2'>{{count * 100}}</ng-template>
    <ng-template ngSwitchDefault>{{count * 1000}}</ng-template>
  </div>`
})
export class AppComponent {
  count: number = 5;
}
```

Директива ngSwitch в качестве значения принимает некоторое выражение — в данном случае это свойство `count`. В элемент `ng-template` помещается вариант инструкции `*ngSwitchCase`, который сравнивает значение выражения из ngSwitch с указанным выражением. Если оба выражения равны, то используется данный элемент `ng-template`. Если иначе — выполнение переходит к следующим инструкциям `ngSwitchCase`. Если же ни одна из инструкций `ngSwitchCase` не была выполнена, то вызывается инструкция `ngSwitchDefault`.

8.2. Директивы атрибутов

Angular предлагает два вида встроенных директив — директивы атрибутов и структурные директивы. Директивы атрибутов слушают и изменяют поведение других HTML-элементов, атрибутов, свойств и компонентов. Вы обычно применяете их к элементам, как если бы они были атрибутами HTML, отсюда и название.

Многие NgModules, такие как RouterModule и FormsModule определяют свои собственные директивы атрибутов. Наиболее распространенные атрибуты:

- NgClass — добавляет и удаляет набор классов CSS;
- NgStyle — добавляет и удаляет набор стилей HTML;
- NgModel — добавляет двустороннюю привязку данных к элементу форма HTML.

Директива NgClass. Добавляет или удаляет несколько классов CSS одновременно. Рассмотрим фрагмент app.component.html:

```
<!-- toggle the "special" class on/off with a property -->
<div [ngClass]="isSpecial ? 'special' : ''">Это специальный раздел</div>
```

Для того чтобы добавить или удалить отдельный класс, используйте привязку класса, а не директиву NgClass.

Рассмотрим метод setCurrentClasses() компонента, который устанавливает свойство компонента currentClasses, с объектом, который добавляет или удаляет три класса на основе состояния true/false трех других свойств компонента. Каждый ключ объекта является именем класса CSS: его значение true — если класс должен быть добавлен, false — если он должен быть удален.

```
currentClasses: {};
setCurrentClasses() {
  // CSS classes: added/removed текущее состояние свойств компонента
  this.currentClasses = {
    'saveable': this.canSave,
    'modified': !this.isUnchanged,
    'special': this.isSpecial
  };
}
```

Директива NgStyle. Используется NgStyle для одновременной и динамической установки множества встроенных стилей в зависимости от состояния компонента.

Ниже приведен метод setCurrentStyles(), который устанавливает свойство компонента currentStyles с помощью объекта, который определяет три стиля на основе состояния трех других свойств компонента:

```
currentStyles: {};  
setCurrentStyles() {  
  // CSS styles: установить текущее состояние свойств компонента  
  this.currentStyles = {  
    'font-style': this.canSave      ? 'italic' : 'normal',  
    'font-weight': !this.isUnchanged ? 'bold'   : 'normal',  
    'font-size':   this.isSpecial   ? '24px'   : '12px'  
  };  
}
```

Добавление привязки свойства `ngStyle` для установки стилей `currentStyles` элемента имеет следующий вид:

```
<div [ngStyle]="currentStyles"> Этот элемент div первоначально имеет  
  стили italic, normal weight, extra large (24px). </div>
```

Контрольные вопросы и задание

1. Зачем нужны директивы при рендеринге (визуализации) HTML-кода?
2. Атрибутивные директивы изменяют поведение уже существующего элемента, к которому они применяются. Найдите примеры в коде.
3. Структурные директивы изменяют структуру документа с помощью добавления, изменения или удаления элементов HTML. Найдите примеры в коде.

9. Сервисы и инъекции зависимостей

Сервисы в *Angular* образуют довольно широкий набор классов, которые выполняют некоторые фиксированные задачи, например, логирование, работу с базами данных, хранение и предоставление глобальной информации и т. д.

В отличие от компонентов и директив сервисы не работают с представлениями, т. е. с разметкой HTML, не оказывают на нее прямого влияния. Они выполняют строго определенные и достаточно узкие задачи:

- доставка данных приложению — сервис может сам хранить данные в памяти либо для получения данных может обращаться к какому-нибудь источнику данных, например к серверу БД;
- сервис может предоставлять общие (глобальные для приложения) данные для взаимодействия между отдельными компонентами приложения;
- сервис может выполнять бизнес-логику, различные вычислительные задачи, задачи по журнализации, которые лучше выносить из компонентов. Тем самым код компонентов будет сосредоточен непосредственно на работе с представлением. Кроме того, также можно решить проблему повторного использования кода, если потребуется выполнить одну и ту же задачу в разных компонентах и классах.

Пусть у нас есть классическая базовая структура проекта. В папке *shared* будем хранить объявления вспомогательных классов: сущности, константы и др. Добавим в папку *src/app/shared* новый файл, который назовем *phone.ts*. Определим в этом файле следующий класс:

```
export class Phone{
  constructor(public name: string, public price: number) { }
}
```

Данный класс будет представлять сущности, с которыми мы будем работать.

В папке *services* будем хранить объявления сервисов. Далее добавим в папку *src/app/services* новый файл *data.service.ts*, который будет содержать код сервиса. Согласно соглашениям о названиях файлов после названия самого сервиса должен идти суффикс *.service*. Если название сервиса состоит из нескольких слов, которые начинаются с большой буквы, то в названии файла все эти слова пишутся с маленькой буквы и разделяются дефисами. Например, если сервис называется *SpecialSuperService*, то файл сервиса будет называться *special-super.service.ts*.

Определим в этом файле следующий сервис:

```
import {Phone} from './shared/phone';
export class DataService{
  private data: Phone[] = [
    { name:'Айфон', price: 56000},
    { name: 'Хайвей', price: 56000},
    { name: 'Самсунг', price: 25000}
  ];
  getData(): Phone[] {
    return this.data;
  }
  addData(name: string, price: number){
    this.data.push(new Phone(name, price));
  }
}
```

В сервисе определен массив телефонов и методы для работы с ним. В реальном приложении эти данные можно получать с сервера или из какого-либо внешнего хранилища.

Теперь определим код компонента:

```
import { Component, OnInit } from '@angular/core';
import { DataService } from './services/data.service';
import { Phone } from './shared/phone';
@Component({
```

```

selector: 'my-app',
template: `<div class="panel">
  <div class="form-inline">
    <div class="form-group">
      <div class="col-md-8">
        <input class="form-control" [(ngModel)]='name'
placeholder = 'Модель' />
      </div>
    </div>
    <div class="form-group">
      <div class="col-md-6">
        <input type="number" class="form-control"
[(ngModel)]='price' placeholder='Цена' />
      </div>
    </div>
    <div class="form-group">
      <div class="col-md-offset-2 col-md-8">
        <button class="btn btn-default"
(click)='addItem(name, price)'>Добавить</button>
      </div>
    </div>
  </div>
  <table class="table table-striped">
    <thead>
      <tr>
        <th>Модель</th>
        <th>Цена</th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor='let item of items'>
        <td>{{item.name}}</td>
        <td>{{item.price}}</td>
      </tr>
    </tbody>
  </table>
</div>`,
providers: [DataService]
})
export class AppComponent implements OnInit {
  items: Phone[] = [];
  constructor(private dataService: DataService) {}
  addItem(name: string, price: number) {
    this.dataService.addData(name, price);
  }
}

```

```
ngOnInit() {  
    this.items = this.dataService.getData();  
}  
}
```

Для того чтобы задействовать сервис в компоненте, его необходимо не только импортировать: `import {DataService} from './services/data.service'`; но и добавить в коллекцию `providers` компонент `providers: [DataService]`.

Все используемые сервисы должны быть перечислены в коллекции параметра *providers*, после чего можно задействовать встроенный в Angular механизм `dependency injection` и получить объект сервиса в конструкторе компонента, а затем использовать его по необходимости:

```
constructor (private dataService: DataService) {}
```

Для загрузки данных обычно реализуется метод компонента `ngOnInit()` интерфейса `OnInit`, который вызывается после конструктора.

Поскольку используется директива `ngModel` для работы с элементами формы, надо подключить модуль *FormsModule* в файле *app.module.ts*:

```
import { NgModule }      from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { FormsModule }   from '@angular/forms';  
import { AppComponent }  from './app.component';  
@NgModule({  
    imports:      [ BrowserModule, FormsModule ],  
    declarations: [ AppComponent ],  
    bootstrap:   [ AppComponent ]  
})  
export class AppModule { }
```

В итоге можно выводить данные и добавлять новые элементы с помощью сервиса.

Внедрение сервиса в другой сервис

Вполне вероятна ситуация, когда необходимо использовать один сервис в другом сервисе, например ранее был создан сервис для работы с данными. Для логирования всех операций с данными определим

новый сервис, для чего добавим в папку *src/app/services* новый файл *log.service.ts* со следующим содержимым:

```
export class LogService{
  write(logMessage:string){
    console.log(logMessage);
  }
}
```

Для логирования в сервисе определен метод `write`, который выводит некоторое сообщение на консоль. Для использования данного сервиса изменим код в файле *data.service.ts*:

```
import {Injectable} from '@angular/core';
import {Phone} from '../shared/phone';
import {LogService} from '../services/log.service';
@Injectable()
export class DataService{
  private data: Phone[] = [
    { name:'Айфон', price: 56000},
    { name: 'Хайвей', price: 56000},
    { name: 'Самсунг', price: 25000}
  ];
  constructor(private logService: LogService){}
  getData(): Phone[] {
    this.logService.write('Операция получения данных');
    return this.data;
  }
  addData(name: string, price: number){
    this.data.push(new Phone(name, price));
    this.logService.write('Операция добавления данных');
  }
}
```

Для того чтобы указать, что сервис сам может использовать другие сервисы, к классу сервиса применяется декоратор `Injectable`. Если класс не будет иметь подобного декоратора, то встроенный механизм внедрения зависимостей не сможет создать объект этого класса и выдаст ошибку.

Существуют общие рекомендации для разработчиков Angular — применять `Injectable` к любому классу сервиса, хотя в принципе это необязательно. Ранее использовался сервис в компоненте без при-

менения к компоненту декоратора `Injectable`. Дело в том, что декоратор `Component`, который применяется к компоненту, является подтипом `Injectable`. В случае с `DataService` сервис `LogService` также надо зарегистрировать в списке провайдеров `AppComponent`:

```
import { Component, OnInit } from '@angular/core';
import { DataService } from '../sources/data.service';
import { LogService } from '../sources/log.service';
import { Phone } from '../shared/phone';
@Component({
  selector: 'my-app',
  template: `<div class="panel">
.....
    </div>`,
  providers: [DataService, LogService]
})
export class AppComponent implements OnInit {
  items: Phone[] = [];
  constructor(private dataService: DataService) {}
  addItem(name: string, price: number) {
    this.dataService.addData(name, price);
  }
}
```

Несмотря на то что напрямую `LogService` не используется в компоненте `AppComponent`, он используется в `DataService`, который вызывается в `AppComponent`.

Контрольные вопросы

1. В чем существенное отличие компонента от сервиса?
2. Зачем в описание компонента включается фрагмент `providers: [DataService]`?
3. Сервисы для получения данных обращаются к серверу баз данных или к серверу микросервисной архитектуры?
4. Что обеспечивает класс `Observable`?
5. Где сервис ищет данные?

10. Работа с формами

Обработка пользовательского ввода с помощью форм является краеугольным камнем многих распределенных приложений. Приложения используют формы, чтобы пользователи могли войти в систему, обновить профиль, ввести конфиденциальную информацию и выполнить множество других задач ввода данных.

Angular предоставляет два разных подхода к обработке пользовательского ввода с помощью форм, управляемых шаблоном, и реактивных форм. Оба способа обрабатывают события ввода данных пользователем, проверяют пользовательский ввод, создают модель формы и модель данных для обновления и предоставляют инструменты отслеживания изменений. Реактивные формы и формы на основе шаблонов получают и обрабатывают данные формы по-разному. Каждый подход предлагает разные преимущества.

Реактивные формы обеспечивают прямой явный доступ к базовой объектной модели форм. В сравнении с формами на основе шаблонов они более надежны, более масштабируемы, многократно используются и хорошо протестированы. Если формы являются важной частью вашего приложения, используйте реактивные формы.

Формы, управляемые шаблоном, полагаются на директивы в шаблоне для создания и управления базовой объектной моделью. Они полезны для добавления в приложение простой формы, например формы регистрации в списке рассылки. Их легко добавить в приложение, но они не так масштабируются, как реактивные формы. Если существуют базовые требования к форме и логике, которыми можно управлять только в шаблоне, то могут подойти формы на основе шаблона.

В таблице 1 приведены основные различия между реактивными и шаблонными формами.

Таблица 1

Формы Angular

Функции	Реактивные формы	Управляемые шаблоном
Настройка модели формы	Явная, созданная в классе компонента	Неявная, созданная директивами
Модель данных	Структурированная и неизменная	Неструктурированная и изменяемая
Обновление	Синхронное	Асинхронное
Проверка формы	Функции	Директивы

Если формы являются центральной частью вашего приложения, то **масштабируемость** очень важна. Возможность повторного использования моделей форм в компонентах имеет решающее значение.

Реактивные формы лучше масштабируются, чем формы на основе шаблонов. Они обеспечивают прямой доступ к базовому API форм и синхронный доступ к модели данных форм, что облегчает создание крупномасштабных форм. Реактивные формы требуют меньше настроек для тестирования, а тестирование не требует глубокого понимания обнаружения изменений, чтобы правильно тестировать обновления и валидацию форм.

Формы на основе шаблонов ориентированы на простые сценарии и не так многообразны. Они абстрагируют базовый API формы и предоставляют только асинхронный доступ к модели данных формы. Абстракция шаблонно-управляемых форм также влияет на тестирование. Тесты в значительной степени зависят от ручного выполнения обнаружения изменений и требуют дополнительной настройки.

Настройки модели формы. Реактивные и управляемые шаблоном формы отслеживают изменения значений между входными элементами формы, с которыми взаимодействуют пользователи, и данными формы в вашей компонентной модели. Оба подхода используют общие строительные блоки, но отличаются тем, как создаются и управляются общие экземпляры управления формой.

И реактивные, и управляемые шаблонами формы построены на следующих базовых классах:

- *FormControl* отслеживает значение и статус проверки отдельного элемента управления формы;
- *FormGroup* отслеживает те же значения и статус для коллекции элементов управления формой;

- *FormArray* отслеживает те же значения и статус для массива элементов управления формы;
- *ControlValueAccessor* создает мост между экземплярами *FormControl* Angular и собственными элементами DOM.

Настройка в реактивных формах. С помощью реактивных форм можно определить модель формы непосредственно в классе компонентов. Директива `[formControl]` связывает явно созданный экземпляр *FormControl* конкретного элемента формы в представлении с помощью внутреннего доступа к значениям.

Следующий компонент содержит поле ввода для одного элемента управления с помощью реактивных форм. В этом примере модель формы является экземпляром *FormControl*:

```
import { Component } from '@angular/core';
import { FormControl } from '@angular/forms';
@Component({
  selector: 'app-reactive-favorite-color',
  template: `
    Favorite Color: <input type="text"
      [formControl]="favoriteColorControl"> `
})
export class FavoriteColorComponent {
  favoriteColorControl = new FormControl('');
}
```

Настройка в формах, управляемых шаблонами. В управляемых шаблоном формах модель формы является неявной. Директива `NgModel` создает экземпляр *FormControl* для данного элемента формы и управляет им. Следующий компонент реализует одно и то же поле ввода для одного элемента управления с помощью формы на основе шаблонов:

```
content_copyimport { Component } from '@angular/core';
@Component({
  selector: 'app-template-favorite-color',
  template: `
    Favorite Color: <input type="text" [(ngModel)]="favoriteColor">
  `
})
export class FavoriteColorComponent {
  favoriteColor = '';
}
```

Работоспособность формы обеспечивается шаблоном. Прямого программного доступа к экземпляру FormControl нет.

Поток данных в формах

Если приложение содержит форму, Angular должен синхронизировать представление с моделью компонента, а модель компонента — с представлением. Поскольку пользователи изменяют значения и делают выбор с помощью представления, новые значения должны быть отражены в модели данных. Аналогично, когда логика программы изменяет значения в модели данных, эти значения должны быть отражены в представлении.

Реактивные и управляемые шаблонами формы отличаются тем, как они обрабатывают данные от пользователя или от методов. Далее описаны диаграммы, которые иллюстрируют оба вида потока данных для каждого типа формы с использованием поля ввода избранного цвета.

Поток данных в реактивных формах. В реактивных формах каждый элемент формы в представлении напрямую связан с моделью формы (экземпляром FormControl). Обновления из представления в модель и из модели в представление выполняются синхронно и не зависят от того, как отображается пользовательский интерфейс.

Диаграмма представления к модели показывает, как данные передаются при изменении значения поля ввода из представления с помощью следующих шагов.

- пользователь вводит значение в элемент ввода;
- элемент ввода формы генерирует событие 'input' с самым последним значением;
- метод доступа к управляющему значению, прослушивающий события в элементе ввода формы, немедленно передает новое значение экземпляру FormControl;
- экземпляр FormControl излучает новое значение через наблюдаемые valueChanges;
- любые подписчики на наблюдаемые valueChanges получают новое значение.

Поток данных в шаблонных формах. В формах, управляемых шаблоном, каждый элемент формы связан с директивой, которая управляет моделью формы внутри.

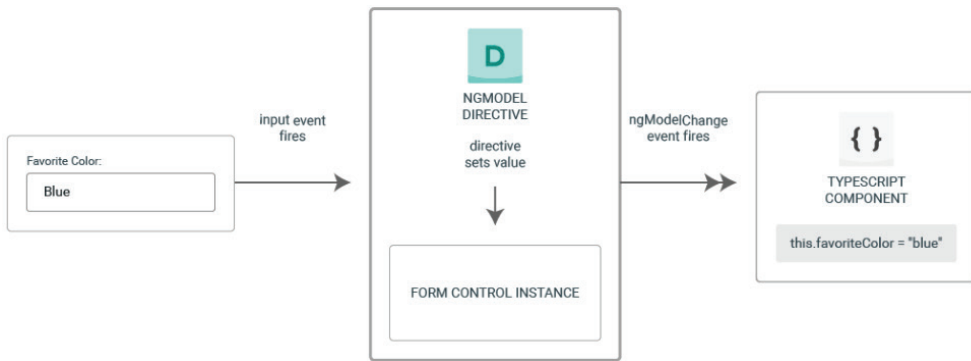


Рис. 17. Диаграмма «Представление-модель»

На диаграмме «Представление-модель» (рис. 17) показано, как происходит движение данных при изменении значения входного поля представления с помощью следующих шагов:

- пользователь вводит цвет Blue в элемент ввода;
- элемент input выдает событие 'input' со значением Blue;
- Controlvalueaccessor, подключенный к входу, запускает метод setValue() в экземпляре элемента управления формой;
- экземпляр элемента управления формой выдает новое значение с помощью наблюдаемых изменений;
- все подписчики на наблюдаемые изменения значения получают новое значение;
- Controlvalueaccessor также вызывает метод NgModel.viewToModelUpdate(), который выдает событие изменения модели;
- поскольку шаблон компонента использует двустороннюю привязку данных для свойства favoriteColor, свойство favoriteColor в компоненте обновляется до значения, посылаемого событием ngModelChange (Blue).

Аналогично выглядит перетекание данных из модели в представление, когда меняется любимый цвет, с помощью следующих шагов

- значение избранного цвета обновляется в компоненте;
- начинается обнаружение изменений;
- во время обнаружения изменений обработчик жизненного цикла ngOnChanges вызывается в экземпляре директивы NgModel, поскольку изменилось значение одного из его входных данных;

- метод `ngOnChanges()` ставит в очередь `async` задание, чтобы задать значение для экземпляра внутреннего элемента управления формой;
- обнаружение изменений завершено;
- на следующем тике выполняется задача по установке значения экземпляра элемента управления формой;
- экземпляр элемента управления формой выдает последнее модифицированное значение;
- все подписчики на наблюдаемые изменения значения получают новое значение;
- управление доступом в представлении обновляет элемент ввода формы с помощью последнего модифицированного значения `favoriteColor`.

Изменения модели данных. Метод отслеживания изменений играет определенную роль в эффективности вашего приложения. Реактивные формы поддерживают чистоту модели данных, предоставляя ее в виде неизменяемой структуры данных. Каждый раз, когда в модели данных инициируется изменение, экземпляр `FormControl` возвращает новую модель данных, а не обновляет существующую модель. Это дает вам возможность отслеживать уникальные изменения в модели данных с помощью `observable` элемента управления. Обнаружение изменений более эффективно, потому что ему нужно только обновить уникальные изменения. Поскольку обновления данных следуют реактивным паттернам, вы можете интегрироваться с наблюдаемыми операторами для преобразования данных.

Управляемые шаблоном формы полагаются на изменяемость с двусторонней привязкой данных для обновления модели данных в компоненте по мере внесения изменений в шаблон. Поскольку при использовании двусторонней привязки данных в модели данных нет уникальных изменений для отслеживания, обнаружение изменений менее эффективно при определении того, когда требуются обновления. Это различие показано в предыдущих примерах, в которых используется элемент ввода `favorite-color`.

При использовании реактивных форм экземпляр `FormControl` всегда возвращает новое значение при обновлении значения элемента управления.

При использовании шаблонных форм свойство «любимый цвет» всегда изменяется на новое значение.

Валидация форм. Валидация является неотъемлемой частью управления любым набором форм. Независимо от того, проверяете ли вы обязательные поля или запрашиваете внешний API для существующего имени пользователя, Angular предоставляет набор встроенных валидаторов, а также возможность создавать пользовательские валидаторы. Реактивные формы используют пользовательские валидаторы как функции, которые получают элемент управления для проверки. Управляемые шаблоном формы привязаны к директивам шаблона и должны предоставлять пользовательские директивы валидатора, которые оборачивают функции проверки.

Проверка ввода в формах, управляемых шаблоном. Для того чтобы добавить проверку в форму на основе шаблона, необходимо добавить те же атрибуты проверки, что и при проверке формы HTML. Angular использует директивы для сопоставления этих атрибутов с функциями валидатора платформы.

Каждый раз, когда значение элемента управления формы изменяется, Angular запускает проверку и генерирует либо список ошибок проверки, которые приводят к состоянию INVALID, либо значение NULL, которое приводит к состоянию VALID. Затем вы можете проверить состояние элемента управления путем экспорта `ngModel` в локальную переменную шаблона. В следующем примере выполняется экспорт `NgModel` в переменную с именем `name`:

```
<input id="name" name="name" class="form-control"
      required minlength="4" appForbiddenName="Kot"
      [(ngModel)]="name" #name="ngModel" >
<div *ngIf="name.invalid && (name.dirty || name.touched)"
      class="alert alert-danger">
  <div *ngIf="name.errors.required"> Name is required. </div>
  <div *ngIf="name.errors.minlength">
    Name must be at least 4 characters long.
  </div>
  <div *ngIf="name.errors.forbiddenName"> Name cannot be Bob. </div>
</div>
```

Элемент `<input>` несет в себе атрибуты валидации HTML: `required` и `minlength`. Он также несет пользовательскую директиву валидатора `forbiddenName`. Переменная шаблона `#name=»ngModel»` экспортирует `NgModel` в локальную переменную с именем `name`. `NgModel` отражает многие свойства своего базового экземпляра `FormControl`, так что

это можно использовать в шаблоне для проверки состояний управления, таких как `valid` и `dirty`.

Директива `*ngIf` на элементе `<div>` показывает набор вложенных сообщений только для неверных значений. Каждый вложенный `<div>` может представить свое сообщение для одной из возможных ошибок проверки. Есть сообщения для `required`, `minlength` и `forbiddenName`.

Валидация ввода в реактивных формах. В реактивной форме ответственным за валидацию является класс компонентов. Вместо добавления валидаторов через атрибуты в шаблоне вы добавляете функции валидатора непосредственно в модель управления формой в классе компонента. Затем Angular вызывает эти функции всякий раз, когда значение элемента управления изменяется.

Функции валидатора могут быть синхронными или асинхронными:

- **синхронизирующие валидаторы** — синхронные функции, которые берут экземпляр управления формой и немедленно возвращают либо набор ошибок валидации, либо `null` (их можно передать в качестве второго аргумента при создании экземпляра `FormControl`);
- **асинхронные валидаторы** — асинхронные функции, которые берут экземпляр элемента управления и возвращают `Promise` или `Observable`, которые позже выдают набор ошибок валидации или `null` (их можно передать в качестве третьего аргумента при создании экземпляра `FormControl`).

Из соображений производительности Angular запускает только асинхронные валидаторы, если все валидаторы синхронизации пропущены. Каждый должен завершиться, прежде чем ошибки будут установлены.

Можно написать свои собственные функции валидатора или использовать некоторые из встроенных валидаторов Angular. Те же самые встроенные валидаторы, которые доступны как атрибуты в управляемых шаблонах форм (`required` и `minlength`) доступны для использования в качестве функций из класса `Validators`.

Контрольные вопросы

1. В чем существенное отличие реактивных и шаблонных форм?
2. Зачем нужно проверять правильность ввода данных на стороне клиента?

11. HTTP и взаимодействие с сервером

11.1. HttpClient и отправка запросов

Для взаимодействия с сервером и отправки запросов по протоколу *HTTP* применяется класс *HttpClient*. Этот класс определяет ряд методов для отправки различного рода запросов: GET, POST, PUT, DELETE. Данный класс построен поверх стандартного объекта в JavaScript — *XMLHttpRequest*. Для использования этого класса в проект необходимо установить пакет *@angular/common*:

```
"dependencies": {  
  "@angular/common": "~9.0.0",  
  // остальные пакеты  
}
```

Также в файле модуля *AppModule* должен быть импортирован класс *HttpClientModule* из пакета *@angular/common/http*:

```
.....  
import { HttpClientModule } from '@angular/common/http';  
@NgModule({  
  imports:      [ BrowserModule, FormsModule, HttpClientModule ],  
  declarations: [ AppComponent ],  
  bootstrap:   [ AppComponent ]  
})  
export class AppModule { }
```

При взаимодействии с сервером, как правило, обращения происходят не из самого компонента, а из вспомогательных сервисов, поскольку сервис может определять дополнительную логику обработки полученных с сервера данных, которую мы могли бы выполнить. Кро-

ме того, сервисы могут определять функционал, который будет использоваться несколькими компонентами. Компоненты же выступают в качестве потребителей данных, которые они получают от сервисов.

11.2. Обработка ошибок

При работе с сетью и объектом `http` могут возникать ошибки, например в момент запроса сеть стала недоступна, неправильно указан адрес и соответственно не найден ресурс, либо доступ к ресурсу ограничен, либо другие ошибки. Перехват ошибок позволит выяснить проблему и каким-то образом обработать их, например, вывести пользователю сообщение об ошибке.

Рассмотрим пример, в котором был изменен код компонента `AppComponent`:

```
constructor(private httpService: HttpService){}
ngOnInit() {
    this.httpService.getUsers().subscribe(
        data=>this.users=data,
        error => {this.error = error.message;
                  console.log(error);}
    );
}
```

Метод `subscribe()` в качестве второго параметра принимает функцию-обработчик, которая вызывается в случае возникновения ошибки. В этом обработчике можно получить ошибку и обработать ее. Ошибка представляет объект, из которого можно получить ряд данных. В частности, свойство `message` позволяет получить сообщение об ошибке, а свойство `statusCode` — статусный код ответа.

Кроме того, для перехвата ошибок к объекту `Observable`, который является результатом запроса, можно применить функцию `catchError()`.

11.3. Отправка данных в запросе

Нередко возникает необходимость обращаться с более сложными запросами к удаленному ресурсу, передавая ему некоторые параметры. Вначале рассмотрим способ передачи параметров в get-запросе.

Может использоваться любая технология на сервере сервисов. Но вне зависимости от выбранной технологии следует учитывать ограничения на кросс-доменные запросы. В частности, если наше приложение на Angular запускается в одном домене, а сервер, обрабатывающий запросы, запущен на другом домене, то на сервере нам надо включить технологию CORS.

Поскольку данные передаются через запрос GET, то мы можем конкатенировать нужное число в строке запроса.

Класс `HttpParams`. Для определения параметров в запросе можно использовать класс `HttpParams`. В частности, изменим сервис `HttpService` следующим образом:

```
import {Injectable} from '@angular/core';
import {HttpClient, HttpParams} from '@angular/common/http';
@Injectable()
export class HttpService{
  constructor(private http: HttpClient){ }
  getResult(num: number){
    const params = new HttpParams().set('number', num.toString());
    return this.http.get('http://localhost:8080/GetResult', {params});
  }
}
```

С помощью метода `set ()` объекта `HttpParams` устанавливаются параметры, и затем этот объект передается в запрос.

POST-запросы. Кроме GET-запросов Angular позволяет отправлять остальные типы запросов: POST, PUT, DELETE. Рассмотрим способы выполнения POST-запросов.

В файле `http.service.ts` определим сервис для отправки данных:

```
constructor(private http: HttpClient){ }
postData(user: User){
  const body = {name: user.name, age: user.age};
  return this.http.post('http://localhost:8080/api/values', body);
}
```

Здесь определен метод `postData`, который получает для отправки объект `User`. Сами отправляемые данные представляют объект `body`. Для отправки применяется метод `http.post()`, с помощью которого передается адрес сервера и отправляемый объект.

```
constructor(private httpService: HttpService){}
submit(user: User){
    this.httpService.postData(user)
        .subscribe(
            (data: User) => {this.receivedUser=data;
            this.done=true;},
            error => console.log(error)
        );
}
}
```

При отправке можно устанавливать различные заголовки с помощью объекта `HttpHeaders`:

```
constructor(private http: HttpClient){ }
postData(user: User){
    const myHeaders = new HttpHeaders().set('Authorization',
    'my-auth-token');
    return this.http.post('http://localhost:60820/api/values',
    user, {headers:myHeaders});
}
}
```

Контрольные вопросы

1. Зачем нужен класс `HttpClient`?
2. Зачем нужен класс сервисов?
3. Какие методы исполнения запросов поддерживает класс `HttpClient`?

12. Маршрутизация

12.1. Определение маршрутов

Маршрутизация позволяет направлять запросы к определенным ресурсам внутри приложения. Ключевым для работы маршрутизации является модуль *RouterModule*, который располагается в пакете *@angular/router*, поэтому при работе с маршрутизацией данный пакет должен быть указан в списке зависимостей в файле `package.json`:

```
{ .....
  "dependencies": {
    "@angular/router": "~9.0.0",
    // остальные пакеты
  },
  "devDependencies": {
    // остальные пакеты
  }
}
```

Для определения маршрутов указывается базовый адрес приложения. Для этого возьмем веб-страницу `index.html` и добавим в секцию `<head>` элемент `<base>`:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <base href="/" />
```

```
<title>Hello Angular 7</title>
<link rel="stylesheet"
      href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.2/css/
      bootstrap.min.css">
</head>
<body>
  <my-app>Загрузка...</my-app>
  <script src="public/polyfills.js"></script>
  <script src="public/app.js"></script>
</body>
</html>
```

В качестве базового адреса будет использоваться корень приложения.

Каждый маршрут сопоставляется с определенным компонентом, поэтому добавим в проект ряд компонентов. Так, добавим в папку *src/app* новый файл *home.component.ts*:

```
import { Component } from '@angular/core';

@Component({
  selector: 'home-app',
  template: `<h3>Главная</h3>`
})
export class HomeComponent { }
```

Этот простенький компонент выводит обычный заголовок. Далее добавим в папку *src/app* новый файл *about.component.ts*:

```
import { Component } from '@angular/core';
@Component({
  selector: 'about-app',
  template: `<h3>О сайте</h3>`
})
export class AboutComponent { }
```

И также добавим еще один файл *not-found.component.ts*:

```
import { Component } from '@angular/core';
@Component({
  selector: 'not-found-app',
  template: `<h3>Страница не найдена</h3>`
})
export class NotFoundComponent { }
```

RouterModule и добавление маршрутов. Кроме главного компонента AppComponent, в проекте определено еще три компонента, которые просто выводят некоторый заголовок. Для каждого компонента можно определить свой маршрут — изменим код модуля AppModule:

```
import { NgModule }           from '@angular/core';
import { BrowserModule }      from '@angular/platform-browser';
import { Routes, RouterModule } from '@angular/router';
import { AppComponent }       from './app.component';
import { AboutComponent }     from './about.component';
import { HomeComponent }      from './home.component';
import { NotFoundComponent }  from './not-found.component';
// определение маршрутов
const appRoutes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', component: NotFoundComponent }
];
@NgModule({
  imports:      [ BrowserModule, RouterModule.forRoot(appRoutes) ],
  declarations: [ AppComponent, HomeComponent, AboutComponent,
    NotFoundComponent ],
  bootstrap:   [ AppComponent ]
})
export class AppModule { }
```

Во-первых, здесь импортируются модуль маршрутизации *RouterModule* и класс *Routes*, представляющий коллекцию маршрутов:

```
import { Routes, RouterModule } from '@angular/router';
```

Далее определяется коллекция маршрутов:

```
const appRoutes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', component: NotFoundComponent }
];
```

Здесь определено три маршрута, которые будут отображаться в виде отдельного компонента. Для указания маршрута применяется параметр path. Например, путь 'about' будет представлять запрос

типа «`http://localhost:3000/about`» и будет обрабатываться классом `AboutComponent`.

Если запрос не содержит никаких сегментов, например просто имя домена «`http://localhost:3000/`», то такой запрос будет сопоставляться с путем «» (пустая строка) и будет обрабатываться с помощью компонента `HomeComponent`. Если приложение получит запрос, который не подходит ни для одного из вышеопределенных маршрутов, то запрос будет сопоставляться с шаблоном «`**`», у которого две звездочки представляют *любой* путь.

Для того чтобы применить маршруты, они передаются методу `RouterModule.forRoot (appRoutes)`:

```
imports: [ BrowserModule, RouterModule.forRoot (appRoutes) ],
```

Метод `RouterModule.forRoot()` возвращает модуль, который содержит сконфигурированный сервис `Router`. Когда приложение загружается, `Router` выполняет начальную навигацию по текущему URL, который указан в адресной строке браузера.

Элемент `RouterOutlet`. Мы определили три разных компонента для разных маршрутов. Однако в качестве главного компонента выступает `AppComponent`. Этот компонент выступает в качестве контейнера для остальных компонентов, которые будут обслуживать запросы к приложению. Однако чтобы можно было внедрить в `AppComponent` тот компонент, который обрабатывает запрос, необходимо использовать элемент *`RouterOutlet`*. Для этого изменим код `AppComponent`:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  template: `<div>
    <h1>Маршрутизация в Angular</h1>
    <router-outlet></router-outlet>
  </div>`
})
export class AppComponent {}
```

На месте элемента `<router-outlet>` будет размещаться компонент, выбранный для обработки запроса.

URL Matching и порядок маршрутов. При выполнении перехода по определенному пути, например `'/about'`, система маршрутизации сопоставляет последовательно URL запроса с параметрами `path` у каждого маршрута. Данный процесс называется `URL matching`. Однако при определении маршрутов следует учитывать их порядок. Вполне возможно, что определенному запросу будет соответствовать сразу несколько маршрутов. В этом случае запрос будет обрабатываться *первым* из них. Другие же маршруты не будут учитываться.

Переадресация. Вполне возможно, что по какому-то маршруту нужно сделать переадресацию по другому пути. Например, в случае, если нужного маршрута для запроса не найдено, можно переадресовать пользователя на главную страницу:

```
const appRoutes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', redirectTo: '/' }
];
```

Для переадресации указываем параметр `redirectTo`, значение которого представляет путь переадресации. В данном случае слеш указывает на первый маршрут или на главную страницу. Также можно задать критерий соответствия строки запроса маршруту с помощью параметра `pathMatch`:

```
const appRoutes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'contact', redirectTo: '/about', pathMatch: 'full' },
  { path: '**', redirectTo: '/' }
];
```

Значение `pathMatch: <full>` указывает на то, что запрошенный адрес должен полностью соответствовать маршруту, должно быть полное соответствие. Например, запрос `/contact` полностью соответствует маршруту `{ path: <contact>, redirectTo: </about>, pathMatch: <full> }`, поэтому будет выполнена переадресация на адрес `/about`.

А запрос `/contact/5` не будет соответствовать этому маршруту, так как после «`contact`» идут другие сегменты.

12.2. Создание ссылок

Для удобного перехода между различными частями приложения, как правило, применяется система навигации, состоящая из ссылок. К примеру, возьмем проект из прошлой темы и добавим в него навигацию. В этом проекте есть два компонента `HomeComponent` и `AboutComponent`, которые обрабатывают различные запросы и результат обработки которых вставляется в шаблон главного компонента класса `AppComponent`.

Для этого определим в `AppComponent` набор ссылок для навигации:

```
import { Component } from '@angular/core';

@Component({
  selector: 'my-app',
  template: `<div>
    <nav>
      <a routerLink="">Главная</a>
      <a routerLink="/about">О сайте</a>
    </nav>
    <router-outlet></router-outlet>
  </div>`
})
export class AppComponent {}
```

Для определения адресов ссылок применяется директива *`routerLink`*.

В прошлой теме в модуле `AppModule` было определено три маршрута:

```
const appRoutes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', component: NotFoundComponent }
];
```

В соответствии с первым маршрутом `HomeComponent` обрабатывает запросы URL без сегментов, поэтому для создания ссылки на этот компонент указывается пустая строка: `<a routerLink="">`.

`AboutComponent` обрабатывает запросы, которые содержат в адресе 'about', поэтому ссылка имеет соответствующий адрес: `<a router-`

`Link="/about">`. В данном случае стоит отметить слеш, с которого начинается адрес. Если необходимо определить набор ссылок в главном компоненте (как здесь), то слеш в принципе можно не использовать. Если же ссылки определяются в дочерних компонентах, например в `AboutComponent`, то без слеша можно получить некорректный путь. Слеш позволяет задать путь относительно **корня** приложения.

Стилизация активных ссылок. Для стилизации активных ссылок применяется специальная директива `routerLinkActive`, которая указывает на класс `css`, применяемый к активной ссылке. Так, изменим класс компонента:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  styles: [
    .active {color:red;}
  ],
  template: `<div>
    <nav>
      <a routerLink=""
        routerLinkActive="active">Главная</a>
      <a routerLink="/about"
        routerLinkActive="active">О сайте</a>
    </nav>
    <router-outlet></router-outlet>
  </div>`
})
export class AppComponent { }
```

В то же время при стилизации следует учитывать следующий момент: если нажать на ссылку «О сайте», то будут стилизованы обе ссылки. Всё дело в том, что путь, представляющий первую ссылку (`routerLink=""`), совпадает с URL второй ссылки (`routerLink="/about"`), так как первый путь можно свести к любому другому пути. В этом случае необходимо применить дополнительные настройки с помощью директивы `routerLinkActiveOptions`:

```
<nav>
  <a routerLink="" routerLinkActive="active"
    [routerLinkActiveOptions]="{exact:true}">Главная</a>
  <a routerLink="/about" routerLinkActive="active">О сайте</a>
</nav>
```

Значение `{exact: true}` указывает на то, что для установки активной ссылки будет применяться полное соответствие.

Элементы навигации необязательно представляют голые ссылки. Нередко они заключаются в списки, в какие-то блоки. И, возможно, появится желание стилизовать не просто активную ссылку, а весь элемент, в котором данная ссылка расположена. В этом случае можно применить директиву `routerLinkActive` к элементу-контейнеру ссылки:

```
import { Component } from '@angular/core';
@Component({
  selector: 'my-app',
  styles: [`
    .nav{ clear: both;}
    a {float: left;}
    .active a { color: red;}
  `],
  template: `<div>
    <ul class="nav">
      <li routerLinkActive="active"
[routerLinkActiveOptions]="{exact:true}">
        <a routerLink="">Главная</a>
      </li>
      <li routerLinkActive="active">
        <a routerLink="/about">О сайте</a>
      </li>
    </ul>
    <router-outlet></router-outlet>
  </div>`
})
export class AppComponent { }
```

12.3. Программная навигация

Angular поддерживает программную навигацию — программным образом из любого места приложения можно перейти к любому ресурсу. Для этого применяется сервис *Router*, который определен в пакете 'angular/router' и который передается в компоненты с помощью механизма *dependency injection*.

Так, определим в компоненте кнопку и обработчик кнопки, который будет перенаправлять на определенный ресурс:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
@Component({
  selector: 'my-app',
  template: `<div>
    <router-outlet></router-outlet>
    <button (click)="goHome()">На главную</button>
  </div>`
})
export class AppComponent {
  constructor(private router: Router) {}
  goHome() {
    this.router.navigate(['']);
  }
}
```

С помощью конструктора получаем объект Router и в обработчике кнопки вызываем его метод *navigate()*. Этому методу передается путь перехода. Если необходимо перейти на главную страницу, которая представлена HomeComponent, то передается пустая строка. Для того чтобы перейти, допустим, к компоненту AboutComponent, можно было бы передать соответствующий путь: `this.router.navigate(['/about'])`.

Для того чтобы передать компоненту нужные параметры методу *navigate()*, первым параметром передается набор значений, первое из которых — собственно путь, а все последующие — значения для параметров маршрута. Вторым параметром представляет объект javascript, который содержит все нужные значения для параметров строки запроса.

Контрольные вопросы

1. Зачем нужен элемент RouterOutlet?
2. Как влияет порядок указания маршрутов на визуализацию?
3. Что делает атрибут routerLink="путь"?
4. Что делает фрагмент кода `this.router.navigate(['путь'])`?

13. Angular Material

Б иблиотека Angular Material позволяет на высоком профессиональном уровне выполнить дизайн и запрограммировать функциональность веб-приложения. Установка библиотеки выполняется с помощью команды:

```
ng add @angular/material
```

Продemonстрируем возможности библиотеки на примере работы с таблицей продуктов в базе данных. Создайте несколько компонентов:

```
ng g component products
ng g component product-detail
ng g component product-add
ng g component product-edit
```

Затем откройте и отредактируйте `src/app/app-routing.module.ts`, добавьте их импорты:

```
import { ProductsComponent } from './products/products.component';
import { ProductDetailComponent } from './product-detail/product-detail.component';
import { ProductAddComponent } from './product-add/product-add.component';
import { ProductEditComponent } from './product-edit/product-edit.component';
```

Добавьте эти массивы к существующей константе маршрутов в файле `app-routing.module.ts`:

```
import { NgModule } from '@angular/core';
import { ProductsComponent } from '../products/products.component';
import { ProductDetailComponent } from '../product-detail/product-detail.
  component';
import { ProductAddComponent } from '../product-add/product-add.
  component';
import { ProductEditComponent } from '../product-edit/product-edit.
  component';
import { Routes, RouterModule } from '@angular/router';
const routes: Routes = [
  {
    path: 'products',
    component: ProductsComponent,
    data: { title: 'List of Products' }
  },
  {
    path: 'product-details/:id',
    component: ProductDetailComponent,
    data: { title: 'Product Details' }
  },
  {
    path: 'product-add',
    component: ProductAddComponent,
    data: { title: 'Add Product' }
  },
  {
    path: 'product-edit/:id',
    component: ProductEditComponent,
    data: { title: 'Edit Product' }
  },
  { path: '',
    redirectTo: '/products',
    pathMatch: 'full'
  }
];
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Откройте и отредактируйте src/app/app.component.html:

```
<div style="text-align:center">
  
```

```
</div>
<div class="container">
  <router-outlet></router-outlet>
</div>
```

Откройте и отредактируйте `src/app/app.component.css`:

```
.container {
  padding: 20px;
}
```

В приложении используется модель, содержащая объекты класса **Product**, например:

```
export class Product {
  // Объявления полей объекта из таблицы базы данных,
  // приходящего из REST-сервиса в формате JSON
  // Например:
  // {id: 1, prodName: "Хлеб", prodDesc: "Насушный", prodPrice: 30.5,
  // updatedAt: Wed Oct 09 2019 00:00:00 GMT+0500 (Екатеринбург,
  // стандартное время)}
  id: string;
  // tslint:disable-next-line:variable-name
  prod_name: string;
  // tslint:disable-next-line:variable-name
  prod_desc: string;
  // tslint:disable-next-line:variable-name
  prod_price: number;
  // tslint:disable-next-line:variable-name
  updated_at: Date;
}
```

Откройте и отредактируйте `src/app/api.service.ts`:

```
import { Injectable } from '@angular/core';
import { Observable, of, throwError } from 'rxjs';
import { HttpClient, HttpHeaders, HttpResponse } from '@angular/
  common/http';
import { catchError, tap, map } from 'rxjs/operators';
import { Product } from './product';
const httpOptions = {
  headers: new HttpHeaders({'Content-Type': 'application/json'})
};
```

```

const apiUrl = '/api/v1/products';
@Injectable({
  providedIn: 'root'
})
export class ApiService {
  constructor(private http: HttpClient) { }
  private handleError<T>(operation = 'operation', result?: T) {
    return (error: any): Observable<T> => {
      // TODO: send the error to remote logging infrastructure
      console.error(error); // log to console instead
      // Let the app keep running by returning an empty result.
      return of(result as T);
    };
  }
  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>(apiUrl)
      .pipe(
        tap(product => console.log('fetched products')),
        catchError(this.handleError('getProducts', []))
      );
  }
  getProduct(id: number): Observable<Product> {
    const url = `${apiUrl}/${id}`;
    return this.http.get<Product>(url).pipe(
      tap(_ => console.log(`fetched product id=${id}`)),
      catchError(this.handleError<Product>(`getProduct id=${id}`))
    );
  }
  addProduct(product: Product): Observable<Product> {
    return this.http.post<Product>(apiUrl, product, httpOptions).pipe(
      tap((prod: Product) => console.log(`added product w/
        id=${product.id}`)),
      catchError(this.handleError<Product>('addProduct'))
    );
  }
  updateProduct(id: any, product: Product): Observable<any> {
    const url = `${apiUrl}/${id}`;
    return this.http.put(url, product, httpOptions).pipe(
      tap(_ => console.log(`updated product id=${id}`)),
      catchError(this.handleError<any>('updateProduct'))
    );
  }
  deleteProduct(id: any): Observable<Product> {
    const url = `${apiUrl}/${id}`;
    return this.http.delete<Product>(url, httpOptions).pipe(

```

```
        tap(_ => console.log(`deleted product id=${id}`)),
        catchError(this.handleError<Product>('deleteProduct'))
    );
}
}
```

13.1. Показ списка продуктов

Откройте и отредактируйте `src/app/products/products.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { ApiService } from '../api.service';
import { Product } from '../product';
@Component({
  selector: 'app-products',
  templateUrl: './products.component.html',
  styleUrls: ['./products.component.css']
})
export class ProductsComponent implements OnInit {
  displayedColumns: string[] = ['prod_name', 'prod_price'];
  data: Product[] = [];
  isLoadingResults = true;
  constructor(private api: ApiService) { }
  ngOnInit() {
    this.api.getProducts()
      .subscribe((res: any) => {
        this.data = res;
        console.log(this.data);
        this.isLoadingResults = false;
      }, err => {
        console.log(err);
        this.isLoadingResults = false;
      });
  }
}
```

Откройте и отредактируйте файл `src/app/app.module.ts`:

```
import { BrowserModule } from '@angular/platform-browser';
import { LOCALE_ID, NgModule } from '@angular/core';
import { AppRoutingModule } from './app-routing.module';
```

```

import { AppComponent } from '../app.component';
import { BrowserAnimationsModule } from '@angular/platform-browser/
  animations';
import { ProductsComponent } from '../products/products.component';
import { ProductDetailComponent } from '../product-detail/product-detail.
  component';
import { ProductAddComponent } from '../product-add/product-add.
  component';
import { ProductEditComponent } from '../product-edit/product-edit.
  component';
import { HttpClientModule } from '@angular/common/http';
import { MatInputModule, MatPaginatorModule, MatProgressSpinnerModule,
  MatSortModule, MatTableModule, MatIconModule, MatButtonModule,
  MatCardModule,
  MatFormFieldModule, MatDatepickerModule, MatNativeDateModule,
  MAT_DATE_LOCALE
} from '@angular/material';
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
// Состав модулей приложения
@NgModule({
  declarations: [ AppComponent, ProductsComponent, ProductDetailComponent,
    ProductAddComponent, ProductEditComponent ],
  imports:
    [ BrowserModule, FormsModule, HttpClientModule, AppRoutingModule,
    ReactiveFormsModule, BrowserAnimationsModule, MatInputModule,
    MatTableModule, MatPaginatorModule, MatSortModule,
    MatProgressSpinnerModule,
    MatIconModule, MatButtonModule, MatCardModule, MatFormFieldModule,
    MatDatepickerModule, MatNativeDateModule ],
  exports: [ MatTableModule, MatPaginatorModule ],
  providers: [
    { provide: LOCALE_ID, useValue: 'en-EN' },
    { provide: MAT_DATE_LOCALE, useValue: 'ru-RU' }
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

Затем откройте и отредактируйте `src/app/products/products.component.html`:

```

<div class="example-container mat-elevation-z8">
  <div class="example-loading-shade"
    *ngIf="isLoadingResults">
    <mat-spinner *ngIf="isLoadingResults"></mat-spinner>

```

```
</div>
<!-- Поля для ввода дат-->
<mat-card class="example-card">
  <form [formGroup]="filterForm" (ngSubmit)="onFormSubmit()">
    <h5>Задайте интервал времени для отбора продуктов</h5>
    <mat-form-field>
      <input matInput [matDatepicker]="pickerFrom"
        placeholder="Дата с..." FormControlName="dateFrom">
      <mat-datepicker-toggle matSuffix [for]="pickerFrom"></mat-datepicker-toggle>
      <mat-datepicker #pickerFrom></mat-datepicker>
    </mat-form-field>
    <mat-form-field>
      <input matInput [matDatepicker]="pickerTo"
        placeholder="Дата по..." FormControlName="dateTo">
      <mat-datepicker-toggle matSuffix [for]="pickerTo"></mat-datepicker-toggle>
      <mat-datepicker #pickerTo></mat-datepicker>
    </mat-form-field>
    <div class="button-row">
      <button type="submit" [disabled]="!filterForm.valid"
        mat-flat-button color="primary"><mat-icon>save</mat-icon></button>
    </div>
  </form>
</mat-card>
<!-- Кнопка для добавления продукта-->
<div class="button-row">
  <a mat-flat-button color="primary" [routerLink]="['/product-add']"><mat-icon>add</mat-icon></a>
</div>
<!-- Таблица для отображения продуктов-->
<div class="mat-elevation-z8">
  <table mat-table [dataSource]="dataSource" class="example-table-container"
    matSort matSortActive="id" matSortDisableClear
    matSortDirection="asc">
    <!-- Product id Column -->
    <ng-container matColumnDef="id">
      <th mat-header-cell *matHeaderCellDef>#Ключ</th>
      <td mat-cell *matCellDef="let row">{{row.id}}</td>
    </ng-container>
    <!-- Product Name Column -->
    <ng-container matColumnDef="prod_name">
      <th mat-header-cell *matHeaderCellDef>Название</th>
```



```

        <td mat-cell *matCellDef="let row">{{row.prod_name}}</td>
</ng-container>
<!-- Product Price Column Number-->
<ng-container matColumnDef="prod_price">
    <th mat-header-cell *matHeaderCellDef>Цена</th>
    <td mat-cell *matCellDef="let row">{{row.prod_price |
    currency: 'RUB': 'symbol-narrow'}}</td>
</ng-container>
<!-- Product updated Column Date -->
<ng-container matColumnDef="updated_at">
    <th mat-header-cell *matHeaderCellDef>Когда</th>
    <td mat-cell *matCellDef="let row">{{(row.updated_at | date:'dd
MM yyyy').toString()}}</td>
</ng-container>
<!--Реакция на выбор строки мышкой-->
    <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
    <tr mat-row *matRowDef="let row; columns: displayedColumns;"
    [routerLink]="['/product-details/', row.id]"></tr>
</table>
<!-- Навигатор страниц-->
    <mat-paginator [pageSizeOptions]="[5, 10, 15]" showFirstLastButtons></
mat-paginator>
</div>
</div>

```

Откройте и отредактируйте `src/app/products/products.component.css`:

```

.example-container {
    position: relative;
    padding: 5px;
}
.example-table-container {
    position: relative;
    max-height: 400px;
    overflow: auto;
}
.table {
    width: 100%;
}
.example-loading-shade {
    position: absolute;
    top: 0;
    left: 0;
    bottom: 56px;
    right: 0;
}

```

```
background: rgba(0, 0, 0, 0.15);
z-index: 1;
display: flex;
align-items: center;
justify-content: center;
}
.example-rate-limit-reached {
  color: #980000;
  max-width: 360px;
  text-align: center;
}
.mat-column-number,
.mat-column-state {
  max-width: 64px;
}
.mat-column-created {
  max-width: 54px;
}
.mat-flat-button {
  margin: 5px;
}
```

Пагинация. Для того чтобы разбить данные таблицы на страницы, добавьте `<mat-paginator>` после таблицы. Если используется `MatTableDataSource` для источника данных вашей таблицы, просто предоставьте `MatPaginator` вашему источнику данных. Он будет автоматически прослушивать изменения страницы, сделанные пользователем, и отправлять правильные данные по страницам в таблицу.

Реализовать разбиение на страницы в таблице данных несложно. Во-первых, необходимо импортировать сервис `MatPaginatorModule` в `app.module.ts`. Далее добавить после тега `</table>` тег разбиения на страницы:

```
<mat-paginator [pageSizeOptions]="[5, 10, 15]"
  showFirstLastButtons></mat-paginator>
```

Если все правильно, на экране появится таблица (рис. 18).

#Ключ	Название	Цена	Когда
1	Хлеб	₽30.50	09.10.2019
2	Масло	₽255.00	10.09.2019
3	Молоко	₽45.00	01.05.2019
4	gener	₽123.00	11.10.2019
5	gener	₽123.00	11.10.2019

Items per page: 5 1 – 5 of 29 < < > >

Рис. 18. Таблица, разбитая на страницы

13.2. Показ деталей продуктов

Чтобы отобразить сведения о продукте после щелчка или нажатия на одну из строк внутри таблицы материалов, откройте и отредактируйте `src/app/product-detail/product-detail.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { ApiService } from '../api.service';
// @ts-ignore
import { Product } from '../product';
@Component({
  selector: 'app-product-detail',
  templateUrl: './product-detail.component.html',
  styleUrls: ['./product-detail.component.css']
})
export class ProductDetailComponent implements OnInit {
  product: Product = { id: '', prod_name: '', prod_desc: '',
    prod_price: null, updated_at: null };
  isLoadingResults = true;
  private id: any;
  constructor(private route: ActivatedRoute, private api: ApiService,
```

```
        private router: Router) { }
getProductDetails(id: any) {
    // Запрос детали по ключу у сервиса
    this.api.getProduct(id)
        .subscribe((data: any) => {
            this.product = data;
            console.log(this.product);
            this.isLoadingResults = false;
        });
}
deleteProduct(id: any) {
    // Удаление детали через сервис
    this.isLoadingResults = true;
    this.id = id;
    this.api.deleteProduct(id)
        .subscribe(res => {
            this.isLoadingResults = false;
            this.router.navigate(['/products']);
        }, (err) => {
            console.log(err);
            this.isLoadingResults = false;
        })
}
ngOnInit() {
    this.getProductDetails(this.route.snapshot.params.id);
}
}
```

Для просмотра откройте и отредактируйте `src/app/product-detail/product-detail.component.html`.

```
<!--Форма для просмотра продукта-->
<div class="example-container mat-elevation-z8">
  <div class="example-loading-shade"
    *ngIf="isLoadingResults">
    <mat-spinner *ngIf="isLoadingResults"></mat-spinner>
  </div>
  <div class="button-row">
    <a mat-flat-button color="primary" [routerLink]="['/products']"><mat-
icon>list</mat-icon></a>
  </div>
  <mat-card class="example-card">
    <mat-card-header>
      <mat-card-title><h2>{{product.prod_name}}</h2></mat-card-title>
```

```

        <mat-card-subtitle>{{product.id}}</mat-card-subtitle>
    </mat-card-header>
<!--      Поля с данными о продукте-->
    <mat-card-content>
        <dl>
            <dt>#Ключ:</dt>
            <dd>{{product.id}}</dd>
            <dt>Название:</dt>
            <dd>{{product.prod_name}}</dd>
            <dt>Описание:</dt>
            <dd>{{product.prod_desc}}</dd>
            <dt>Цена:</dt>
            <dd>{{product.prod_price}}</dd>
            <dt>Изменен:</dt>
            <dd>{{product.updated_at | date:'yyyy MM dd'}}</dd>
        </dl>
    </mat-card-content>
<!--      Кнопки навигации-->
    <mat-card-actions>
        <a mat-flat-button color="primary" [routerLink]="['/product-edit',
            product.id]"><mat-icon>edit</mat-icon></a>
        <a mat-flat-button color="warn" (click)="deleteProduct
            (product.id)"><mat-icon>delete</mat-icon></a>
    </mat-card-actions>
</mat-card>
</div>

```

Откройте и редактируйте `src/app/product-detail/product-detail.component.scss`:

```

.example-container {
  position: relative;
  padding: 5px;
}
.example-loading-shade {
  position: absolute;
  top: 0;
  left: 0;
  bottom: 56px;
  right: 0;
  background: rgba(0, 0, 0, 0.15);
  z-index: 1;
  display: flex;
  align-items: center;
  justify-content: center;
}

```

```
.mat-flat-button {  
  margin: 5px;  
}
```

13.3. Добавление продуктов

Для того чтобы создать форму для добавления продукта, откройте и отредактируйте `src/app/product-add/product-add.component.ts`.

```
// Добавление нового продукта  
import { Component, OnInit } from '@angular/core';  
import { Router } from '@angular/router';  
import { ApiService } from '../api.service';  
import { FormControl, FormGroupDirective, FormBuilder, FormGroup,  
  NgForm, Validators } from '@angular/forms';  
import { ErrorStateMatcher } from '@angular/material/core';  
export class MyErrorStateMatcher implements ErrorStateMatcher {  
  isErrorState(control: FormControl | null, form: FormGroupDirective |  
    NgForm | null): boolean {  
    const isSubmitted = form && form.submitted;  
    return !(control && control.invalid && (control.dirty || control.  
      touched || isSubmitted));  
  }  
}  
  
@Component({  
  selector: 'app-product-add',  
  templateUrl: './product-add.component.html',  
  styleUrls: ['./product-add.component.css']  
})  
export class ProductAddComponent implements OnInit {  
  productForm: FormGroup; // Форма для ввода полей  
  prod_name = '';  
  prod_desc = '';  
  prod_price: number = null;  
  updated_at: Date = new Date();  
  isLoadingResults = false;  
  matcher = new MyErrorStateMatcher();  
  constructor(private router: Router, private api: ApiService, private  
    formBuilder: FormBuilder) { }  
  ngOnInit() {  
    // Создание формы ввода  
    this.productForm = this.formBuilder.group({
```

```

    id: [null],
    prod_name : [null, Validators.required],
    prod_desc : [null, Validators.required],
    prod_price : ['00', Validators.required],
    updated_at : [this.updated_at, Validators.required]
  });
}
onFormSubmit() {
  // Сохранение продукта через сервис
  this.isLoadingResults = true;
  this.api.addProduct(this.productForm.value)
    .subscribe((res: any) => {
      console.log('res = ' + res);
      // const id = res._id;
      this.isLoadingResults = false;
      console.log('addProduct')
      // this.router.navigate(['/product-details', id]);
    }, (err: any) => {
      console.log(err);
      this.isLoadingResults = false;
    });
}
}
}

```

Далее, откройте и отредактируйте файл `src/app/product-add/product-add.component.html`:

```

<!--Окно для ввода данных о продукте-->
<div class="example-container mat-elevation-z8">
  <div class="example-loading-shade"
    *ngIf="isLoadingResults">
    <mat-spinner *ngIf="isLoadingResults"></mat-spinner>
  </div>
  <div class="button-row">
    <a mat-flat-button color="primary"
      [routerLink]="['/products']"><mat-icon>list</mat-icon></a>
  </div>
  <mat-card class="example-card">
    <form [formGroup]="productForm" (ngSubmit)="onFormSubmit()">
      <mat-form-field class="example-full-width">
        <input matInput placeholder="Название"
          FormControlName="prod_name"
          [errorStateMatcher]="matcher">
        <mat-error>
          <span *ngIf="!productForm.get('prod_name').valid &&

```

```

        productForm.get('prod_name').touched">Please enter Product
        Name</span>
      </mat-error>
    </mat-form-field>
    <mat-form-field class="example-full-width">
      <input matInput placeholder="Описание" formControlName=
        "prod_desc"
        [errorStateMatcher]="matcher">
      <mat-error>
        <span *ngIf="!productForm.get('prod_desc').valid &&
        productForm.get('prod_desc').touched">Please enter
        Product Description</span>
      </mat-error>
    </mat-form-field>
    <mat-form-field class="example-full-width">
      <input matInput placeholder="Цена" formControlName=
        "prod_price"
        [errorStateMatcher]="matcher">
      <mat-error>
        <span *ngIf="!productForm.get('prod_price').valid &&
        productForm.get('prod_price').touched">Please enter
        Product Price</span>
      </mat-error>
    </mat-form-field>
    <!--Ввод даты с использованием диалога-->
    <mat-form-field>
      <input matInput [matDatepicker]="pickerTo" placeholder="Дата"
        formControlName="updated_at">
      <mat-datepicker-toggle matSuffix [for]="pickerTo"></
        mat-datepicker-toggle>
      <mat-datepicker #pickerTo></mat-datepicker>
    </mat-form-field>
    <div class="button-row">
      <button type="submit" [disabled]="!productForm.valid"
        mat-flat-button color="primary"><mat-icon>save</mat-icon></button>
    </div>
  </form>
</mat-card>
</div>

```

Наконец, откройте и отредактируйте `src/app/product-add/product-add.component.scss`:

```

.example-container {
  position: relative;

```



```
padding: 5px;
}
.example-form {
  min-width: 150px;
  max-width: 500px;
  width: 100%;
}
.example-full-width {
  width: 100%;
}
.example-full-width:nth-last-child(0) {
  margin-bottom: 10px;
}
.button-row {
  margin: 10px 0;
}
.mat-flat-button {
  margin: 5px;
}
```

13.4. Редактирование продуктов

Мы поместили кнопку редактирования внутри компонента сведений о продукте для вызова страницы редактирования. Теперь откройте и редактируйте `src/app/product-edit/product-edit.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { ApiService } from '../api.service';
import { FormControl, FormGroupDirective, FormBuilder, FormGroup,
  NgForm, Validators } from '@angular/forms';
import { ErrorStateMatcher } from '@angular/material';
import { formatCurrency, formatDate } from '@angular/common';
export class MyErrorStateMatcher implements ErrorStateMatcher {
  isErrorState(control: FormControl | null, form: FormGroupDirective |
    NgForm | null): boolean {
    const isSubmitted = form && form.submitted;
    return !!(control && control.invalid && (control.dirty ||
      control.touched || isSubmitted));
  }
}
```

```
@Component({
  selector: 'app-product-edit',
  templateUrl: './product-edit.component.html',
  styleUrls: ['./product-edit.component.css']
})
export class ProductEditComponent implements OnInit {
  productForm: FormGroup; // Форма для ввода данных
  // id = 1;
  // tslint:disable-next-line:variable-name
  prodName = '';
  // tslint:disable-next-line:variable-name
  prodDesc = '';
  // tslint:disable-next-line:variable-name
  prod_price: number = null;
  // tslint:disable-next-line:variable-name
  updatedAt: Date = null;
  isLoadingResults = false;
  matcher = new MyErrorStateMatcher();
  id: number;
  constructor(private router: Router, private route: ActivatedRoute,
    private api: ApiService, private FormBuilder: FormBuilder) { }
  getProduct(id: any) {
    // Запрос продукта у сервиса
    console.log('getProduct id= ' + id);
    this.api.getProduct(id).subscribe((data: any) => {
      this.id = data.id;
      // Заполнение формы ввода данных
      this.productForm.setValue({
        prod_name: data.prod_name,
        prod_desc: data.prod_desc,
        prod_price: formatCurrency(data.prod_price, 'en-EN', '', 'RUB'),
        updated_at: formatDate(data.updated_at, 'yyyy-MM-dd', 'en-EN' )
      });
      console.log('data.prodName = ' + data.prod_name);
    });
  }
  onFormSubmit() {
    // Прием данных с заполненной формы добавления продукта
    console.log(this.productForm.value);
    this.prod_price = this.productForm.get('prod_price').value;
    // Приведение формата для цены
    this.productForm.patchValue({prod_price: this.prod_price});
    // console.log('onFormSubmit product price = ' + this.productForm.
      get('prodPrice').value);
    // console.log('onFormSubmit product updated = ' + this.productForm.
```

```

        get('updatedAt').value);
    console.log('this.prodPrice = ' + this.prod_price);
    console.log(this.productForm.value);
    this.isLoadingResults = true;
    // Сохранение данных на сервисе
    this.api.updateProduct(this.id, this.productForm.value)
        .subscribe((res: any) => {
            // const id = res.id;
            this.isLoadingResults = false;
            this.router.navigate(['/product-details', this.id]);
            console.log('onFormSubmit id= ' + this.id);
        }, (err: any) => {
            console.log(err);
            this.isLoadingResults = false;
        })
    );
}
productDetails() {
    this.router.navigate(['/product-details', this.id]);
}
ngOnInit() {
    this.route.params.subscribe(params => this.id = params['id']);
    console.log('Edit OnInit id = ' + this.id);
    this.getProduct(this.id);
    // this.getProduct(this.route.snapshot.params[this.id]);
    // Создание формы ввода данных
    this.productForm = this.formBuilder.group({
        prod_name : [null, Validators.required],
        prod_desc : [null, Validators.required],
        prod_price : [null, Validators.required],
        updated_at : [null, Validators.required]
    });
}
}

```

Далее, откройте и отредактируйте файл `src/app/product-edit/product-edit.component.html`:

```

<!--Страница для редактирования записи о продукте-->
<div class="example-container mat-elevation-z8">
    <div class="example-loading-shade"
        *ngIf="isLoadingResults">
        <mat-spinner *ngIf="isLoadingResults"></mat-spinner>
    </div>
    <div class="button-row">

```

```
<a mat-flat-button color="primary" (click)="productDetails()">
  <mat-icon>info</mat-icon></a>
</div>
<!-- Форма ввода-->
<mat-card class="example-card">
  <form [formGroup]="productForm" (ngSubmit)="onFormSubmit()">
    <mat-form-field class="example-full-width">
      <input matInput placeholder="Название"
        formControlName="prod_name"
        [errorStateMatcher]="matcher">
      <mat-error>

        <span *ngIf="!productForm.get('prod_name').valid &&
          productForm.get('prod_name').touched">Please enter
          Product Name</span>
      </mat-error>
    </mat-form-field>
    <mat-form-field class="example-full-width">
      <input matInput placeholder="Описание" formControlName=
        "prod_desc"
        [errorStateMatcher]="matcher">
      <mat-error>
        <span *ngIf="!productForm.get('prod_desc').valid &&
          productForm.get('prod_desc').touched">Please enter
          Product Description</span>
      </mat-error>
    </mat-form-field>
    <mat-form-field class="example-full-width">
      <input matInput placeholder="Цена" formControlName="prod_price"
        [errorStateMatcher]="matcher">
      <mat-error>
        <span *ngIf="!productForm.get('prod_price').valid &&
          productForm.get('prod_price').touched">Please enter
          Product Price</span>
      </mat-error>
    </mat-form-field>
    <!--Ввод даты в стандартном окне-->
    <mat-form-field>
      <input matInput [matDatepicker]="pickerTo" placeholder=
        "Дата" formControlName="updated_at">
      <mat-datepicker-toggle matSuffix [for]="pickerTo"></mat-datepicker-
toggle>
      <mat-datepicker #pickerTo></mat-datepicker>
    </mat-form-field>
  </div class="button-row">
```

```

        <button type="submit" [disabled]="!productForm.valid"
        mat-flat-button color="primary"><mat-icon>Сохранить</mat-icon>
        </button>
    </div>
</form>
</mat-card>
</div>

```

13.5. Сервисы для получения продуктов

Для получения, добавления и модификации записей БД нужен REST-сервис, разработанный на платформе Spring Boot. Архитектура сервиса аналогична приведенной в главе 4. Ниже приведен файл `ProductController.java` с классом, реализующий необходимые приложению Angular сервисы:

```

package com.product.restproduct.controller;
import com.product.restproduct.exception.ResourceNotFoundException;
import com.product.restproduct.model.Product;
import com.product.restproduct.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.*;
import javax.validation.Valid;
import java.text.DateFormat;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.*;
@RestController
public class ProductController {
    @Autowired
    private ProductRepository productRepository;
    @GetMapping("/products")
    public List<Product> getProducts() {

```

```
        return productRepository.findAll();
    }
    @GetMapping("/products/{id}")
    public Product getProductById(@PathVariable Long id) {
        Optional<Product> product1 = productRepository.findById(id);
        return product1.orElseThrow(() -> new
            ResourceNotFoundException("Product not found. Id = " + id));
    }
    @GetMapping("/products/interval")
    // get with query Date parameters e.g. /products/interval?fromD=
    // "dd.MM.yyyy"&toD="dd.MM.yyyy"
    // @ApiOperation(value = "Get the Products by fromD and toD")
    public List<Product> getProductByInterval(@RequestParam("fromD")
        String fromD, @RequestParam("toD") String toD) {
        Date dfrom = new Date();
        Date dto = new Date();
        System.out.println("from D= " + fromD + " toD = " + toD);
        SimpleDateFormat formatter1 = new SimpleDateFormat("dd.MM.yyyy",
            Locale.ENGLISH);
        SimpleDateFormat formatter2 = new SimpleDateFormat("dd.MM.yyyy",
            Locale.ENGLISH);
        try {
            dfrom = formatter1.parse(fromD);
            dto = formatter2.parse(toD);
        } catch (Exception e) {
            System.out.println("Exception = " + e.getMessage() +
                " from = " + dfrom + " to = " + dto);
        };
        System.out.println("from = " + dfrom + " to = " + dto);
        List<Product> products = productRepository.findByDateInterval(
            dfrom, dto);
        if (products.size() > 0){
            return products;
        }
        throw new ResourceNotFoundException("Products not found.
            from = " + fromD + " to = " + toD);
    }
    @PostMapping("/products")
    public Product createProduct(@Valid @RequestBody Product product) {
        return productRepository.save(product);
    }
    @PutMapping("/products/{productId}")
    public Product updateProduct(@PathVariable Long productId,
        @Valid @RequestBody Product productRequest) throws ParseException {
        // DateFormat df = new SimpleDateFormat("yyyy-MM-dd ");
    }
```

```

        Date updated_at =productRequest.getUpdated_at();
        // Date updated_at = df.parse(productRequest.getUpdated_at().
        toString());
        //System.out.println("product_save getProd_name =
        " + productRequest.getProd_name());
        //System.out.println("product_save getUpdated_at() =
        " + updated_at);
        //System.out.println("product_save price = " + productRequest.
        getProd_price());
        Double price = productRequest.getProd_price();
        // Double price = Double.parseDouble((new Formatter()).
        format("%.2f",productRequest.getProd_price()).toString());
        return productRepository.findById(productId)
            .map(product -> {
                product.setProd_name(productRequest.getProd_name());
                product.setProd_desc(productRequest.getProd_desc());
                product.setProd_price(price);
                product.setUpdated_at(updated_at);
                //System.out.println("product_saved =
                " + product.getProd_name());
                return productRepository.save(product);
            }).orElseThrow(() -> new ResourceNotFoundException
            ("Product not found with id " + productId));
    }
    @DeleteMapping("/products/{productId}")
    public ResponseEntity<?> deleteProduct(@PathVariable Long
    productId) {
        return productRepository.findById(productId)
            .map(product -> {
                productRepository.delete(product);
                return ResponseEntity.ok().build();
            }).orElseThrow(() -> new ResourceNotFoundException
            ("Product not found with id " + productId));
    }
}

```

Далее приведен файл с описанием сущности Product.java:

```

package com.product.restproduct.model;
import javax.persistence.*;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Size;
import java.io.Serializable;
import java.util.Date;
@Entity

```

```
@Table(name = "product")
public class Product implements Serializable {
    //private static final long serialVersionUID =
        -300915773224241606L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;
    @Column(name = "prod_name")
    @NotBlank
    @Size(min = 3, max = 50)
    private String prod_name;
    @Column(name = "prod_desc")
    @Size(min = 3, max = 255)
    private String prod_desc;
    @Column(name = "prod_price")
    private double prod_price;
    @Column(name = "updated_at")
    private Date updated_at;
    public Long getId() {
        return id;
    }
    public void setId(Long id) { this.id = id; }
    public String getProd_name() {
        return prod_name;
    }
    public void setProd_name(String prod_name) {
        this.prod_name = prod_name;
    }
    public String getProd_desc() {
        return prod_desc;
    }
    public void setProd_desc(String prod_desc) {
        this.prod_desc = prod_desc;
    }
    public double getProd_price() {
        return prod_price;
    }
    public void setProd_price(double prod_price) {
        this.prod_price = prod_price;
    }
    public Date getUpdated_at() {
        return updated_at;
    }
    public void setUpdated_at(Date updated_at) {
```



```
        this.updated_at = updated_at;
    }
}
```

Приведем сценарий создания таблицы продуктов в СУБД PostgreSQL:

```
CREATE TABLE public.product
(
    id integer NOT NULL DEFAULT nextval('product_id_seq'::regclass),
    prod_name character varying(50) NOT NULL,
    prod_desc character varying(255),
    prod_price double precision NOT NULL,
    updated_at date NOT NULL DEFAULT '2019-01-01'::date,
    CONSTRAINT "PK_Products" PRIMARY KEY (id)
)
WITH (
    OIDS=FALSE
);
ALTER TABLE public.product
    OWNER TO postgres;
```

Контрольные вопросы

1. Зачем нужен элемент RouterOutlet?
2. Как влияет порядок указания маршрутов на визуализацию?
3. Что делает атрибут routerLink="путь"?
4. Что делает фрагмент кода this.router.navigate(['путь'])?

Заключение

Материал пособия позволяет студентам на лабораторных занятиях, в курсовом и дипломном проектировании создавать простые приложения для обслуживания баз данных, содержащих справочные и связанные с ними внешними ключами таблицы. Более полное изложение концепций платформ *Spring*, *Spring Boot* и *Angular*, пришедшим на смену классическим JSP и JSF, можно найти в оригинальных руководствах в сети.

Приведенный материал позволит студенту самостоятельно разобратся в основах мультисервисных архитектур веб-приложений и приступить к выполнению проекта по дисциплинам «Клиент-серверные технологии», «Разработка веб-приложений» и «Разработка веб-сервисов», а также выполнить задание на выпускную работу с использованием современных платформ и технологий на языке программирования *Java*.

Библиографический список

1. Çivici, Çağatay. PrimeFaces user guide 6.2 / Çağatay Çivici. — 664 p. — URL: https://www.primefaces.org/docs/guide/primefaces_user_guide_6_2.pdf (date of access: 22.08.2022).
2. JSF — PrimeFaces Hello World Example using Jetty and Maven // Codenotfound : [Website]. — 2014. — 12th april. — URL: <https://codenotfound.com/jsfprimefaces-hello-world-example-jetty-maven.html> (date of access: 22.08.2022).
3. Krebs Bruno. Developing JSF applications with Spring Booth / Bruno Krebs // Auth0 blog : [Website]. — 2018. — 9th may. — URL: <https://auth0.com/blog/developing-jsf-applications-withspring-boot/> (date of access: 22.08.2022).
4. Spring Boot Connect to PostgreSQL Database Examples // CodeJava : [Website]. — URL: <https://www.codejava.net/frameworks/springboot/connect-to-postgresql-database-examples> (date of access: 22.08.2022).
5. Spring Boot Reference Documentation / Phillip Webb, Dave Syer, Josh Long [et al.] // Spring : [Website]. — 2022. — 18th august. — URL: <https://docs.spring.io/springboot/docs/current/reference/html/> (date of access: 22.08.2022).
6. Spring Boot, PostgreSQL, JPA, Hibernate RESTful CRUD API Example // Callicoder : [Website]. — URL: <https://www.callicoder.com/spring-boot-jpa-hibernate-postgresql-restful-crud-apiexample/> (date of access: 22.08.2022).
7. Spring Web MVC // Spring : [Website]. — URL: <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html> (date of access: 22.08.2022).

8. Spring Framework : учебник // Викиучебник: открытые книги для открытого мира. — URL: https://ru.wikibooks.org/wiki/Spring_Framework (дата обращения: 22.08.2022).
9. Spring MVC — Hello World, пример приложения. Передача параметров формы // Javastudy : [Сайт]. — 2017. — 31 мая. — URL: <http://javastudy.ru/spring-mvc/hello-world-example> (дата обращения: 22.08.2022).
10. Давыдов С. И. IntelliJ IDEA. Профессиональное программирование на Java / С. И. Давыдов, А. А. Ефремов. — Санкт-Петербург : БХВ-Петербург, 2005. — 800 с. — ISBN 978-5-94157-607-4.
11. Руководство по Angular 9 // METANIT.COM: Сайт о программировании : [Сайт]. — 2020. — 15 февраля. — URL: <https://metanit.com/web/angular2/> (дата обращения: 22.08.2022).
12. Руководство по Spring (полная версия) // PROSELYTE. Software Engineering : [Сайт]. — URL: <http://proselyte.net/spring-tutorial-full-version/> (дата обращения: 22.08.2022).

Приложение 1

Краткое описание комбинаций клавиш в IntelliJ IDEA

Редактирование

Ctrl + Z	Undo, отменить последнее действие
Ctrl + Shift + Z	Redo, отменить последнюю отмену действия
Ctrl + Shift + V	Расширенная вставка из буфера обмена (с историей)
Ctrl (+ Shift) + W	Инкрементальное выделение выражения
Ctrl + влево/вправо	Перемещение между словами
Ctrl + вверх/вниз	Прокрутка кода без изменения позиции курсора
Ctrl + Home/End	Переход в начало или конец файла
Shift + Del (Ctrl + Y)	Удаление строки, отличие в том, где потом окажется курсор
Ctrl + Del	Удалить от текущей позиции до конца слова
Ctrl + Backspace	Удалить от текущей позиции до начала слова
Ctrl + D	Дублировать текущую строку
Tab/Shift + Tab	Увеличить или уменьшить текущий отступ
Ctrl + Alt + I	Выравнивание отступов в коде
Ctrl + Alt + L	Приведение кода в соответствие с code style
Ctrl + /	Закомментировать или раскомментировать текущую строку
Ctrl + Shift + /	Закомментировать или раскомментировать выделенный код
Ctrl + -/+	Фолдинг, свернуть или развернуть
Ctrl + Shift + -/+	Фолдинг, свернуть или развернуть все
Ctrl + Shift + .	Сделать текущий скоуп сворачиваемым и свернуть его
Ctrl + .	Сделать текущий скоуп несворачиваемым
Ctrl + R	Замена в тексте
Ctrl + Shift + R	Замена во всех файлах

Окна, вкладки

Alt + влево/вправо	Перемещение между вкладками
Ctrl + F4	Закрыть вкладку
Alt + циферка	Открытие/закрытие окон Project, Structure, Changes и тд
Ctrl + Tab	Switcher, переключение между вкладками и окнами
Shift + Esc	Закрыть активное окно
F12	Открыть последнее закрытое окно
Ctrl + колесико	Zoom, если он был вами настроен

Закладки

F11	Поставить или снять закладку
Ctrl + F11	Аналогично с присвоением буквы или цифры
Shift + F11	Переход к закладке (удаление с помощью клавиши Delete)
Ctrl + Число	Быстрый переход к закладке с присвоенным числом

Подсказки и документация

Ctrl + Q	Документация к тому, на чём сейчас курсор
Ctrl + Shift + I	Показать реализацию метода или класса
Alt + Q	Отобразить имя класса или метода, в котором мы находимся
Ctrl + P	Подсказка по аргументам метода
Ctrl + F1	Показать описание ошибки или варнинга
Alt + Enter	Показать, что нам предлагают «лампочки»

Поиск

Дважды Shift	Быстрый поиск по всему проекту
Ctrl + Shift + A	Быстрый поиск по настройкам, действиям и т.д.
Alt + вниз/вверх	Перейти к следующему или предыдущему методу
Ctrl + [и Ctrl +]	Перемещение к началу и концу текущего скоупа
Ctrl + F	Поиск в файле
Ctrl + Shift + F	Поиск по всем файлам (переход — F4)
Ctrl + F3	Искать слово под курсором
F3/Shift + F3	Искать вперед или назад
Ctrl + G	Переход к строке или строке: номеру_символа
Ctrl + F12	Список методов с переходом к их объявлению

Ctrl + E	Список недавно открытых файлов с переходом к ним
Ctrl + Shift + E	Список недавно измененных файлов с переходом к ним
Ctrl + H	Иерархия наследования текущего класса и переход по ней
Ctrl + Alt + H	Иерархия вызовов выбранного метода
Ctrl + N	Поиск класса по имени и переход к нему
Ctrl + Shift + N	Поиск файла по имени и переход к нему
Ctrl + B	Перейти к объявлению переменной, класса, метода
Ctrl + Alt + B	Перейти к реализации
Ctrl + Shift + B	Определить тип и перейти к его реализации
Shift + Alt + влево	Перемещение назад по стеку поиска
Shift + Alt + вправо	Перемещение вперед по стеку поиска
F2/Shift + F2	Переход к следующей или предыдущей ошибке
Shift + Alt + 7	Найти все места, где используется метод или переменная
Ctrl + Alt + 7	Так же как предыдущий пункт, только во всплывающем окне

Генерация кода и рефакторинг

Ctrl + Space	Полный автокомплит
Ctrl + Shift + Space	Автокомплит с фильтрацией по подходящему типу
Alt + /	Простой автокомплит по словам, встречающимся в проекте
Ctrl + I	Реализовать интерфейс
Ctrl + O	Переопределить метод родительского класса
Ctrl + J	Генерация шаблонного кода (обход по итератору и т. д.)
Ctrl + Alt + J	Обернуть выделенный код в один из шаблонов
Alt + Insert	Генератор кода — сеттеров, зависимостей в pom.xml и т. д.
Shift + F6	Переименование переменной, класса и т. д. во всем коде
Ctrl + F6	Изменение сигнатуры метода во всем коде
F6	Перемещение метода, класса или пакета
F5	Создать копию класса, файла или каталога
Shift + F5	Создать копию класса в том же пакете
Alt + Delete	Безопасное удаление класса, метода или атрибута
Ctrl + Alt + M	Выделение метода
Ctrl + Alt + V	Выделение переменной
Ctrl + Alt + F	Выделение атрибута
Ctrl + Alt + C	Выделение константы (public final static)
Ctrl + Alt + P	Выделение аргумента метода
Ctrl + Alt + N	Инлайнинг метода, переменной, аргумента или константы
Ctrl + Alt + O	Оптимизация импортов

Приложение 2

Основные аннотации Java Spring

@Autowired — внедрение зависимости. В поле, помеченное этой аннотацией, фреймворк подставит компонент соответствующего типа. Если компонентов заданного типа несколько, нужно указать название необходимого компонента. Аннотация **@Autowired** может быть указана к сеттеру, конструктору или методу. Тогда внедрение зависимости происходит в параметры этих функций.

@Component — аннотация указывает на то, что класс должен управляться контейнером Spring.

@Repository, **@Service** и **@Controller** — специальные виды компонентов:

- **@Repository** — аннотация указывает на то, что класс выполняет роль репозитория данных;
- **@Service** — компонент уровня бизнес-логики;

@Controller — контроллер или компонент уровня приложения.

@Bean — помечает методы, ответственные за создание компонентов.

@SpringBootApplication — дополнительная аннотация Spring Boot, предназначенная для обозначения главного класса приложения и включающая в себя три аннотации Spring:

- **@Configuration** — класс содержит описание методов, отвечающих за создание компонентов;
- **@EnableAutoConfiguration** — указывает фреймворку на то, что надо автоматически включить инструменты создания компонентов, которые расположены в Class Path;
- **@ComponentScan** — конфигурирует поиск компонентов, которые необходимо включить в приложение.

@RequestMapping — аннотация, описывающая маршруты, имеющая два основных параметра:

- ***method*** — http-метод;
- ***value*** — шаблон маршрута.

Фрагменты маршрута, являющиеся параметрами, заключаются в фигурные скобки, например:


```
@RequestMapping(method = RequestMethod.GET,  
    value = "/students/{id}")  
public String student(@PathVariable Long id, Model vars) {  
    ...  
}
```

@RequestParam — параметр запроса.

@PathVariable — параметр запроса, извлеченный из пути.

Оглавление

Список сокращений и определений	3
Предисловие	4
Введение	6
1. Платформа Spring	8
1.1. Контейнеры Spring. Область видимости и жизненный цикл бинов	15
1.2. Конфигурирование с помощью аннотаций и кода Java	18
Контрольные вопросы	18
2. Фреймворк Spring-MVC	19
2.1. DispatcherServlet	19
2.2. Разработка контроллера	20
2.3. Создание просмотра	22
2.4. Разработка Spring MVC веб-приложений в IntelliJ IDEA.....	23
Контрольные вопросы	33
3. Фреймворк Spring Boot	34
3.1. Введение в Spring Boot	34
3.2. Структура кода приложения.....	38
3.3. Запуск приложения.....	41
3.4. Работы с базами данных SQL	43
Контрольные вопросы	47
4. Разработка приложения с СУБД PostgreSQL	48
Контрольные вопросы	62

5. Проверка данных	63
5.1. Создание объекта PersonForm	64
5.2. Создание веб-контроллера	65
5.3. Страницы для пользователя	67
5.4. Создание класса Application	68
Контрольные вопросы	69
 6. Микросервисная архитектура приложений.....	70
Платформа Angular	71
Контрольные вопросы	79
 7. Принципы работы приложений Angular	80
7.1. Компоненты	80
7.2. Модули	82
7.3. Загрузка приложения	84
7.4. Стили и шаблоны компонента	85
7.5. Привязки	86
7.6. Работа с компонентами	92
7.7. Взаимодействие между компонентами	95
7.8. Привязка к событиям дочернего компонента	97
7.9. Жизненный цикл компонента	100
Контрольные вопросы	109
 8. Директивы.....	110
8.1. Структурные директивы	110
8.2. Директивы атрибутов.....	114
Контрольные вопросы и задание.....	116
 9. Сервисы и инъекции зависимостей	117
Внедрение сервиса в другой сервис	120
Контрольные вопросы	122
 10. Работа с формами	123
Поток данных в формах	126
Контрольные вопросы	130
 11. HTTP и взаимодействие с сервером	131
11.1 HttpClient и отправка запросов.....	131
11.2. Обработка ошибок	132
11.3. Отправка данных в запросе.....	133
Контрольные вопросы	134

12. Маршрутизация	135
12.1. Определение маршрутов	135
12.2. Создание ссылок	140
12.3. Программная навигация	142
Контрольные вопросы	143
13. Angular Material	144
13.1. Показ списка продуктов	148
13.2. Показ деталей продуктов	153
13.3. Добавление продуктов	156
13.4. Редактирование продуктов	159
13.5. Сервисы для получения продуктов	163
Контрольные вопросы	167
Заключение	168
Библиографический список	169
Приложение 1	171
Краткое описание комбинаций клавиш в IntelliJ IDEA	171
Приложение 2	174
Основные аннотации Java Spring	174

Учебное издание

Суханов Владимир Иванович

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ
НА ПЛАТФОРМЕ SPRING**

Редактор И. В. Коршунова
Корректор Ю. М. Ананченко
Верстка О. П. Игнатевой

Подписано в печать 31.10.2023. Формат 70×100/16.
Бумага офсетная. Цифровая печать. Усл. печ. л. 14,5.
Уч.-изд. л. 9,4. Тираж 30 экз. Заказ 76.

Издательство Уральского университета
Редакционно-издательский отдел ИПЦ УрФУ
620049, Екатеринбург, ул. С. Ковалевской, 5
Тел.: +7 (343) 375-48-25, 375-46-85, 374-19-41
E-mail: rio@urfu.ru

Отпечатано в Издательско-полиграфическом центре УрФУ
620083, Екатеринбург, ул. Тургенева, 4
Тел.: +7 (343) 358-93-06, 350-58-20, 350-90-13
Факс: +7 (343) 358-93-06
<http://print.urfu.ru>

